

Hybrid Obfuscated Key Exchange and KEMs

Felix Günther 
IBM Research Europe – Zurich
mail@felixguenther.info

Michael Rosenberg 
Cloudflare
research@mrosenberg.pub

Douglas Stebila 
University of Waterloo
dstebila@uwaterloo.ca

Shannon Veitch 
ETH Zurich
shannon.veitch@inf.ethz.ch

Version 1.1, August 16, 2025

Abstract

Hiding the metadata in Internet protocols serves to protect user privacy, dissuade traffic analysis, and prevent network ossification. *Fully encrypted protocols* require even the initial key exchange to be obfuscated: a passive observer should be unable to distinguish a protocol execution from an exchange of random bitstrings. Deployed obfuscated key exchanges such as Tor’s pluggable transport protocol **obfs4** are Diffie–Hellman-based, and rely on the Elligator encoding for obfuscation. Recently, Günther, Stebila, and Veitch (CCS ’24) proposed a post-quantum variant **pq-obfs**, using a novel building block called *obfuscated key encapsulation mechanisms* (OKEMs): KEMs whose public keys and ciphertexts look like random bitstrings.

For transitioning real-world protocols, pure post-quantum security is not enough. Many are taking a *hybrid* approach, combining traditional and post-quantum schemes to hedge against security failures in either component. While hybrid KEMs are already widely deployed (e.g., in TLS 1.3), existing hybridization techniques fail to provide hybrid obfuscation guarantees for OKEMs. Further, even if a hybrid OKEM existed, the **pq-obfs** protocol would still not achieve hybrid obfuscation.

In this work, we address these challenges by presenting the first OKEM combiner that achieves hybrid IND-CCA security with hybrid ciphertext obfuscation guarantees, and using this to build **Drive1**, a modification of **pq-obfs** that is compatible with hybrid OKEMs. Our OKEM combiner allows for a variety of practical instantiations, e.g., combining obfuscated versions of DHKEM and ML-KEM. We additionally provide techniques to achieve unconditional public key obfuscation for LWE-based OKEMs, and explore broader applications of hybrid OKEMs, including a construction of the first hybrid password-authenticated key exchange (PAKE) protocol secure against adaptive corruptions in the UC model.

Keywords: KEM combiners, hybrid, obfuscation, anonymity, key exchange, ML-KEM, quantum-safe

Contents

1	Introduction	3
1.1	Post-quantum Obfuscated Key Exchange and Beyond	3
1.2	Barriers to Hybrid Obfuscated Key Exchange	4
1.3	Contributions	4
2	Preliminaries	6
2.1	Obfuscated KEMs	7
3	OEINC: An OKEM Combiner	10
3.1	Overview of Security	11
3.2	IND-CPA and IND-CCA Security	12
3.3	SPR-CCA Security	13
3.4	Ciphertext Uniformity	17
3.5	Strong Second Pseudorandomness	19
3.6	Public Key Uniformity	19
3.7	Instantiating the Combiner	20
4	A Hybrid Obfuscated Key Exchange Protocol	21
4.1	Shortcomings of the <code>pq-obfs</code> Protocol	22
4.2	The <code>Drive1</code> Protocol	23
4.3	Security	25
4.4	Additional Features	27
5	Hybrid PAKE with Adaptive Security	28
5.1	CAKE	29
5.2	Achieving Hybrid CAKE	30
6	Future Work	31
A	Additional Definitions	40
B	Relations Between Anonymity Notions	41
C	Security Proofs for OEINC	43
C.1	IND-CPA / IND-CCA Security	43
C.2	Strong Second Pseudorandomness	44
C.3	Public Key Uniformity	47
D	An OKEM from DHKEM	47
E	Obfuscated Key Exchange Security Model	50
E.1	Session and Game Variables	50
E.2	Session Identifiers, Contributive Identifiers, and Partnering	51
E.3	Security Definition	51
E.4	Single-challenge Selective Security	51
F	Security Proof for <code>Drive1</code> (Theorem 4.1)	51
G	Changelog	62

1 Introduction

Increasingly, Internet protocols are hiding not just message contents, but the protocol metadata itself, due to metadata being repeatedly leveraged to violate user security and privacy, and to degrade connectivity [Hal24, XAR⁺24, WMSM11, And12]. As a pertinent example, network censors often block traffic based on certain identifiable (plaintext) features of the protocol [WSS⁺23]. To evade this form of censorship, circumvention tools often encrypt metadata to avoid protocol fingerprinting and blocklists [The19, sha23, vme19]. These *fully encrypted protocols* [FJ24], such as the **obfs4** protocol integrated in the Tor ecosystem [The19], produce a stream of random-looking bytes intended to avoid classification. Such streams can also be used in steganography due to their lack of structure [WJJS23]. Another motivation for encrypting protocol metadata, e.g., in pseudorandom cTLS [SP22], is to prevent *network ossification*, whereby network middleboxes expect a specific protocol mode (such as TLS 1.2) and fail to parse and route traffic of newer protocol versions (such as TLS 1.3 without its intentional 1.2 compatibility). If protocol metadata appears to a middlebox as a random string of bytes, then no parsers can be written in the first place. Random-looking metadata as an anti-ossification tool is already widely deployed, appearing in the QUIC protocol [IT21], where elements of the initial packet are encrypted to make the wire image of the protocol closer to pseudorandom, and in TLS Encrypted Client Hello [ROSW24], where the server “accept” response is encoded as a truncated MAC and hidden in the randomness sent from server to client. Both these techniques are also employed to improve privacy guarantees.

Recent formalizations capture the guarantees provided by fully encrypted protocols in the context of censorship circumvention [FJ24, GSV24]. A shared secret key allows encrypting (meta)data into random-looking strings of bytes. However, it is less obvious how to make the initial key exchange of a connection look random—this is what *obfuscated key exchange* protocols aim to achieve.

1.1 Post-quantum Obfuscated Key Exchange and Beyond

As timelines for cryptographically relevant quantum computers shorten, there is an increased need to update current obfuscated key exchange protocols which derive their security from elliptic curve Diffie–Hellman. Recent work [GSV24] proposes a quantum-safe variant of **obfs4** called **pq-obfs**. The construction relies on the newly defined *obfuscated key encapsulation mechanism* (OKEM): a KEM that supports random-looking encoding of public keys and ciphertexts. That work shows how to construct an efficient OKEM from the Module Learning with Errors (MLWE) assumption by introducing the **Kameleon** encoding which—akin to the **Elligator2** encoding for elliptic curve Diffie–Hellman [BHK13]—turns ML-KEM public keys and ciphertexts into random strings.

Due to caution over switching to newer assumptions, some governments [Ger24, Fre22] and cloud providers [Wes24] opt instead for *hybrid* constructions, i.e., constructions whose assumptions combine the hardness of two input primitives, usually one post-quantum and one well-established. The reasons for this approach are twofold. First, using two hardness assumptions hedges risk of mathematical advances, since the resulting construction is secure even if one of the underlying assumptions fails, as happened with NIST PQC Round 3 alternates and a finalist [BBC⁺22, Beu22, CD23]. Second, it hedges risk of implementation errors, since newer post-quantum implementations may be more prone to bugs [GJN20, GHJ⁺22, HSC⁺23, BBB⁺24]. For these reasons, hybrid protocols, such as the

X25519MLKEM768 key agreement protocol for TLS 1.3 have already been deployed to millions worldwide [Rad]. It would thus be ideal to migrate obfuscated key exchange protocols to post-quantum using a hybrid solution rather than a purely lattice-based one.

To date, the work on building hybrid KEMs and key exchange has focused primarily on IND-CCA security [GHP18, WW24, BCD⁺24, BBF⁺19]. And although it has already been argued that a broader study of security properties should be considered for post-quantum KEMs [GMP22], works exploring additional properties in the hybrid setting are very sparse [GHJ25]. To this end, we initiate a study of obfuscation and anonymity guarantees in the hybrid setting, towards constructing quantum-safe metadata-hiding protocols that are in line with modern techniques of hybridization.

1.2 Barriers to Hybrid Obfuscated Key Exchange

One might think that a straightforward approach to construct hybrid obfuscated key exchange is to instantiate `pq-obfs` with a hybrid OKEM combining two OKEMs using a standard KEM combiner. This however does not work, for two reasons.

First, there are no OKEM-specific hybridization methods, and the existing methods for KEM hybridization do not produce hybrid OKEMs. A KEM combiner takes as input two KEMs and produces a single, hybrid KEM whose security is maintained as long as one of the two input KEMs remains secure. Existing combiner techniques [GHP18, WW24, BCD⁺24, Con25] do not apply to the obfuscation properties of OKEMs, where the goal is to have public keys and ciphertexts indistinguishable from random strings. Take for example the Xyber hybrid KEM [WW24], which encapsulates by performing a Kyber encapsulation and X25519 key agreement in parallel, and where the final shared secret is a key derived from the two resulting shared secrets. While parallel combiners like this provide hybrid IND-CCA security, they do not provide hybrid obfuscation for OKEMs: if an adversary receives two concatenated public keys or ciphertexts, then they need only observe a non-uniformity in *one* of them to determine that the joint public key/ciphertext is not random. Most existing combiners are in this parallel style, depicted in Figure 1(a), which even when applied to OKEMs only provides obfuscation as strong as the *weakest* of the underlying obfuscation assumptions.

Second, even if there were an OKEM hybridization technique, there is still a hurdle in using it to make `pq-obfs` hybrid. `pq-obfs` requires its OKEM to have *public key uniformity*: public keys must appear uniformly random. Since a KEM public key must be available for encapsulation prior to any other communication, it appears the only way to represent the combined public key is as the concatenation of the underlying public keys, or something equivalent. As a consequence, constructing a combined OKEM with hybrid public key uniformity seems to require that the two underlying OKEMs have unconditional public key uniformity. Unconditional public key uniformity, however, is uncommon for post-quantum (O)KEMs, where public keys have structure that becomes distinguishable from random if the underlying hardness assumption breaks down. This bars the path to make `pq-obfs` into a hybrid obfuscated key exchange using standard components.

1.3 Contributions

We discuss how we overcome these limitations to achieve hybrid obfuscated key exchange, and introduce an additional application in hybrid password-authenticated key exchange (PAKE) protocols.

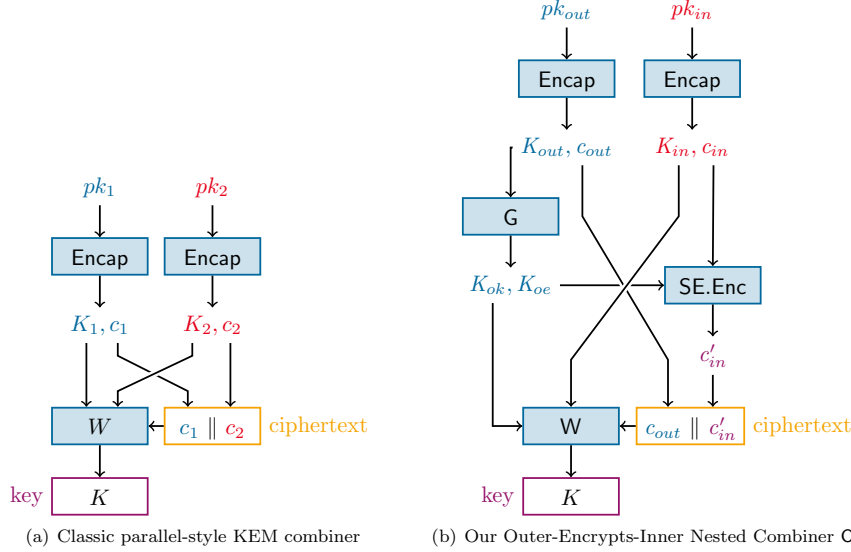


Figure 1: Structure of `Encap` in KEM Combiners. `W` is a split-key PRF, `G` is a pseudorandom generator, and `SE` is symmetric encryption.

OKEM combiner. Our first contribution is OEINC (“oink”), our *Outer-Encrypts-Inner Nested Combiner* for OKEMs which yields hybrid IND-CCA security and ciphertext obfuscation (without public key obfuscation) properties. The combiner is not fully generic, as it requires one of the component OKEMs to have statistically uniform ciphertexts. This allows us to use a nested approach in which we leverage the stronger OKEM as the *outer* one, using its shared secret towards encrypting the ciphertext of the weaker, *inner* OKEM.¹ Our nested approach is depicted in Figure 1(b). This provides hybrid guarantees for ciphertext uniformity and strong pseudorandomness (which in turn implies hybrid anonymity).

We show that the necessary statistical uniformity of ciphertexts can be achieved using the DHKEM construction from HPKE [BBLW22], over a prime-order curve such as P-256 [NIS23], with the `Elligator`² [Tib14] encoding algorithm, or HPKE over X25519 or Ristretto [LHT16] with the `Elligator2` [BHK13] encoding algorithm. This obfuscated variant of DHKEM can then be combined with a post-quantum OKEM, such as ML-Kem [GSV24], to achieve the desired hybrid guarantees. We use OEINC as a building block for the following applications.

Application: Hybrid obfuscated key exchange. Naturally, we would like to apply our OKEM combiner to obtain a hybrid obfuscated key exchange protocol, but, as noted above, hybrid `pq-obfs` would require hybrid public key uniformity, which OEINC does not achieve and generally seems elusive for hybrid OKEMs with post-quantum security.

We present `Drivel`, a modification to the `pq-obfs` protocol which permits the use of OKEMs *without* needing to obfuscate public keys. Our key insight is that public keys need not be sent in the clear, but can instead be encrypted with intermediate secrets in the key exchange. We describe the new protocol in detail and prove that `Drivel` achieves the same

¹The idea of leveraging the KEM shared key towards encrypting parts of a combined KEM’s output also shows up in the deniability combiner for authenticated KEMs proposed by Gajland, Hwang, and Janneck [GHJ25].

security guarantees as `pq-obfs`.

Application: Hybrid PAKE with adaptive corruptions. Finally, we consider how to apply our combiner to construct a hybrid password-authenticated key exchange (PAKE) protocol. Existing hybrid PAKEs are secure only in the static corruption model, where the adversary may not corrupt any parties during the protocol execution [HR24, LL24]. Prior work [HR24] asked whether there exists a method for constructing adaptively secure hybrid PAKEs. We answer this question in the affirmative.

We observe that **CAKE** [BCP⁺23], which is proven secure in the UC model [Can01] with adaptive corruptions, can be instantiated with any OKEM with ciphertext and public key uniformity. As previously noted, public key uniformity guarantees for combiners seem elusive unless both underlying OKEMs have unconditional public key uniformity. We describe a method to achieve unconditional public key uniformity in LWE-based OKEMs. We observe that some standards-track KEMs, such as FrodoKEM [BCD⁺16, NAB⁺20], have a *dual*-LWE structure: both public keys and ciphertexts are LWE samples. This is done to optimize ciphertext size, but is not strictly necessary. We show how removing this optimization (reverting back to earlier ideas of [Reg05]) yields a statistically uniform public key, at the expense of a $15\times$ increase in ciphertext size.

Combining this LWE-based OKEM with a classical OKEM with unconditional ciphertext uniformity via OEINC, and then using it in **CAKE**, we achieve hybrid PAKE. This is the first known hybrid PAKE that is secure under adaptive corruptions.

2 Preliminaries

We write $y \leftarrow A(z)$ to denote assigning y the output of a deterministic algorithm $A(z)$. When A is non-deterministic, we write $y \leftarrow_s A(z)$. Similarly, we write $y \leftarrow_s S$ to denote uniformly sampling an element from the set S .

A *pseudorandom function* (PRF) $F: \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$ treats the first input as a key and the second as a label. For a randomly chosen key, the outputs on an adversarially-chosen label of F should be indistinguishable from those of a random function on the label input. The *swap* of F is $F'(y, x) = F(x, y)$. We say that F is a *swap-PRF* if its swap F' is a PRF. A *dual-PRF* is both a PRF and a swap-PRF. We defer the standard definitions of PRF security, pseudorandom generator (PRG) security, and OT-IND\$ security (one-time indistinguishability of ciphertexts from random) of symmetric encryption to Appendix A.

We adopt the definition of a *split-key pseudorandom function* from [GHP18], which is a generalization of a pseudorandom function to multiple key inputs that behaves like a random function if at least one of its key inputs is picked uniformly at random.

Definition 2.1 (Split-key pseudorandom function). *A split-key pseudorandom function $F: \mathcal{K}_1 \times \dots \times \mathcal{K}_n \times \mathcal{X} \rightarrow \mathcal{Y}$ takes as input a finite number of keys in $\mathcal{K}_1 \times \dots \times \mathcal{K}_n$ and a label in $\mathcal{X}$ and produces an output in $\mathcal{Y}$. We define the split-key pseudorandom function (skPRF) advantage of an adversary $\mathcal{A}$ against a function F as*

$$\text{Adv}_{F,i}^{\text{skPRF}}(\mathcal{A}) := \Pr \left[\mathcal{A}^{F(\dots, k_i, \dots)}() \Rightarrow 1 \mid k_i \leftarrow_s \mathcal{K}_i \right] \\ - \Pr \left[\mathcal{A}^{R(\dots)}() \Rightarrow 1 \mid R \leftarrow_s \{\text{all functions} : \mathcal{K}_{n \setminus i} \times \mathcal{X} \rightarrow \mathcal{Y}\} \right],$$

where $\mathcal{K}_{n \setminus i}$ denotes $\mathcal{K}_1 \times \dots \times \mathcal{K}_{i-1} \times \mathcal{K}_{i+1} \times \dots \times \mathcal{K}_n$.

2.1 Obfuscated KEMs

Günther, Stebila, and Veitch [GSV24] introduced notions of obfuscated key encapsulation mechanisms (OKEMs). We present equivalent definitions next, with slight modifications for simplicity.²

Definition 2.2 (Key encapsulation mechanism). *A key encapsulation mechanism $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$ consists of three algorithms:*

- $\text{KGen}() \xrightarrow{\$} (sk, pk)$ is a probabilistic key generation algorithm that generates a secret key sk and corresponding public key pk .
- $\text{Encap}(pk) \xrightarrow{\$} (c, K)$ is a probabilistic encapsulation algorithm that takes as input a KEM public key pk , and outputs a ciphertext c and shared secret K .
- $\text{Decap}(sk, c) \rightarrow K$ is a deterministic decapsulation algorithm that takes as input a secret key sk and ciphertext c , and outputs a shared secret K .

Definition 2.3 (KEM encapsulation/decapsulation correctness). *We say that a KEM $\text{KEM} = (\text{KGen}, \text{Encap}, \text{Decap})$ is δ_{KEM} -correct if*

$$\Pr \left[\text{Decap}(sk, c) \neq K \mid \begin{array}{l} (sk, pk) \xleftarrow{\$} \text{KGen}(), \\ (c, K) \xleftarrow{\$} \text{Encap}(pk) \end{array} \right] \leq \delta_{\text{KEM}}.$$

IND-CCA/CPA and SPR-CCA/CPA security. We will consider two security flavors for KEMs: (1) indistinguishability under chosen-ciphertext or chosen-plaintext attacks (IND-CCA / IND-CPA), capturing that the shared secret is indistinguishable from random given a real ciphertext, and (2) strong pseudorandomness under chosen-ciphertext attacks (SPR-CCA) [Xag22] as well as its chosen-plaintext version (SPR-CPA), capturing that a real (ciphertext, shared secret) pair is indistinguishable from a random pair. In SPR-CCA/SPR-CPA, pseudorandomness of the ciphertext is defined with respect to a simulator $\mathcal{S}$ defining a ciphertext target distribution. When the simulator $\mathcal{S}$ outputs uniformly random bitstrings, we write SPR-\$\text{CCA}/\text{SPR}\$-CPA.

We provide the formal security games in Figure 2 and define the respective advantages of an adversary $\mathcal{A}$ against a KEM K and $\text{atk} \in \{\text{CPA}, \text{CCA}\}$ as

$$\begin{aligned} \text{Adv}_{\text{K}}^{\text{IND-atk}}(\mathcal{A}) &:= 2 \cdot \Pr [\text{G}_{\text{K}}^{\text{IND-atk}}(\mathcal{A}) \Rightarrow 1] - 1 \quad \text{and} \\ \text{Adv}_{\text{K}, \mathcal{S}}^{\text{SPR-atk}}(\mathcal{A}) &:= 2 \cdot \Pr [\text{G}_{\text{K}, \mathcal{S}}^{\text{SPR-atk}}(\mathcal{A}) \Rightarrow 1] - 1. \end{aligned}$$

Definition 2.4 (KEM public key collision probability). *Let KEM be a KEM. We define the public key collision probability of KEM for $n \in \mathbb{N}$ public keys as*

$$\text{pkcoll}_{\text{KEM}}(n) := \Pr \left[\begin{array}{l} pk_i = pk_j \\ \wedge i \neq j \end{array} \mid \begin{array}{l} (sk_i, pk_i) \xleftarrow{\$} \text{KEM.KGen}() \\ \text{for } i \in [1, n] \end{array} \right].$$

Definition 2.5 (Obfuscated KEM). *An obfuscated key encapsulation mechanism (OKEM) $\text{O} = (\text{KGen}, \text{Encap}, \text{Decap}, \text{DecodePk})$ with obfuscated key length $ol \in \mathbb{N}$ and obfuscated ciphertext length $cl \in \mathbb{N}$ consists of the following algorithms:*

²Concretely, we integrate the encoding/decoding steps into the key generation, encapsulation, and decapsulation algorithms rather than having separate encoding/decoding algorithms. This is because in some cases, e.g., when Kameleon is used with rejection sampling, not every key/ciphertext can be encoded. This means that the failure rate of the encoding algorithm is captured directly in the security property of the OKEM, rather than as a separate property.

$\mathbf{G}_K^{\text{IND-CCA}}(\mathcal{A})$:	$\mathbf{G}_{K,S}^{\text{SPR-CCA}}(\mathcal{A})$:	$\mathcal{O}_{\text{Decap}}(c)$:
1 $b \leftarrow \$ \{0, 1\}$	7 $b \leftarrow \$ \{0, 1\}$	14 if $c = c^*$ then return $\perp$
2 $(pk, sk) \leftarrow \$ \text{KGen}()$	8 $(pk, sk) \leftarrow \$ \text{KGen}()$	15 $K \leftarrow \text{Decap}(sk, c)$
3 $(c^*, K_1^*) \leftarrow \$ \text{Encap}(pk)$	9 $(c_1^*, K_1^*) \leftarrow \$ \text{Encap}(pk)$	16 return K
4 $K_0^* \leftarrow \$ \mathcal{K}$	10 $c_0^* \leftarrow \$ \mathcal{S}; K_0^* \leftarrow \$ \mathcal{K}$	
5 $b' \leftarrow \$ \mathcal{A}^{\mathcal{O}_{\text{Decap}}(\cdot)}(pk, c^*, K_b^*)$	11 $c^* \leftarrow c_b^*$	
6 return $\llbracket b = b' \rrbracket$	12 $b' \leftarrow \mathcal{A}^{\mathcal{O}_{\text{Decap}}(\cdot)}(pk, c_b^*, K_b^*)$	
	13 return $\llbracket b = b' \rrbracket$	

Figure 2: Security games for IND-CCA and SPR-CCA security of a KEM $K = (\text{KGen}, \text{Encap}, \text{Decap})$ with key space $\mathcal{K}$. The IND-CPA and SPR-CPA games are obtained by removing the decapsulation oracle from IND-CCA and SPR-CCA, respectively. We write SPR\$-CCA or SPR\$-CPA when $\mathcal{S}$ outputs uniformly random bitstrings of some appropriate length cl .

- $\text{KGen}() \xrightarrow{\$} (sk, pk, \widehat{pk})$ is a probabilistic key generation algorithm that generates a secret key sk , public key pk , and obfuscated public key $\widehat{pk} \in \{0, 1\}^{\text{ol}}$.
- $\text{Encap}(pk) \xrightarrow{\$} (c, K)$ is a probabilistic encapsulation algorithm that takes as input a KEM public key pk and outputs an (obfuscated) ciphertext $c \in \{0, 1\}^{\text{cl}}$ and key K .
- $\text{Decap}(sk, c) \rightarrow K$ is a deterministic decapsulation algorithm that takes as input a secret key sk and (obfuscated) ciphertext c , and outputs a key K .
- $\text{DecodePk}(\widehat{pk}) \rightarrow pk$ is a deterministic decoding algorithm that on input an obfuscated public key $\widehat{pk} \in \{0, 1\}^{\text{ol}}$ outputs a public key pk .

Note that the tuple of algorithms $(\text{KGen}, \text{Encap}, \text{Decap})$, when ignoring the obfuscated public key $\widehat{pk}$ output by KGen , is a KEM as in Definition 2.2.

Beyond KEM correctness, as per Definition 2.3, we demand that public keys generated by KGen can be successfully decoded:

$$\Pr \left[\text{DecodePk}(\widehat{pk}) = pk \mid (sk, pk, \widehat{pk}) \leftarrow \$ \text{KGen}() \right] = 1.$$

IND-CPA, IND-CCA, SPR-CPA, and SPR-CCA security of an OKEM are defined exactly as for the underlying KEM (see Figure 2), merely ignoring the encoded public key $\widehat{pk}$ output by KGen . We also adopt notions of public key and ciphertext uniformity of (O)KEMs from [GSV24].³

Definition 2.6 (Public key uniformity). *Let $\mathcal{O}$ be an OKEM. We measure the uniformity of the obfuscated public keys of length ol generated by $\mathcal{O}.\text{KGen}$ against an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\mathcal{O}}^{\text{pk-unif}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(\widehat{pk}_b) = b \mid \begin{array}{l} b \leftarrow \$ \{0, 1\}, \widehat{pk}_0 \leftarrow \$ \{0, 1\}^{\text{ol}}, \\ (sk_1, pk_1, \widehat{pk}_1) \leftarrow \$ \mathcal{O}.\text{KGen}() \end{array} \right] - 1.$$

For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{pk-unif}}(\mathcal{A})$ statistical.

³Our `Drive1` protocol does not require public key uniformity. We still discuss the notion, as it shows up in the prior analysis of `obfs4` and `pq-obfs` [GSV24], and for PAKEs.

Definition 2.7 (Ciphertext uniformity – strong or regular). *Let $\mathcal{O}$ be an OKEM. We measure the strong or regular ciphertext uniformity of the obfuscated ciphertext of length cl generated by $\mathcal{O}.\text{Encap}$ against an adversary $\mathcal{A}$ as*

$$\text{Adv}_{\mathcal{O}}^{\text{atk-ctxt-unif}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(\overset{\text{---}}{\underset{\text{---}}{sk}}, \overset{\text{---}}{\underset{\text{---}}{pk}}, c_b) = b \mid \begin{array}{l} b \leftarrow_{\$} \{0, 1\}, c_0 \leftarrow_{\$} \{0, 1\}^{\text{cl}}, \\ (sk, pk, \widehat{pk}) \leftarrow_{\$} \mathcal{O}.\text{KGen}(), \\ (c_1, K_1) \leftarrow_{\$} \mathcal{O}.\text{Encap}(pk) \end{array} \right] - 1,$$

where $\text{atk} \in \{\text{strong}, \text{reg}\}$ and the code in the *[dashed box]* is only included for $\text{atk} = \text{strong}$. For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{atk-ctxt-unif}}(\mathcal{A})$ statistical.

We note that statistical regular ciphertext uniformity implies strong ciphertext uniformity, because a computationally unbounded adversary can simply enumerate all secret keys sk that produce the corresponding public key pk . Therefore, we simply write statistical ciphertext uniformity (without qualifier).

When paired with an encoding function like *Elligator*² [Tib14], a Diffie–Hellman-based KEM over a prime-order curve (e.g., *DHKEM*(P-256) [BBLW22]) is an OKEM with statistical ciphertext uniformity and statistical public key uniformity (see Appendix D). Additionally, *ML-KEM* paired with the *Kameleon* encoding is an OKEM with (computational) regular ciphertext uniformity and (computational) public key uniformity, as proven in [GSV24, §2.4]. Finally, *Saber* [DKRV18] and *FrodoKEM* [BCD⁺16] have the same properties as *ML-KEM* with *Kameleon* and are “natural” OKEMs, i.e., KEMs where no encoding step is necessary in order to achieve obfuscation. This is because both KEMs, based on Module Learning-with-Rounding (MLWR) and unstructured Learning-with-Errors (LWE) respectively, use power-of-two moduli, making their public keys and ciphertexts pack perfectly into bitstrings. The IND-CCA and SPR-CCA properties of *Saber* and *FrodoKEM* are stated in [Xag22, GMP22, MX23].⁴

The notions of strong pseudorandomness, public key uniformity, and (strong and regular) ciphertext uniformity are related to existing notions of anonymity and uniformity for KEMs in the literature, summarized in Figure 3. We elaborate on these relations in Appendix B, and note that all related notions are weaker than or equivalent to one of the properties we use here. The stronger notions of anonymity and uniformity are required for our later applications (see Sections 4 and 5). We adopt the above definitions of ciphertext and public key uniformity from [GSV24] (rather than prior definitions of uniformity), because we are interested in uniformity properties defined over the space of bitstrings $\{0, 1\}^*$ rather than the space of public keys/ciphertexts as used in some prior works. Although *regular* ciphertext uniformity is implied by *SPR-CCA* (*SPR-CCA* defined with respect to a simulator that outputs uniformly random bitstrings), we consider it separately because in some results we only require the weaker assumption.

Finally, to prove PAKE security, we include the following notion of strong second pseudorandom (*strong-2nd-pr*), which is “in between” the two *ctxt-unif* notions: $\text{strong-ctxt-unif} \implies \text{strong-2nd-pr} \implies \text{reg-ctxt-unif}$. We rephrase the original in terms of OKEMs.

Definition 2.8 (Strong second pseudorandomness [JRX25, Def. 2.5]). *Let $\mathcal{O}$ be an OKEM with obfuscated ciphertexts of length cl . We measure the strong second pseudorandomness*

⁴Specifically, [GMP22] shows that *FrodoKEM* achieves computational ANO-CCA, and [Xag22] suggests that it also achieves computational SPR-CCA.

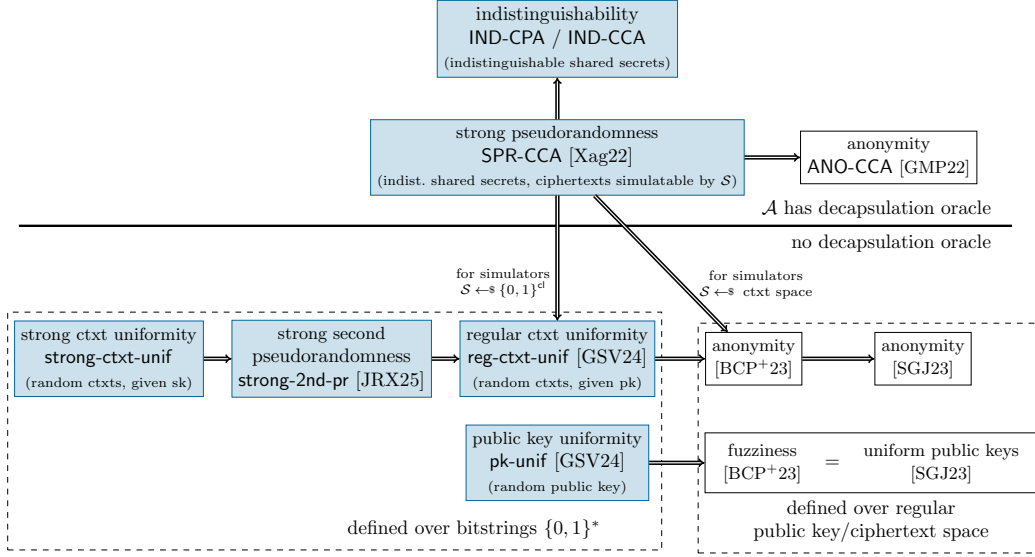


Figure 3: Relations between anonymity and uniformity notions for KEMs. Arrows $\implies$ indicate strict implications. Properties shaded in blue are used in this paper.

of $\mathcal{O}$ against an adversary $\mathcal{A}$ as

$$\text{Adv}_{\mathcal{O}}^{\text{strong-2nd-pr}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(pk, c_b, K) = b \mid \begin{array}{l} b \leftarrow_{\$} \{0, 1\}, c_0 \leftarrow_{\$} \{0, 1\}^{\text{cl}}, \\ (sk, pk, \cdot) \leftarrow_{\$} \mathcal{O}.\text{KGen}(), \\ (c_1, \cdot) \leftarrow_{\$} \mathcal{O}.\text{Encap}(pk), \\ K \leftarrow \text{Decap}(sk, c_b) \end{array} \right] - 1.$$

For an unbounded adversary $\mathcal{A}$, we call the advantage $\text{Adv}_{\mathcal{O}}^{\text{strong-2nd-pr}}(\mathcal{A})$ statistical.

3 OEINC: An OKEM Combiner

We now present our OKEM combiner and prove the necessary security properties for use in our hybrid obfuscated key exchange protocol (Section 4).

Shortcomings of existing combiners. A KEM combiner merges two ingredient KEMs into a single, hybrid KEM such that security of the hybrid is maintained as long as one of the ingredient KEMs remains secure. The main focus of prior work analyzing KEM combiners [GHP18, BBF⁺19] has been on achieving hybrid IND-CCA security: an attacker should not be able to learn the secret key by breaking only one of the underlying KEMs. Although this is a natural starting point, some applications such as broadcast encryption, anonymous credential systems, and auction protocols, require anonymity properties not implied by IND-CCA security [GMP22]. Likewise, for our use case of constructing a hybrid obfuscated key exchange, we require that the combined KEM has *hybrid obfuscation* properties, to ensure that an adversary cannot distinguish the key exchange transcript from random by breaking obfuscation of only one of the underlying primitives.

Prior work on KEM combiners constructed *parallel combiners*: the combined public keys, secret keys, and ciphertexts are the concatenation of the ingredient KEM's public keys, secret

keys, and ciphertexts, respectively (perhaps with an added MAC). The combined KEM's encapsulation and decapsulation routines first execute the underlying KEM's encapsulation and decapsulation routines and then join the resulting shared secrets using a *split-key PRF* [GHP18] to derive the final shared secret. Figure 1(a) illustrates a parallel combiner.

These existing combiners, however, do not provide hybrid obfuscation. Specifically, if an ingredient KEM's ciphertext is distinguishable from random, then its concatenation with another ciphertext is trivially distinguishable from random as well. For example, since the uniformity of ML-KEM ciphertexts relies on the MLWE assumption, breaking this assumption would be sufficient to violate the ciphertext uniformity of any parallel combiner using ML-KEM, such as Xyber [WW24] or X-Wing [BCD⁺24]. Furthermore, it is not obvious how to fix this: if one were to encrypt each component ciphertext under a key from the respective other ingredient KEM, the result is impossible to decapsulate. Therefore, we require a more thoughtful construction.

The OEINC construction. Our OKEM combiner's encapsulation function operates sequentially, rather than in parallel. This new method, depicted in Figure 1(b), operates as follows. The first (*outer*) ingredient OKEM is run, and (a key derived from) its shared secret is used to encrypt the second (*inner*) ingredient OKEM's ciphertext. The final ciphertext is the outer ciphertext concatenated with the encrypted inner ciphertext. The shared secrets are combined with a split-key PRF. We call this the *Outer-Encrypts-Inner Nested Combiner* (OEINC).

The ciphertext uniformity of this new scheme relies on the security of the encryption scheme and the ciphertext uniformity of one of the two ingredient OKEMs. We use the OKEM with the stronger ciphertext uniformity property as the outer OKEM. When the outer OKEM has statistical ciphertext uniformity, e.g., as in DHKEM combined with Elligator2 (Appendix D), we find that the resulting combiner achieves hybrid security for the relevant notions.

Our detailed combiner construction is given in Figure 4. Its building blocks include a length-preserving, OT-IND\$-secure symmetric encryption scheme SE with key space $\mathcal{K} = \{0, 1\}^{k_{\text{SE}}}$ and message length $\text{cl}_{\text{inOKEM}}$, a function $G: \{0, 1\}^{k_l} \rightarrow \{0, 1\}^{k_l + k_{\text{SE}}}$ assumed to be a pseudorandom generator, and a split-key PRF W .

3.1 Overview of Security

We now prove that OEINC satisfies IND-CPA, IND-CCA, SPR-CCA, ciphertext uniformity (ctxt-unif), and public key uniformity (pk-unif), given some properties of the underlying OKEMs. We show IND-CPA, IND-CCA, and ciphertext uniformity separately from SPR-CCA (even though they are implied by the latter, cf. Figure 3), since the weaker notions are sufficient for some applications and can be argued from simpler assumptions. More specifically, for OEINC instantiated with an outer OKEM outOKEM and an inner OKEM inOKEM :

- IND-CPA/IND-CCA: If either outOKEM or inOKEM is IND-CPA/IND-CCA, then the combined OKEM is IND-CPA/IND-CCA.
- SPR-CCA: If either (1) outOKEM is SPR-CCA, or (2) inOKEM is SPR-CCA and outOKEM is strong-ctxt-unif, then the combined OKEM is SPR-CCA.
- Ciphertext uniformity: If either (1) outOKEM is reg-ctxt-unif and outOKEM is IND-CCA, or (2) outOKEM is reg-ctxt-unif and inOKEM is reg-ctxt-unif, then the combined OKEM is reg-ctxt-unif.

<u>KGen():</u> 1 $(sk_{out}, pk_{out}, \widehat{pk}_{out}) \leftarrow \$ \text{outOKEM.KGen}()$ 2 $(sk_{in}, pk_{in}, \widehat{pk}_{in}) \leftarrow \$ \text{inOKEM.KGen}()$ 3 return $((sk_{out}, sk_{in}), (pk_{out}, pk_{in}), \widehat{pk}_{out} \parallel \widehat{pk}_{in})$ <u>Encap(pk):</u> 4 $(pk_{out}, pk_{in}) \leftarrow pk$ 5 $(K_{out}, c_{out}) \leftarrow \$ \text{outOKEM.Encap}(pk_{out})$ 6 $(K_{oe}, K_{ok}) \leftarrow G(K_{out})$ // keys for enc. & key deriv. 7 $(K_{in}, c_{in}) \leftarrow \$ \text{inOKEM.Encap}(pk_{in})$ 8 $c'_{in} \leftarrow \text{SE.Enc}(K_{oe}, c_{in})$ 9 $c \leftarrow c_{out} \parallel c'_{in}$ 10 $K \leftarrow W(K_{ok}, K_{in}, c)$ 11 return (K, c)	<u>DecodePk(pk):</u> 12 $\widehat{pk}_{out} \parallel \widehat{pk}_{in} \leftarrow \widehat{pk}$ 13 $pk_{out} \leftarrow \text{outOKEM.DecodePk}(\widehat{pk}_{out})$ 14 $pk_{in} \leftarrow \text{inOKEM.DecodePk}(\widehat{pk}_{in})$ 15 return (pk_{out}, pk_{in}) <u>Decap(sk, c):</u> 16 $(sk_{out}, sk_{in}) \leftarrow sk$ 17 $c_{out} \parallel c'_{in} \leftarrow c$ // ciphertexts are fixed-length 18 $K_{out} \leftarrow \text{outOKEM.Decap}(sk_{out}, c_{out})$ 19 $(K_{oe}, K_{ok}) \leftarrow G(K_{out})$ 20 $c_{in} \leftarrow \text{SE.Dec}(K_{oe}, c'_{in})$ 21 $K_{in} \leftarrow \text{inOKEM.Decap}(sk_{in}, c_{in})$ 22 $K \leftarrow W(K_{ok}, K_{in}, c)$ 23 return K
---	---

Figure 4: Our OKEM combiner OEINC[outOKEM, inOKEM, SE, G, W] for two OKEMs outOKEM and inOKEM, SE a length-preserving symmetric encryption scheme, G a PRG, and W a split-key PRF.

- Public key uniformity: If both outOKEM and inOKEM are pk-unif, then the combined OKEM is pk-unif.
- Strong second pseudorandomness: If either (1) outOKEM is strong-2nd-pr and outOKEM is IND-CPA, or (2) outOKEM is strong-2nd-pr and inOKEM is strong-2nd-pr, then the combined OKEM is strong-2nd-pr.

In particular, when outOKEM has statistical ciphertext uniformity and has statistical strong-2nd-pr, the combined OKEM ensures IND-CPA/CCA, SPR-CCA, reg-ctxt-unif, and strong-2nd-pr security as long as one of the underlying OKEMs is secure.

Public key uniformity stands out because it requires *both* underlying OKEMs to be pk-unif. While this limitation appears to be inherent to the primitive (when only the public key is known, there is no key material with which to obfuscate it), it is not a deal breaker for deployment. Our key exchange protocol in Section 4 does not require public key uniformity at all, since it encrypts ephemeral public keys using intermediate secrets derived from static keys. And where public key uniformity is necessary, e.g., in password-authenticated key exchange in Section 5, we show that it is possible to construct lattice-based OKEMs with statistical public key uniformity. This comes at the cost of significantly larger public keys and ciphertexts compared to, e.g., ML-KEM with Kameleon.

Formally, we establish *hybrid* security when the outer OKEM has unconditional pseudorandomness properties by proving *two* advantage bounds for each targeted security notion: one that relies on the outer OKEM's security, and another that relies on the inner OKEM's security. Jointly, these bounds establish the hybrid security claims stated informally above.

3.2 IND-CPA and IND-CCA Security

Intuitively, IND-CPA and IND-CCA security of OEINC should naturally hold, since the final shared secret is derived from the two underlying shared secrets using a split-key PRF. Importantly, OEINC does not use the same secret for encrypting ciphertexts (K_{oe}) as for

key derivation (K_{ok}). The straightforward proof is deferred to Appendix C.1 and follows similarly to the proof of IND-CCA security for existing KEM combinators [GHP18], up to the step of applying PRG security of G .

Theorem 3.1 (IND-CPA / IND-CCA security of OEINC). *Let $OEINC = OEINC[outOKEM, inOKEM, SE, G, W]$ be a combined OKEM as defined in Figure 4. Then for any adversary $\mathcal{A}$ against the IND-atk security of OEINC, for $atk \in \{CPA, CCA\}$, we give algorithms $\mathcal{B}_1, \mathcal{B}_2, \mathcal{B}_3, \mathcal{C}_1, \mathcal{C}_2$ such that*

$$\text{Adv}_{OEINC}^{\text{IND-atk}}(\mathcal{A}) \leq \text{Adv}_{outOKEM}^{\text{IND-atk}}(\mathcal{B}_1) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3),$$

and

$$\text{Adv}_{OEINC}^{\text{IND-atk}}(\mathcal{A}) \leq \text{Adv}_{inOKEM}^{\text{IND-atk}}(\mathcal{C}_1) + \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2).$$

3.3 SPR-CCA Security

Consider the case where the underlying computational assumption of $inOKEM$ is broken. Then, assuming the SPR-CCA security of the outer OKEM holds, this implies that the outer ciphertext is uniformly random and the inner ciphertext is encrypted (using an OT-IND\$ symmetric encryption scheme) with a random key, and thus also pseudorandom. The final shared secret is also random since the split-key PRF W takes as input the shared secret of the outer OKEM.

Now, consider the case where the underlying computational assumption of $outOKEM$ is broken, i.e., IND-CCA and SPR-CCA no longer hold (note that strong ciphertext uniformity cannot fail when it is statistical). Then the outer ciphertext is still uniformly random, and the inner one is a keyed permutation of a uniformly random value, and thus still random even though the encryption key may be compromised. The shared secret is also uniformly random, since the split-key PRF W still consumes the shared secret of the inner OKEM.

Recall that we use SPR\$-CCA to denote SPR-CCA security when the simulator outputs uniformly random ciphertexts of length corresponding to the OKEM's obfuscated ciphertext length cl .

Theorem 3.2 (SPR\$-CCA Security of OEINC). *Let $OEINC = OEINC[outOKEM, inOKEM, SE, G, W]$ be a combined OKEM as defined in Figure 4, where SE is a length-preserving symmetric encryption scheme, G a PRG, and W a split-key PRF. Then OEINC has strong pseudorandomness under chosen-ciphertext attack (SPR-CCA) if either (I) the outer KEM $outOKEM$ is SPR-CCA-secure, SE is OT-IND\$-secure, G is a PRG, and W is a PRF, or (II) $inOKEM$ is SPR\$-CCA-secure, $outOKEM$ has strong ciphertext uniformity, and W is a PRF.*

More precisely, for any adversary $\mathcal{A}$ against the SPR-CCA security of OEINC, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_4$ and $\mathcal{C}_1$ – $\mathcal{C}_3$ such that

$$\begin{aligned} \text{Adv}_{OEINC, \mathcal{S}_1}^{\text{SPR-CCA}}(\mathcal{A}) &\leq \text{Adv}_{outOKEM, \mathcal{S}_{outOKEM}}^{\text{SPR-CCA}}(\mathcal{B}_1) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3) \\ &\quad + \text{Adv}_{SE}^{\text{OT-IND\$}}(\mathcal{B}_4), \end{aligned}$$

where $\mathcal{S}_1$ is a simulator that returns a ciphertext $\tilde{c}_{outOKEM} || \tilde{c}_{inOKEM}$ where $\tilde{c}_{outOKEM} \leftarrow_{\$} \mathcal{S}_{outOKEM}$ and $\tilde{c}_{inOKEM} \leftarrow_{\$} \{0, 1\}^{cl_{inOKEM}}$, and

$$\text{Adv}_{OEINC}^{\text{SPR$-CCA}}(\mathcal{A}) \leq \text{Adv}_{inOKEM}^{\text{SPR$-CCA}}(\mathcal{C}_1) + \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2) + \text{Adv}_{outOKEM}^{\text{strong-ctxt-unif}}(\mathcal{C}_3).$$

We note that when $\mathcal{S}_{\text{outOKEM}}$ is the simulator that samples ciphertexts uniformly at random from $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$, then the first case also satisfies SPR-CCA.⁵

Proof. We separately consider the two advantage bounds via two proof cases.

Case I.

In the first case, we reduce to SPR-CCA security of the outer OKEM, and proceed via a series of game hops.

Game I.0. We start with the security game for SPR-CCA ($\text{G}_{\text{OEINC}, S_1}^{\text{SPR-CCA}}(\mathcal{A})$).

Game I.1. In $\text{G}_{\text{I.1}}$, we replace (in the challenge encapsulation yielding K_1^* and c_1^*) the outer OKEM shared secret K_{out} and ciphertext c_{out} with a random key $\tilde{K}_{\text{out}} \leftarrow \mathcal{K}_{\text{outOKEM}}$ and simulated ciphertext $\tilde{c}_{\text{out}} \leftarrow \mathcal{S}_{\text{outOKEM}}$. We bound the adversary's difference in advantage by a reduction $\mathcal{B}_1$ to the SPR-CCA of outOKEM w.r.t the simulator $\mathcal{S}_{\text{outOKEM}}$. $\mathcal{B}_1$ obtains the SPR-CCA challenge (pk, c^*, K^*) for outOKEM and simulates the game as follows for $\mathcal{A}$. It uses pk in place of pk_{out} . When $\mathcal{O}_{\text{Decap}}$ is called, if $c_{\text{out}} = c^*$ (in line 17 of Figure 4) then $\mathcal{B}_1$ computes the remainder of Decap using K^* as K_{out} ; else, if $c_{\text{out}} \neq c^*$ then $\mathcal{B}_1$ queries its SPR-CCA decapsulation oracle (for outOKEM) and uses the response as K_{out} .

If (c^*, K^*) are real values then $\mathcal{B}_1$ exactly simulates $\text{G}_{\text{I.0}}$ to $\mathcal{A}$; else, $\mathcal{B}_1$ simulates $\text{G}_{\text{I.1}}$ to $\mathcal{A}$. Therefore:

$$\Pr[\text{G}_{\text{I.0}}] - \Pr[\text{G}_{\text{I.1}}] \leq \text{Adv}_{\text{outOKEM}, \mathcal{S}_{\text{outOKEM}}}^{\text{SPR-CCA}}(\mathcal{B}_1).$$

Game I.2. In $\text{G}_{\text{I.2}}$, for the computation of the challenge shared secret K_1^* , we replace the output $K_{\text{oe}}, K_{\text{ok}}$ of $\text{G}(\tilde{K}_{\text{out}})$ with uniformly random $\tilde{K}_{\text{oe}}, \tilde{K}_{\text{ok}}$, in particular for the challenge. We bound the difference in this step by a reduction $\mathcal{B}_2$ to the PRG security of G . The reduction uses its oracle in place of G , simulating either $\text{G}_{\text{I.1}}$ (if the G oracle output is real) or $\text{G}_{\text{I.2}}$ (if the G oracle output is random), giving:

$$\Pr[\text{G}_{\text{I.1}}] - \Pr[\text{G}_{\text{I.2}}] \leq \text{Adv}_{\text{G}}^{\text{PRG}}(\mathcal{B}_2).$$

Game I.3. We now replace evaluations of $\text{W}(\tilde{K}_{\text{ok}}, \cdot, \cdot)$ with a random function $\tilde{\text{W}}(\cdot, \cdot)$.⁶ This in particular replaces the challenge shared secret K with a uniformly random value, independent of outputs of the decapsulation oracle since the third input c to W must be distinct from the challenge c^* for each query.

We bound this step by a reduction $\mathcal{B}_3$ to the split-key pseudorandomness of W , where $\mathcal{B}_3$ uses its oracle in place of calls to $\text{W}(\tilde{K}_{\text{ok}}, \cdot, \cdot)$. It follows that

$$\Pr[\text{G}_{\text{I.2}}] - \Pr[\text{G}_{\text{I.3}}] \leq \text{Adv}_{\text{W}, 1}^{\text{skPRF}}(\mathcal{B}_3).$$

⁵Our result for SPR-CCA security is specific to the case where the inner KEM produces ciphertexts that look like random bitstrings. It can be generalized to the case where both outer and inner KEMs produce structured, simulatable ciphertexts (e.g., in many non-obfuscated KEMs) by assuming that SE is an ideal cipher, using a generalized definition of strong-ctxt-unif that captures indistinguishability from simulated ciphertexts, and adjusting the proof accordingly. Our focus here is on obfuscated KEMs with uniformly random ciphertexts.

⁶The split-key PRF F_2 takes three distinguished inputs, two keys and one label (cf. Definition 2.1). Accordingly, the random function $\tilde{\text{W}}$ takes two distinguished inputs.

<u>Encap(pk):</u> 1 $(pk_{out}, pk_{in}) \leftarrow pk$ 2 $(\tilde{K}_{out}, \tilde{c}_{out}) \leftarrow \mathcal{K}_{outOKEM} \times \mathcal{S}_{outOKEM} \text{ // } G_{I.1}$ 3 $(\tilde{K}_{oe}, \tilde{K}_{ok}) \leftarrow \mathcal{K}_{inOKEM} \times \dots \text{ // } G_{I.2}$ 4 $(K_{in}, c_{in}) \leftarrow \text{inOKEM.Encap}(pk_{in})$ 5 $c'_{in} \leftarrow \text{SE.Enc}(\tilde{K}_{oe}, c_{in})$ 6 $c \leftarrow c_{out} \parallel c'_{in}$ 7 $K \leftarrow \tilde{W}(K_{in}, c) \text{ // } G_{I.3}$ 8 return (K, c)	<u>Decap(sk, c):</u> 9 $(sk_{out}, sk_{in}) \leftarrow sk$ 10 $c_{out} \parallel c'_{in} \leftarrow c$ 11 if $c_{out} \neq \tilde{c}_{out}$ then ... // handle regularly // else: use $\tilde{K}_{oe}, \tilde{K}_{ok}$ 12 $c_{in} \leftarrow \text{SE.Dec}(\tilde{K}_{oe}, c'_{in})$ 13 $K_{in} \leftarrow \text{inOKEM.Decap}(sk_{in}, c_{in})$ 14 $K \leftarrow \tilde{W}(K_{in}, c) \text{ // } G_{I.4}: K_{in} \leftarrow 0$ 15 return K
---	--

Figure 5: Modifications in the game hops $G_{I.1}$ – $G_{I.4}$ of the SPR-CCA proof, case (I), to the encapsulation and decapsulation algorithms of OEINC (Figure 2, line 9 resp. 15).

Game I.4. Next, we rewrite the decapsulation oracle when queried with the challenge outer ciphertext $\tilde{c}_{out}$ in a way that is unnoticeable to the adversary. First, we fix the K_{in} input to $\tilde{W}$ (Figure 5, line 14) to a zero string. Since K_{in} is deterministically derived from the second input $c = \tilde{c}_{out} \parallel c'_{in}$ (as $\tilde{K}_{oe}$ and sk_{in} are fixed), this does not change the distribution of K : decapsulating a ciphertext $c = \tilde{c}_{out} \parallel c'_{in}$ will yield distinct, randomly sampled shared secrets for distinct c'_{in} . Then

$$\Pr[G_{I.3}] = \Pr[G_{I.4}].$$

From this point on, we no longer need to decapsulate the corresponding inner ciphertext (lines 12 and 13 of Figure 5 highlighted in gray), and hence also do not need to use $\tilde{K}_{oe}$ anymore to decrypt c'_{in} when answering decapsulation queries on ciphertexts containing the challenge outer ciphertext $\tilde{c}_{out}$.

Game I.5. Finally, in $G_{I.5}$ we replace the encrypted inner ciphertext c'_{in} of the challenge with random bits of length cl_{inOKEM} , by a reduction $\mathcal{B}_4$ to the OT-IND\$ security of SE. $\mathcal{B}_4$, on input c_{in} , obtains a challenge ciphertext c^* and simulates the game for $\mathcal{A}$, replacing c'_{in} with c^* in the challenge ciphertext. Importantly, $\mathcal{B}_4$ does not need to know the encryption key $\tilde{K}_{oe}$ to answer decapsulation queries on ciphertexts containing the challenge outer ciphertext $\tilde{c}_{out}$, as per $G_{I.4}$.

Hence $\mathcal{B}_4$ exactly simulates $G_{I.4}$ or $G_{I.5}$ to $\mathcal{A}$. Therefore:

$$\Pr[G_{I.4}] - \Pr[G_{I.5}] \leq \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_4).$$

At this point, by $G_{I.1}$ and $G_{I.5}$, the ciphertext received by $\mathcal{A}$ is uniformly random and by $G_{I.3}$ the key is uniformly random. Therefore,

$$\text{Adv}_{\text{OEINC}, S_1}^{G_{I.5}}(\mathcal{A}) = 0.$$

Collecting the bounds yields the theorem statement.

Case II.

In the second case, we reduce to SPR\$-CCA security of inOKEM and strong-ctxt-unif security of outOKEM.

Game II.0. We start with the security game for SPR\$-CCA ($G_{\text{OEINC}}^{\text{SPR\$-CCA}}(\mathcal{A})$).

Game II.1. In $G_{II.1}$, we replace (in the challenge encapsulation yielding K_1^* and c_1^*) the inner OKEM shared secret K_{in} and ciphertext c_{in} with a random shared secret $\tilde{K}_{in} \leftarrow \$ \mathcal{K}_{inOKEM}$ and random ciphertext $\tilde{c}_{in} \leftarrow \$ \{0, 1\}^{cl_{inOKEM}}$. The adversary $\mathcal{A}$'s advantage is then bounded by a reduction $\mathcal{C}_1$ to the SPR\$-CCA security of inOKEM, i.e., for a simulator that samples ciphertexts uniformly at random from $\{0, 1\}^{cl_{inOKEM}}$. Reduction $\mathcal{C}_1$ obtains an SPR\$-CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ by using pk in place of pk_{in} . When $\mathcal{O}_{Decap}$ is called, if $c_{in} = c^*$ (in line 20 of Figure 4), then $\mathcal{C}_1$ uses K^* in place of K_{in} ; else (when $c_{in} \neq c^*$), $\mathcal{C}_1$ queries its SPR\$-CCA decapsulation oracle (for inOKEM) and uses the response as K_{in} .

If (c^*, K^*) are real then $\mathcal{C}_1$ exactly simulates $G_{II.0}$ to $\mathcal{A}$. Otherwise, $\mathcal{C}_1$ simulates $G_{II.1}$ to $\mathcal{A}$. Thus,

$$\Pr[G_{II.0}] - \Pr[G_{II.1}] \leq \text{Adv}_{inOKEM}^{\text{SPR\$-CCA}}(\mathcal{C}_1).$$

Game II.2. Next, we replace evaluations of $W(\cdot, \tilde{K}_{in}, \cdot)$ with a random function. This in particular replaces K (both in the challenge computation and in the decapsulation oracle) with a uniformly random value, independent of outputs of the decapsulation oracle since the third input c to W is distinct from the challenge c^* for each query. This step is bounded by a reduction $\mathcal{C}_2$ to the split-key pseudorandomness of W , where $\mathcal{C}_2$ uses its oracle in place of calls to $W(\cdot, \tilde{K}_{in}, \cdot)$. Since $\tilde{K}_{in}$ is random by $G_{II.1}$, it follows that

$$\Pr[G_{II.1}] - \Pr[G_{II.2}] \leq \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2).$$

Game II.3. In $G_{II.3}$, we replace (in the challenge encapsulation) the encrypted inner ciphertext c'_{in} with a random ciphertext in $\{0, 1\}^{cl_{inOKEM}}$. This follows directly from the fact that $\text{SE.Enc}(K_{oe}, \cdot)$ is a permutation over $\{0, 1\}^{cl_{inOKEM}}$ and that $\tilde{c}_{in}$ is a random bitstring (by $G_{II.1}$). Therefore,

$$\Pr[G_{II.2}] = \Pr[G_{II.3}].$$

Game II.4. Now, the final shared secret K and the inner ciphertext are random (and independent of K_{out}), so it only remains to replace the outer ciphertext c_{out} with random. In $G_{II.4}$, we replace c_{out} with a randomly sampled ciphertext from $\{0, 1\}^{cl_{outOKEM}}$. We bound this change by a reduction $\mathcal{C}_3$ to the strong-ctxt-unif of outOKEM. In particular, $\mathcal{C}_3$ simulates the game for $\mathcal{A}$ by using its challenge ciphertext c^* from the strong-ctxt-unif game in place of c_{out} . To answer $\mathcal{O}_{Decap}$ queries, $\mathcal{C}_3$ uses the given sk from the strong-ctxt-unif challenge to simulate calls to $\text{outOKEM.Decap}(sk_{out}, c_{out})$. Then, $\mathcal{C}_3$ exactly simulates $G_{II.3}$ (if the strong-ctxt-unif challenge is real) or $G_{II.4}$ (if the strong-ctxt-unif challenge is random) to $\mathcal{A}$. Therefore,

$$\Pr[G_{II.3}] - \Pr[G_{II.4}] \leq \text{Adv}_{outOKEM}^{\text{strong-ctxt-unif}}(\mathcal{C}_3).$$

Now, each of K , c_{out} , and c'_{in} are random. Hence, the adversary $\mathcal{A}$ can only guess the challenge bit b :

$$\text{Adv}_{OEINC}^{G_{II.4}}(\mathcal{A}) = 0.$$

Collecting the bounds yields the theorem statement. $\square$

3.4 Ciphertext Uniformity

A hybrid ciphertext uniformity guarantee holds if the outer OKEM has statistical ciphertext uniformity. When the computational assumption underlying `inOKEM` is broken and the assumption underlying `outOKEM` holds, the outer KEM ciphertext is still uniformly random by its ciphertext uniformity, and the shared secret derived from `outOKEM` is used to encrypt the inner ciphertext, thus retaining its uniformity. This suggests relying on the `IND-CPA` security of the outer OKEM and its (regular) ciphertext uniformity. However, we observe that these two properties together are equivalent to `SPR$-CPA` security and hence use the latter property, which simplifies the proof. Notably, the encryption step should not provide any additional information about the inner ciphertext; any use of authentication (such as AE or a MAC) would provide an oracle to the adversary distinguishing inner ciphertexts from random. On the other hand, if the `outOKEM` assumption is broken, then the encryption key no longer provides any guarantee, so ciphertext uniformity relies on that of both the outer and inner ciphertexts.

Theorem 3.3 (reg-ctxt-unif of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, \text{G}, \text{W}]$ be a combined OKEM as defined in Figure 4, where SE is a length-preserving symmetric encryption scheme, G a PRG, and W a split-key PRF. For any adversary $\mathcal{A}$ against the regular ciphertext uniformity (Definition 2.7) of OEINC, we give algorithms $\mathcal{B}_1$ - $\mathcal{B}_3$ and $\mathcal{C}_1$ - $\mathcal{C}_2$ such that*

$$\text{Adv}_{\text{OEINC}}^{\text{reg-ctxt-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{outOKEM}}^{\text{SPR\$-CPA}}(\mathcal{B}_1) + \text{Adv}_{\text{G}}^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_3)$$

and

$$\text{Adv}_{\text{OEINC}}^{\text{reg-ctxt-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{inOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_1) + \text{Adv}_{\text{outOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_2).$$

Proof. Game 0. We proceed via a series of game hops, starting with the security game for `reg-ctxt-unif`, $\text{G}_{\text{OEINC}}^{\text{reg-ctxt-unif}}(\mathcal{A})$.

Case I.

In the first case, we begin by reducing to `SPR$-CPA` of the outer KEM.

Game I.0. In $\text{G}_{\text{I.0}}$, we replace the outer KEM shared secret K_{out} with a uniformly random shared secret $\tilde{K}_{\text{out}}$ and the outer KEM ciphertext c_{out} with a random ciphertext sampled uniformly from $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$. All values derived from K_{out} use the random value $\tilde{K}_{\text{out}}$.

We bound the difference in this step by a reduction $\mathcal{B}_1$ to the `SPR$-CPA` security of `outOKEM`. Reduction $\mathcal{B}_1$ obtains the `SPR$-CPA` challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ by using pk , c^* , and K^* as pk_{out} , c_{out} , and K_{out} , respectively. (Note that there are no decapsulation queries to be handled for `reg-ctxt-unif` security.) If K^* is the real KEM shared secret and c^* the real ciphertext, then $\mathcal{B}_1$ has exactly simulated G_0 to $\mathcal{A}$; else, if K^* and c^* are random, then $\mathcal{B}_1$ has exactly simulated $\text{G}_{\text{I.0}}$ to $\mathcal{A}$. Therefore:

$$\Pr[\text{G}_0] - \Pr[\text{G}_{\text{I.0}}] \leq \text{Adv}_{\text{outOKEM}}^{\text{SPR\$-CPA}}(\mathcal{B}_1).$$

Game I.1. In $G_{I.1}$, we replace (K_{oe}, K_{ok}) with uniformly random values $\tilde{K}_{oe}$, and $\tilde{K}_{ok}$. We bound the difference in this step by a reduction $\mathcal{B}_2$ to the PRG security of G . The reduction uses its input in place of K_{oe}, K_{ok} . Since K_{out} is random by $G_{I.0}$, $\mathcal{B}_2$ simulates either $G_{I.0}$ or $G_{I.1}$, giving:

$$\Pr[G_{I.0}] - \Pr[G_{I.1}] \leq \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2).$$

Game I.2. Finally, in $G_{I.2}$, we replace the encrypted inner ciphertext c'_{in} of the challenge with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. Note that K_{oe} is random by $G_{I.1}$, so this change can be bounded by a reduction $\mathcal{B}_3$ to the OT-IND\$ security of SE. $\mathcal{B}_3$ obtains a challenge ciphertext c^* and simulates the game for $\mathcal{A}$, replacing c'_{in} with c^* . It follows that:

$$\Pr[G_{I.1}] - \Pr[G_{I.2}] \leq \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_3).$$

Now, the ciphertext received by $\mathcal{B}$ consists of a random $c_{out} \in \{0, 1\}^{\text{cl}_{\text{outOKEM}}}$ by $G_{I.0}$ and a random $c'_{in} \in \{0, 1\}^{\text{cl}_{\text{inOKEM}}}$ by $G_{I.1}$. Hence, $\mathcal{A}$ has no better chance than guessing the challenge bit b :

$$\text{Adv}_{\text{OEINC}}^{G_{I.2}}(\mathcal{A}) = 0.$$

Case II.

This case reduces to **reg-ctxt-unif** of **inOKEM** and **outOKEM**.

Game II.0. In $G_{II.0}$, we replace the inner ciphertext c_{in} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. We bound the difference by a reduction $\mathcal{C}_1$ to the **reg-ctxt-unif** of **inOKEM**. $\mathcal{C}_1$ uses its ciphertext challenge c_b in place of c_{in} , and pk, K in place of pk_{in}, K_{in} . This simulates G_0 if c_b is real, and $G_{II.0}$ if c_b is random:

$$\Pr[G_0] - \Pr[G_{II.0}] \leq \text{Adv}_{\text{inOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_1).$$

Game II.1. Next, we replace c'_{in} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$ in $G_{II.1}$. c_{in} is random by $G_{II.0}$, so it follows directly from the fact that $\text{SE.Enc}(K_{oe}, \cdot)$ is a permutation over $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$, for a fixed key K_{oe} , that:

$$\Pr[G_{II.0}] = \Pr[G_{II.1}].$$

Game II.2. Lastly, in $G_{II.2}$, we replace the outer ciphertext c_{out} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$. We bound this by a reduction $\mathcal{C}_2$ to **reg-ctxt-unif** of **outOKEM**. $\mathcal{C}_2$ simulates either $G_{II.1}$ or $G_{II.2}$ by using its challenge ciphertext in place of c_{out} (and pk, K in place of pk_{out}, K_{out}). Therefore:

$$\Pr[G_{II.1}] - \Pr[G_{II.2}] \leq \text{Adv}_{\text{outOKEM}}^{\text{reg-ctxt-unif}}(\mathcal{C}_2).$$

At this point, both c'_{in} and c_{out} are random bitstrings. Thus, $\mathcal{A}$ can only guess the challenge bit b :

$$\text{Adv}_{\text{OEINC}}^{G_{II.2}}(\mathcal{A}) = 0.$$

Collecting the bounds yields the theorem statement. $\square$

3.5 Strong Second Pseudorandomness

We show that our combiner provides strong second pseudorandomness so that we may use it in the CAKE password-authenticated key exchange protocol (Section 5). At a high level, this property is similar to ciphertext uniformity. A hybrid strong second pseudorandomness guarantee holds if the outer OKEM has statistical strong second pseudorandomness. More precisely, if the outer OKEM has statistical strong second pseudorandomness, then strong second pseudorandomness of the combiner is achieved if either the inner OKEM is strong second pseudorandom or the outer OKEM is IND-CPA-secure.

We also remark that the theorem requires that the split-key PRF W be defined on all inputs, i.e., even when an OKEM decapsulation has failed. This either means that the OKEMs' explicit rejection symbol $\perp$ be considered ordinary inputs to W (e.g., specially encoded), or that the OKEMs never output an explicit rejection (e.g., by implicitly rejecting). Notably, any OKEM aiming to satisfy **strong-2nd-pr** has to be implicitly rejecting (since an explicitly rejecting OKEM can trivially be distinguished via the rejecting key output), hence we deem this a mild additional assumption.

Theorem 3.4 (Strong second pseudorandomness of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, G, W]$ be a combined OKEM as defined in Figure 4, where outOKEM is δ_{outOKEM} -correct. For any adversary $\mathcal{A}$ against the strong second pseudorandomness (Definition 2.8) of OEINC, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_5$ and $\mathcal{C}_1$ – $\mathcal{C}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{OEINC}}^{\text{strong-2nd-pr}}(\mathcal{A}) &\leq 2 \cdot \left(\delta_{\text{outOKEM}} + \text{Adv}_{\text{outOKEM}}^{\text{IND-CPA}}(\mathcal{B}_1) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3) \right) \\ &\quad + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_4) + \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{B}_5), \end{aligned}$$

and

$$\text{Adv}_{\text{OEINC}}^{\text{strong-2nd-pr}}(\mathcal{A}) \leq \delta_{\text{outOKEM}} + \text{Adv}_{\text{inOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_1) + \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_2).$$

We defer the proof of strong second pseudorandomness to Appendix C.2.

3.6 Public Key Uniformity

Finally, OEINC only has public key uniformity as long as *both* underlying OKEMs retain their public key uniformity. Since no shared key material is available when distributing a public key, there is little that would help in obfuscating a public key if the underlying assumption is broken. Thus, if one of the schemes does not have statistical public key uniformity, there does not appear to be a way to construct a scheme with hybrid public key uniformity, barring some additional PKI-like infrastructure. OEINC does achieve public key uniformity conditioned on the public key uniformity of *both* underlying schemes:

Theorem 3.5 (pk-unif of OEINC). *Let $\text{OEINC} = \text{OEINC}[\text{outOKEM}, \text{inOKEM}, \text{SE}, G, W]$ be a combined OKEM as defined in Figure 4. For any adversary $\mathcal{A}$ against the public key uniformity (Definition 2.6) of OEINC, we give algorithms $\mathcal{B}_1, \mathcal{B}_2$ such that*

$$\text{Adv}_{\text{OEINC}}^{\text{pk-unif}}(\mathcal{A}) \leq \text{Adv}_{\text{outOKEM}}^{\text{pk-unif}}(\mathcal{B}_1) + \text{Adv}_{\text{inOKEM}}^{\text{pk-unif}}(\mathcal{B}_2).$$

We defer the proof of public key uniformity to Appendix C.3.

Table 1: Security properties of combiners instantiated with different input (O)KEMs. Underlying computational assumptions are given in parentheses. Dots in the table indicate: (at least) one of the underlying assumptions must hold, i.e., hybrid guarantee (●); the assumption of the outer KEM must hold (◐); both underlying assumptions must hold (⊙); or the property does not hold (–). For SPR-CCA, subscripts indicate whether the respective simulator samples bitstrings (\$) or uniform ciphertexts from the ciphertext space ($\mathcal{C}$).

Combined KEM		IND-CCA	SPR-CCA	reg-ctxt-unif	pk-unif
OEINC Outer and Inner KEM					
DHKEM-ElI2(GapDH)	ML-Kameleon(MLWE)				
	Saber (MLWR)	●	● _{\$}	●	⊙
	FrodoKEM (LWE)				
ML-Kameleon(MLWE)	DHKEM-ElI2(GapDH)	●	◐ _{\$}	◐	⊙
DHKEM-ElI2(GapDH)	ML-KEM(MLWE)	●	◐ _{\$}	◐	–
DHKEM(GapDH)	ML-KEM(MLWE)	●	◐ _{$\mathcal{C}$}	–	–
X-Wing [BCD⁺24] (MLWE, GapDH)		●	⊙ _{$\mathcal{C}$}	–	–
Parallel Combiner [GHP18, BBF⁺19] ML-Kameleon + DHKEM-ElI2 (MLWE, GapDH)		●	⊙ _{\$}	⊙	⊙

3.7 Instantiating the Combiner

Following our results for OEINC, we now know that we can construct an OKEM with hybrid guarantees when the outer OKEM has statistical ciphertext uniformity. Previous work [GSV24] constructed an MLWE-based OKEM, ML-Kameleon, from ML-KEM and Kameleon encodings, notably whose ciphertext (and public key) uniformity depends on the MLWE assumption, i.e., it is not statistical. Similarly, Saber [DKRV18] and FrodoKEM [BCD⁺16] achieve uniformity from MLWR and LWE assumptions, respectively. These are natural choices for the inner OKEM. As for the outer OKEM, we can construct a (classically-secure) OKEM using DHKEM [BBLW22] and Elligator2 [BHK13] or Elligator² [Tib14] encodings. We provide the details of this construction, denoted DHKEM-ElI2, and related security properties in Appendix D. Importantly, Elligator-style encodings do not rely on any underlying computational assumptions for uniformity guarantees, thus giving the resulting OKEM *statistical* ciphertext uniformity. Both ML-Kameleon and DHKEM-ElI2 are SPR\$-CCA-secure. The symmetric encryption SE can be instantiated with XOR (the one-time pad gives an OT-IND\$ guarantee, and is a permutation over bitstrings for a fixed key). Therefore, instantiating OEINC[DHKEM-ElI2, ML-Kameleon, $\oplus$, G, W] for any reasonable choice of PRG G and split-key PRF W (e.g., see [GHP18, BBF⁺19]) provides the desired hybrid guarantees.

We now explore alternative options for instantiating the combined OKEM, omitting any further discussion of SE, G, and W, as they can be instantiated with $\oplus$, any secure PRG, and any split-key PRF, respectively. Table 1 summarizes the properties of each of the following variants, and compares them with the security properties provided by X-Wing [BCD⁺24] (which does not use obfuscated KEMs) and the generic parallel combiner [GHP18, BBF⁺19] instantiated with two obfuscated KEMs.

Non-obfuscated KEMs. It should be noted that the OEINC construction is not restricted to only OKEMs. Ignoring the step that encodes public keys, one can use any KEM as an

input to the combiner. It is then interesting to ask which properties are maintained by the combiner in such cases. Of course, using any two IND-CCA secure KEMs (e.g., ML-KEM and DHKEM) results in an IND-CCA combined KEM. The remaining properties depend on the input KEMs. For example, using ML-KEM as the inner KEM and DHKEM as the outer KEM, the combined KEM would not achieve public key or ciphertext uniformity; however, it does achieve SPR-CCA security with respect to a simulator that outputs structured ciphertexts (i.e., DH values concatenated with random bitstrings resulting from encrypting ML-KEM ciphertexts), assuming that the outer KEM retains its SPR-CCA security. If the assumption underlying DHKEM is broken, then this construction no longer achieves SPR-CCA security, regardless of the security of the inner KEM.

Outer OKEM and inner KEM. Alternatively, one might ask what happens if only the outer KEM is obfuscated and the inner KEM is not. To illustrate, we consider the combination of DHKEM-ElI2 as the outer KEM and ML-KEM as the inner KEM. It is easy to see that IND-CCA security is maintained as a hybrid guarantee and that public key uniformity is no longer achieved. SPR-CCA security is achieved if the outer OKEM is SPR-CCA-secure, but the combination does not provide a hybrid guarantee. There is no longer a hybrid guarantee for ciphertext uniformity, which now relies on the IND-CPA security and ciphertext uniformity of DHKEM-ElI2 (since ML-KEM has no ciphertext uniformity).

Outer OKEM without strong-ctxt-unif. Finally, we ask what happens if we use ML-Kameleon as the outer KEM and DHKEM-ElI2 as the inner KEM (swapping the order of our initial proposed instantiation). The (hybrid) IND-CCA and (non-hybrid) public key uniformity guarantees remain the same as in the original case. SPR-CCA is not a hybrid guarantee anymore and now depends on the SPR-CCA security of ML-Kameleon, since ML-Kameleon does not have statistical ciphertext uniformity. In particular, when the outer assumption breaks down, the key used for encryption may no longer be random, so even though the inner ciphertexts are simulatable, it is unclear how to simulate the (not necessarily random) keyed permutation of (non-uniform) simulatable ciphertexts. Similarly, ciphertext uniformity requires now that the ciphertext uniformity (and IND-CPA security) of ML-Kameleon is maintained (i.e., no hybrid guarantee).

4 A Hybrid Obfuscated Key Exchange Protocol

We now present **Drivel**, a revised version of the post-quantum variant **pq-obfs** [GSV24] of Tor’s **obfs4** protocol that enables hybrid security guarantees. We recall the purpose of an obfuscated key exchange protocol [GSV24] is to permit a client with prior knowledge of a server’s public key to establish a fresh session key with the server, such that (1) no passive adversary can distinguish the handshake from a random sequence of messages of the same length and ordering; and (2) the server is otherwise quiet, meaning it does not respond to any messages sent by an adversary that does not demonstrate knowledge of the server’s public key. The intention of the **obfs4** protocol in particular is to provide a layer of obfuscated traffic that does not match a censor’s protocol blocklist and is secure against probing by censors. We state the security goals in more detail below.

1. **Key indistinguishability.** A Bellare–Rogaway-style [BR94] notion of key indistinguishability requires that the shared key derived in the protocol must be indistinguishable from random. Formally, an adversary actively interacts with multiple sessions, is allowed to reveal user and session keys, and must guess the challenge bit b used in

a TEST oracle which returns either real or random session keys depending on $\mathbf{b}$. (A Fresh predicate prevents trivial attacks like testing and revealing the same key.) We also aim for *forward secrecy*: session keys must remain secure if the involved server’s long-term secret key is later compromised.

2. **Obfuscation.** The core goal of obfuscated key exchange is to offer protection against passive deep packet inspection by hiding protocol fingerprints. The protocol transcript should therefore be indistinguishable from a simulated transcript, where the *simulator* $\mathcal{S}$ is a parameter of the security definition. Using a simulation-based definition allows one to capture additional structure of the key exchange transcript (e.g., packet lengths, entropy, etc., that are typically used for protocol fingerprinting). In particular, protocols can be divided into *classes* corresponding to simulators, where a class determines its traffic pattern. As with **pq-obfs**, we prove obfuscation with respect to a simulator that outputs random messages of varying lengths. This captures the idea that the protocol should lie within the class of protocols where all transmitted data appears to be random bits.

An adversary’s goal is to guess the challenge bit $\mathbf{b}$ used in a CHALLEXEC oracle which returns either real or simulated transcripts of the protocol (and real or random keys, respectively). The challenge bit $\mathbf{b}$ is the same as is used in TEST queries. The adversary must not violate the ObfFresh predicate, which requires that the server’s secret key is not revealed (i.e., we aim for *strong obfuscation* as defined in [GSV24]).

3. **Probing resistance.** The protocol should be resistant to *active probing*, where censors probe suspicious proxy servers in an attempt to identify them. A responder should not respond to messages from an initiator who has not proven knowledge of the responder’s public key. This is captured by the SEND oracle and the Probed flag: If the adversary successfully elicits a non-empty response from a responder, whose public key is not revealed, in response to a message not previously sent by an honest initiator, then the adversary wins.
4. **Explicit authentication.** Finally, ExplicitAuth ensures explicit authentication of the server. An adversary wins if it causes a client to accept a session without a partnered server session existing.

4.1 Shortcomings of the pq-obfs Protocol

We identify a key issue that prevents **pq-obfs** from achieving hybrid guarantees, even when instantiated with a hybrid OKEM. In the first round of **pq-obfs**, the client, who knows the server’s static KEM public key pk_S , encapsulates to pk_S , yielding (c_S, K_S) and generates an ephemeral keypair $(sk_e, pk_e) \leftarrow \text{KEM.KGen}()$. In the first message, the client sends c_S and pk_e to the server. The server will decapsulate c_S and use pk_e to establish a fresh session key with the client. The issue with this approach is that pk_e is sent to the server *in the clear*, requiring public key uniformity from the ephemeral OKEM. However, we do not know of an existing OKEM with *hybrid* public key uniformity guarantees. In other words, **pq-obfs** with a hybrid OKEM has obfuscation that is only as strong as the weakest **pk-unif** of its underlying OKEMs, voiding the hybrid guarantees on the key exchange level.

4.2 The Drivel Protocol

We propose the **Drivel** protocol which addresses the above hybrid obfuscation barrier of **pq-obfs** while achieving the same security goals. In short, the key insight is to encrypt the ephemeral public key and ciphertext using a key derived from the shared secret from the static OKEM encapsulation. That way, **Drivel**, unlike **pq-obfs**, does not require public-key uniformity of the deployed (hybrid) ephemeral KEM, which is the only OKEM property for which OEINC does not achieve hybrid guarantees. Furthermore, additionally encrypting the ephemeral ciphertext means that for the ephemeral KEM, a regular (non-obfuscated) KEM suffices, improving efficiency. Instantiating **Drivel** with OEINC[DHKEM-ElI2, ML-Kemeleon, $\oplus$, G, W] (for reasonable PRG G and PRF W) as the static KEM and any hybrid IND-1CCA-secure (non-obfuscated) KEM as the ephemeral KEM hence yields a readily implementable hybrid obfuscated key exchange protocol. The final structure remains closely aligned with **pq-obfs** and, thus, the currently deployed **obfs4** protocol.

We describe the protocol at a high level. We retain from **pq-obfs** the general structure of ephemeral and static KEM encapsulations and the key schedule (modulo deriving the additional encryption keys), so that we need only rely on standard assumptions (avoiding any random oracles). The full description can be found in Figure 6. In this setting, we assume that the server has published semi-private information, namely its OKEM public key and an identifier $NodeID \in \{0, 1\}^n$, to a distribution service that only honest clients have access to. In practice, this can be a *bridge distribution service*, as used with Tor, or another out-of-band sharing mechanism. Although pre-shared semi-private information is a seemingly strong assumption, it is already an assumption of **obfs4**.

Client to server message. A client who knows the server’s public key and $NodeID$ begins by generating a fresh ephemeral KEM keypair. It then establishes a shared secret with the server by encapsulating to the server’s static OKEM public key. The client sends this ciphertext along with the ephemeral public key (encrypted, using a key derived from the static OKEM encapsulation), randomized padding, and MACs which include the server’s public key and $NodeID$.

Server to client message. The server decapsulates and checks the MACs. On success, it encapsulates to the client’s ephemeral public key, and sends back the ciphertext (encrypted using a key derived from the static OKEM encapsulation), along with padding and MACs. The client verifies these and derives the session key using the ephemeral-ephemeral and ephemeral-static KEM shared secrets, as well as the corresponding public keys and ciphertexts.

MACs. The MAC tags M_C and M_S are intended to aid in parsing the variable-length messages. In particular, the server may continue sending data immediately following its message to the client, and so the M_S marker identifies where the initial handshake message (and variable length padding) ends. The client’s tag M_C is not strictly necessary, as the server may parse the handshake message from the end; however, we leave it in the description for symmetry and to allow for potentially more flexible designs in the future (e.g., the client sending data immediately following its handshake message).

Replay protection. We assume that, for replay protection, the protocol works in epochs; the client includes its epoch in MAC tag MAC_C . The server checks that this epoch is within a valid range, and stores the set of MAC values seen in its state $\mathcal{S}_{MAC}$ per epoch. To simplify the presentation, we omit epochs here.

Instantiating the protocol. Like **pq-obfs**, **Drivel** relies on a secure pseudorandom

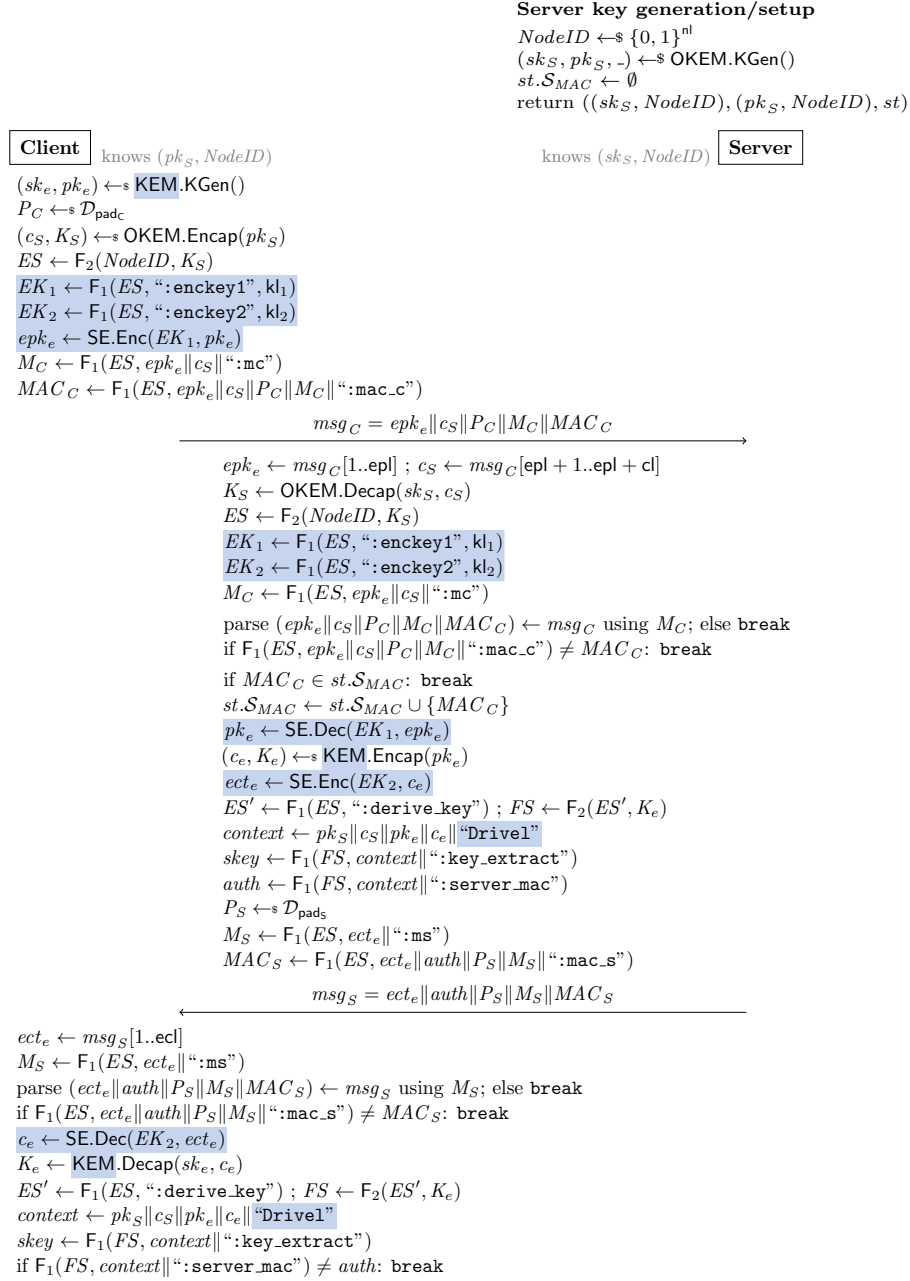


Figure 6: The **Drive1** obfuscated key exchange protocol. OKEM is an OKEM satisfying IND-CCA, SPR\$-CCA, and ciphertext uniformity. KEM is an IND-1CCA-secure KEM. SE is an OT-IND\$-secure symmetric encryption scheme. F_1 is a PRF and F_2 is a dual PRF. Core differences to the **pq-obfs** protocol from [GSV24] are highlighted in blue boxes. Values epl , ecl , cl are the lengths of the encrypted ephemeral KEM public key and ciphertext as well as OKEM ciphertext, respectively; kl_1 , kl_2 the encryption key lengths.

function F_1 with (default) output length fl_1 and a dual-PRF F_2 with output length fl_2 . In addition, we employ an OT-IND $\$$ -secure symmetric encryption scheme SE . To handle potentially varying key lengths, we ask that F_1 also supports variable output lengths, where the output length is optionally specified through the function’s third input (by default, it is fl_1). Unlike `pq-obfs`, `Drive1` does not require the ephemeral KEM to have public key or ciphertext uniformity. Instead, the client’s ephemeral KEM public key is encrypted with a key derived from the static KEM, as is the server’s ciphertext response. Thus, any IND-1CCA-secure KEM can be used for the ephemeral KEM, and any OKEM with IND-CCA security, SPR $\$$ -CCA security, and ciphertext uniformity can be used for the static OKEM.

This can be achieved, for example, with an obfuscated KEM constructed using OEINC from Section 3 (see Table 1 for examples). The ephemeral KEM can be instantiated with any regular (i.e., non-obfuscated) KEM with IND-1CCA security (e.g., X-Wing [BCD⁺24] would provide hybrid guarantees⁷). We can instantiate F_1 with HKDF-Expand [Kra10, KE10] and F_2 with HMAC [BCK96, KBC97]; the latter’s dual-PRF security for fixed-length keys is proven in [BBGS23]. The symmetric encryption scheme, requiring only OT-IND $\$$ security, can be instantiated with XOR with appropriate key lengths kl_1, kl_2 matching the ephemeral KEM’s public key and ciphertext lengths.

4.3 Security

We use the security model of obfuscated key exchange from [GSV24], denoted `sObfKE`, discussed informally at the start of this section; we provide the full details in Appendix E for reference. In addition to the details of the model specific to the desired security properties described above, session identifiers are used to determine that two sessions are partnered, while contributive identifiers are used to determine when a responder session has an honest communication partner. The security analysis makes use of a selective security variant of the game, where the adversary has to commit upfront to winning via (1) a single TEST query to a pre-declared session, or (2) a single CHALLENGE query against a pre-declared server. This variant implies full security via a hybrid argument, as shown in [GSV24].

Simulator definition. We establish `sObfKE` security with respect to the following simulator $\mathcal{S}_{\text{Drive1}}$ which outputs two uniformly random messages of length dependent on the `Drive1` distribution of padding:

```

 $\mathcal{S}_{\text{Drive1}}$ :
1  $P_C \leftarrow \mathcal{D}_{\text{pad}_C}$  ;  $P_S \leftarrow \mathcal{D}_{\text{pad}_S}$  // sample client/server padding according to distribution
2  $m_1 \leftarrow \{0, 1\}^{\text{epl} + \text{cl} + 2 \cdot fl_1 + |P_C|}$  // 1x encrypted pk + 1x ctxt + 2x  $F_1$  outputs (MACs) + client padding
3  $m_2 \leftarrow \{0, 1\}^{\text{ecl} + 3 \cdot fl_1 + |P_S|}$  // 1x encrypted ctxt + 3x  $F_1$  outputs (auth and MACs) + server padding
4 return  $(m_1, m_2)$ 

```

Here, `epl`, `ecl` are the lengths of the encrypted ephemeral KEM public key and ciphertext, respectively, fl_1 is the (default) output length of the PRF F_1 , and `cl` is the ciphertext length of OKEM.

Session and contributive identifiers. We set the session identifier as $sid := (pk_e, c_e, pk_s, c_s)$ where pk_e is the initiator’s KEM public key and c_e is the corresponding KEM

⁷In fact, even faster options are possible. Most lattice-based IND-CCA KEMs use a variant of the Fujisaki–Okamoto transform [FO99], which requires the decapsulator to re-encrypt the message to check for tampering. With the relaxation to IND-1CCA, this step is no longer necessary [JMZ23].

ciphertext, and pk_S is the responder's static KEM public key and c_S the corresponding ciphertext. We set the contributive identifier to $cid := (pk_e, c_S)$ upon the client sending or server receiving the first message.

Theorem 4.1. *Let `Drivel` be defined as in Figure 6. For any sObfKE adversary $\mathcal{A}$ against `Drivel`, we give algorithms $\mathcal{B}_1$ – $\mathcal{B}_{17}$ such that*

$$\begin{aligned} \text{Adv}_{\text{Drivel}, S_{\text{Drivel}}}^{\text{sObfKE}}(\mathcal{A}) &\leq 2 \cdot \left(\text{pkcoll}_{\text{KEM}}(n_s) + \text{pkcoll}_{\text{OKEM}}(n_r) + n_s \cdot (\delta_{\text{OKEM}} + \delta_{\text{KEM}}) \right. \\ &\quad + n_s n_r \cdot \left(\text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_2) + \frac{1}{2^{\text{fl}_1}} \right) \\ &\quad + n_s n_r \cdot \left(\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_4) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_5) \right. \\ &\quad \quad \left. + \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_6) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_7) + \frac{1}{2^{\text{fl}_1}} \right) \\ &\quad + (n_s + n_r q_C) \cdot \left(n_s \cdot \left(\text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_8) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_9) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{10}) \right) \right. \\ &\quad \quad + \text{Adv}_{\text{OKEM}}^{\text{SPR\$-CCA}}(\mathcal{B}_{11}) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_{12}) \\ &\quad \quad + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{13}) + \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_{14}) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{15}) \\ &\quad \quad \left. \left. + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{16}) + \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{17}) \right) \right), \end{aligned}$$

where $\mathcal{A}$ makes at most q_C `CHALLENGE` queries; n_s, n_r are the number of sessions and servers (parties in responder role) that $\mathcal{A}$ interacts with, respectively; fl_1 is the output bit-length of F_1 ; and δ_{OKEM} and δ_{KEM} are the correctness errors of the obfuscated and ephemeral KEM, respectively.

Due to the intentional similarity of `Drivel` to `pq-obfs`, the proof of Theorem 4.1 follows closely (in large parts verbatim) the security proof for `pq-obfs` from [GSV24]. Here, we provide a proof sketch with the main changes compared to the proof for `pq-obfs` underlined; the complete proof is deferred to Appendix F.

Proof sketch. We proceed via a series of game hops and branches, considering each clause in `FINALIZE`. We rule out KEM public key collisions (term $\text{pkcoll}_{\text{KEM}}(n_s) + \text{pkcoll}_{\text{OKEM}}(n_r)$) and assume correct decapsulation in all sessions ($n_s \cdot (\delta_{\text{OKEM}} + \delta_{\text{KEM}})$), which ensures soundness (`Sound`).

A first branch then rules out that $\mathcal{A}$ successfully probes a server (`Probed`): we guess the first server and session that sets `Probed` (with a $n_s \cdot n_r$ loss). From that server's uncompromised semi-private *NodeID*, we apply PRF security twice to turn *ES* and then *MAC_C* into random values. After this, we observe that `Probed` is set if a non-initiator-first message m yields a reply by an unrevealed responder. Such a message m is either rejected due to the replay check, or it is different from all honest initiators' first messages sent to this responder. In the second case, m contains the target client MAC value MAC_C^* which is a random fl_1 -bit value unknown to $\mathcal{A}$. The adversary $\mathcal{A}$ can therefore only guess MAC_C^* ; the corresponding bound is $\text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_2) + 1/2^{\text{fl}_1}$.

A second branch prevents explicit authentication (`ExplicitAuth`) from being violated. We guess the first session violating `ExplicitAuth`, π^* , and its peer, v^* , (losing a factor $n_s \cdot n_r$). Then we can embed an IND-CCA challenge in that the peer's public key, the session's c_S , and the derived K_S shared secret (term $\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3)$). Let $(c_S, K_S) \leftarrow \text{OKEM.Encap}(pk_S)$

be the encapsulation computed in the target session π^* with the long-term public key of v^* . We replace the long-term shared secret K_S with a uniformly random $\tilde{K}_S$ in π^* . All values derived from K_S in π^* use the randomized value $\tilde{K}_S$. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_3$ to the IND-CCA security of OKEM. Next, through four (swap-)PRF hops ($\mathcal{B}_4$ – $\mathcal{B}_7$), we show that this turns *auth* into a random fl_1 -bit value leaving $\mathcal{A}$ with a $1/2^{\text{fl}_1}$ chance to violate **ExplicitAuth**.

We then restrict the adversary to a single-challenge (sObfKE-1) version of the game, applying the hybrid argument from [GSV24, Theorem 4.3] with a $(n_s + n_r q_C)$ loss. We separately treat the following two cases.

For Case I ($\mathcal{A}$ makes a single TEST query), we first guess the (*sid*- or *cid*-) partner session of the tested session (losing a factor n_s). We then embed an IND-1CCA challenge in the ephemeral KEM encapsulation of the test session, π^* , and its partner session, π_p^* (term $\text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_8)$). In particular, we replace the ephemeral KEM key K_e with a uniformly random $\tilde{K}_e$ in the target session π^* . All values derived from K_e in π^* use the randomized value $\tilde{K}_e$. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_8$ to the IND-1CCA security of OKEM. Following this, we apply two (swap-)PRF hops ($\mathcal{B}_9$, $\mathcal{B}_{10}$) to randomize the test session's key, concluding this case.

For Case II ($\mathcal{A}$ makes a single CHALLENGE query to a pre-determined server), we embed an SPR\$-CCA challenge in the long-term KEM encapsulation of the CHALLENGE sessions (term $\text{Adv}_{\text{OKEM}}^{\text{SPR\$-CCA}}(\mathcal{B}_{11})$), randomizing both c_S and K_S . Specifically, we replace the ciphertext c_S , as well as the long-term KEM key K_S as follows. Instead of running **OKEM.Encap**, the initiator samples $c_S \leftarrow_{\$} \{0, 1\}^{\text{cl}}$ and $K_S \leftarrow_{\$} \mathcal{K}$ uniformly at random. We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_{11}$ to the SPR\$-CCA security of OKEM. Subsequently, we proceed via four (swap-)PRF hops ($\mathcal{B}_{12}$ – $\mathcal{B}_{15}$) to randomize the transcript's MAC values as well as the encryption keys and session key. The main difference from the proof of **pq-obfs** is that in the final steps, we apply the OT-IND\$ security of the symmetric encryption scheme **SE** to randomize epk_e and ect_e , corresponding to the terms $\text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{16})$ and $\text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{17})$. At this point, the protocol messages in CHALLENGE sessions are uniformly distributed like outputs of the simulator $\mathcal{S}_{\text{Drivel}}$, concluding this case and the proof. $\square$

Hybrid security. **Drivel** achieves hybrid (classic / quantum-safe) security when using a hybrid construction for the involved OKEM and KEM. (Note that all other components of **Drivel** are symmetric.) Instantiating the OKEM with **OEINC**, advantage terms corresponding to OKEM can be unfolded using the bounds established in Section 3, yielding corresponding hybrid bounds for **Drivel**.

4.4 Additional Features

We now discuss optional features of **Drivel** which are intended to aid in development and deployment, but are not included in our security analysis.

Strong obfuscation vs. DoS resistance. **Drivel**, as described thus far, has *strong obfuscation*, meaning that even an adversary who learns the server's static public key pk_s (recall that this is considered semi-private information in our setting) is not able to passively distinguish a protocol's transcript from a simulated one. The cost of this advantage is that the server must perform a decapsulation for every incoming message, even when the other

party does not know the server’s *NodeID*. Since decapsulations can be costly, this may be used by an attacker to launch a denial-of-service attack against the server.⁸

In order to accommodate different deployment environments, the client can optionally send a *cookie* in the first round of communication, as was done in the original *obfs4* protocol as well as Wireguard [Don17]. Concretely, this cookie may be computed as $\text{HMAC}(pk_s \parallel \text{NodeID}, epk_e \parallel c_S \parallel P_C \parallel M_C \parallel MAC_C)$ (the difference from MAC_C being that a static key $pk_s \parallel \text{NodeID}$ is used rather than an ephemeral key ES). The server can then check this cookie prior to performing decapsulation, preventing the processing of messages from clients that do not know *NodeID*. To prevent replays, the server must store past cookies.

Finally, we note that there are middle ground options between strong and regular obfuscation. For example, a server may publish a temporary pseudorandom token *tok*, along with its public key and *NodeID*, and the client may compute the cookie as $\text{HMAC}(tok \parallel pk_s \parallel \text{NodeID}, epk_e \parallel c_S \parallel P_C \parallel M_C \parallel MAC_C)$. Thus, if a future token is ever revealed, past protocols using old tokens are still not distinguishable from random. Temporary tokens can be eschewed entirely if the server periodically updates its public key pk_s in a way that is not publicly reversible (e.g., by generating a brand new key or re-randomizing an existing key using secret randomness). However for lattice-based OKEMs, public keys are far larger than strictly necessary to achieve this strong(er) obfuscation property.

Padding distribution. Another difference to *pq-obfs* is that we enable an arbitrary distribution of padding to be used. Previously, the padding length (and consequently, the message length) was chosen uniformly at random from a fixed range. An arbitrary padding distribution provides more flexibility in traffic patterns, allowing, for example, a padding distribution that fixes message lengths to match the traffic pattern of another protocol.

Extra data. One may optionally place data encrypted under a key derived from K_S in place of (or in addition to) the padding of the initial message to the server. This is analogous to the *extra data* feature of *ntor* version 3 [Mat21]. We note that this extra data does not benefit from forward secrecy because it is encrypted under a key derived from the static KEM encapsulation alone.

5 Hybrid PAKE with Adaptive Security

In this section we describe a method for constructing a password-authenticated key exchange (PAKE) from an OKEM. When the OKEM has hybrid guarantees, then so does the resulting PAKE. We distinguish our approach from previous constructions [HR24, LL24, VJW⁺25] in two ways. First, our approach achieves adaptive, rather than static, security, since the underlying PAKE protocol is proven to realize the $\mathcal{F}_{\text{PAKE}}$ ideal functionality in the UC model [Can01] with adaptive corruptions. This answers the open question in [HR24] of whether such a scheme exists. Second, our approach achieves a new tradeoff between round complexity and runtime: previous hybrid PAKE constructions include efficient three-round protocols which use standard lattice-based KEMs, as well as more computationally intensive one-round protocols using isogenies; our protocol has two rounds, and has computational overhead that lies in between the two.

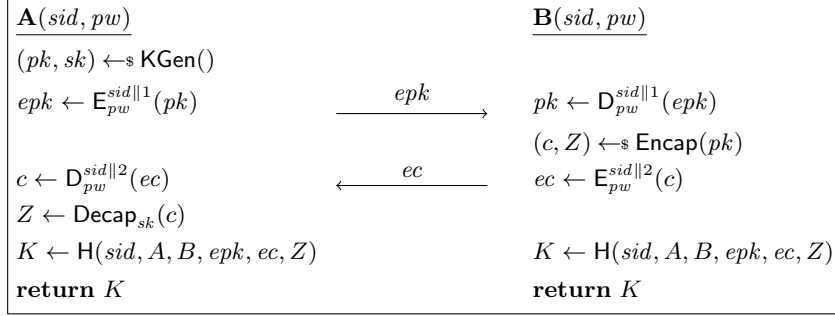


Figure 7: The CAKE PAKE [BCP⁺23]. (E, D) are the encryption and decryption algorithms of an ideal cipher. sid is the protocol execution’s session identifier, and A, B are the party identifiers.

5.1 CAKE

We first provide an overview of CAKE [BCP⁺23], an adaptively-secure PAKE that is generically constructed from an underlying KEM with certain properties. We observe that an OKEM with ciphertext uniformity, public key uniformity, and strong second pseudorandomness is sufficient to instantiate CAKE. Then, we construct a hybrid OKEM with unconditional public key uniformity. Together, this yields the first hybrid adaptively-secure PAKE.

Protocol. An overview of the CAKE protocol is given in Figure 7. The protocol begins with the initiator generating a fresh KEM keypair, encrypting the public key using an ideal cipher with pw as the key, and sending the result to the responder. The responder decrypts the public key, encapsulates to it, and encrypts the encapsulation using the ideal cipher with the same password. These ideal ciphers are domain-separated, represented in the diagram by two distinct session IDs in the superscript of the algorithm. Finally, the initiator decrypts and decapsulates. The session key is the hash of the KEM shared secret along with the protocol transcript.

Required KEM properties. CAKE requires its underlying KEM to have IND-CPA, strong pseudorandomness ([JRX25, Definitions 2.3–2.5]), anonymity (Definition B.2), and fuzziness (Definition B.1). When public keys and ciphertexts are bitstrings, anonymity is strictly weaker than ciphertext uniformity and public key uniformity (Definition 2.6) is equivalent to fuzziness (these claims are treated more rigorously in Appendix B). Finally, [JRX25] defines strong pseudorandomness as ciphertext uniformity plus strong second pseudorandomness.

Thus, CAKE can be instantiated with an OKEM with IND-CPA security, ciphertext uniformity, strong second pseudorandomness, and public key uniformity. Further, since encoded public keys are uniform-looking bitstrings, we may instantiate the ideal cipher with a tweakable wide block cipher, and use the tweak for domain separation. This is easier to implement and more efficient than, for example, a custom, constant-time, keyed permutation on the range $[0, q^{768})$, as suggested by the CAKE authors for ML-KEM-768 public keys.

⁸This tradeoff between strong obfuscation and DoS resistance in **obfs4** has been previously noted [Ang22].

Table 2: Comparison of different hybrid PAKE mechanisms at the 128-bit security level. Estimates of runtime are in cycles on an Intel Skylake CPU, and exclude communication time. We also include communication overhead and indicate whether a security proof exists in the adaptive corruption setting. **ParComb** and **SeqComb** are the parallel and sequential combiners defined in [HR24] and identically in [LL24]. **X-GA-PAKE** is assumed to use the CTIDH group action [BBC⁺21] and passwords of bitlength $\ell = 80$.

Scheme	Rounds	Runtime (cycles)	Comm. (kB)	Adaptive corruptions
ParComb[X-GA-PAKE, SPAKE2]	1	5.0×10^{10}	20.5	×
CAKE[OEINC[DHKEM-El2, StatFrodoKEM]]	2	1.3×10^8	153.8	✓
SeqComb[CAKE[FrodoKEM], SPAKE2]	3	2.3×10^7	19.4	×

5.2 Achieving Hybrid CAKE

Instantiating OEINC with, e.g., DHKEM-El2 (outer) and ML-Kameleon (inner), we obtain an OKEM with hybrid IND-CCA (and, hence, IND-CPA) security and ciphertext uniformity, but not public key uniformity. Recall that the public key uniformity of the combined OKEM is the *weakest* of the underlying public key uniformity properties. Thus, to construct CAKE with hybrid guarantees, we will need to combine two OKEMs with statistical public key uniformity. Since our outer KEM already has this property, it remains only to construct a post-quantum OKEM with statistical public key uniformity. No post-quantum OKEM we have mentioned so far achieves this; however, we are able to construct one based on unstructured LWE. We follow the technique hinted at in [HR24] for constructing a one-round lattice-based PAKE from (superpolynomial modulus) LWE.

Post-quantum statistical public key uniformity. As a starting point, we consider FrodoKEM [BCD⁺16], an LWE-based KEM and also a natural OKEM (its public keys and ciphertexts already perfectly pack into bitstrings). We will use the full IND-CCA-secure KEM, since strong pseudorandomness will indirectly require it. FrodoKEM is a *double*-LWE KEM: public keys and ciphertexts are both LWE samples. This technique, first described in [LP11], affords the scheme a meaningful improvement in both security guarantees and ciphertext size compared to a Regev-style scheme [Reg05], wherein only one of the aforementioned values is an LWE sample. Unfortunately, double-LWE schemes enjoy only computational public key uniformity, as that uniformity relies on the hardness of LWE.

We construct a KEM with statistical public key uniformity by simply undoing the optimization from [LP11]. Concretely, rather than defining the public key vector as an LWE sample $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s} + \mathbf{e}$, we define it as the matrix product $\mathbf{b} \leftarrow \mathbf{A}\mathbf{s}$. To retain security, we must modify the dimensions of $\mathbf{A}$ and $\mathbf{s}$. Rather than using a square matrix $\mathbf{A} \in \mathbb{Z}_q^{n \times n}$, we use a rectangular $\mathbf{A} \in \mathbb{Z}_q^{n \times n'}$ where $n' \geq n \log_2 q$. This is necessary to make public keys statistically close to uniform via the leftover hash lemma [HILL99, LP11]. We call this construction, after appropriate lifting to the full dimension of the public key and ciphertext, StatFrodoKEM.

Strong second pseudorandomness. Recall that OEINC achieves strong second pseudorandomness when outOKEM achieves statistical strong second pseudorandomness, and either outOKEM is IND-CPA-secure or inOKEM is (computationally) strong second pseudorandom. The authors of [JRX25] show hashed Diffie–Hellman achieves perfect strong pseudorandom-

ness, and it is the case that StatFrodoKEM is IND-CPA-secure. Thus it remains only to show that StatFrodoKEM achieves strong second pseudorandomness. Fortunately, the authors of [JRX25] also prove this property for Kyber [ABD⁺21],⁹ ML-KEM’s pre-standardization variant. While Kyber and StatFrodoKEM differ on hardness assumptions, they are nearly identical in structure, and, importantly, use the same variant of the Fujisaki-Okamoto transform, hashing the ciphertext into the final output key (something ML-KEM does not do). For these reasons, the arguments of [JRX25, Appendix C] go through without modification.

Performance tradeoffs. Due to the necessary parameter changes, ciphertexts grow by a factor of about $\log_2 q$. For StatFrodoKEM, this would result in 144kB ciphertexts in the level I parameter set, up from 9.6kB.¹⁰ This would also cause encapsulation and key generation time to increase by a similar factor.

CAKE with hybrid OKEM offers a new set of tradeoffs compared to existing hybrid PAKEs. We summarize these tradeoffs in Table 2. This construction is two rounds, rather than the three-round sequential combiners and one-round parallel combiners of [HR24, LL24]. In addition, it requires less overall computation than the one-round parallel combiner of [LL24], which relies on X-GA-PAKE, a supersingular isogeny group action PAKE [AEK⁺22] (as well as on a classical PAKE such as SPAKE2 [AP05]). Inferring from benchmarks on Intel Skylake CPUs [BBC⁺21, BCD⁺16] and assuming linear scaling for matrix multiplication, we estimate the full runtime of CAKE (ignoring communication costs) to be $384\times$ faster than the one-round alternative. This comes at the cost of an extra communication round and communication overhead of $7.5\times$.

Other hybrid PAKEs. A similar set of tradeoffs, albeit without security against adaptive corruptions, can be achieved by instantiating other KEM-based PAKEs such as OCAKE [BCP⁺23] and CHIC [ABJS24]¹¹ with our hybrid OKEM.

6 Future Work

We briefly describe some unresolved questions, which we leave to future work.

UDP-compatible obfuscated key exchange. A common payload limit for UDP packets on the Internet is 1472 bytes. This is not an issue for Wireguard, which uses elliptic curve Diffie–Hellman for its key exchange. However, when instantiated with some post-quantum KEMs, the first round of Drive1 exceeds this threshold. Since IPv4 fragmentation is frequently left disabled due to abuse concerns, there is no obvious way to perform key exchange over UDP while retaining obfuscation properties. Sequence numbers would violate obfuscation requirements, and using any pre-shared information to encrypt data (e.g., *NodeID*) would surrender strong obfuscation. Another challenge is managing retransmission as a result of fragmentation. In addition, stream IDs are necessary for users to be able to *roam*, i.e., maintain a connection while changing their IP or outgoing port. Unfortunately, embedding a stream ID would break obfuscation entirely, and using a symmetric key would either break strong obfuscation, or require the server to do trial decryptions over its set

⁹Specifically, Kyber’s third round submission to the NIST PQC competition, modified to not perform coefficient compression.

¹⁰There may be routes to improve ciphertext size using ideal-lattice-based schemes and leftover hash analogues [LPR13, DSGKS20], but we leave this to future work.

¹¹CHIC requires the KEM to be *splittable*, i.e., that all public keys contain a uniform bitstring. ML-KEM is already splittable, since the public matrix seed is a uniform bitstring. In addition, DHKEMs are trivially splittable: we can simply add a sample from $\{0, 1\}^\lambda$ to the end of any DHKEM public key. Since combining splittable OKEMs yields a splittable OKEM (via concatenation), we can instantiate hybrid CHIC.

of session keys. It is thus an open problem whether these convenient properties can be recovered in fully encrypted protocols with post-quantum key exchange.

Hybrid ANO-CCA KEMs. Our combiner OEINC immediately yields a KEM providing hybrid anonymity (ANO-CCA), since SPR-CCA implies ANO-CCA [Xag22]. While SPR-CCA security provides an avenue for proving anonymity, some of the properties we required in our combiner (in particular, *strong* ciphertext uniformity) arise specifically to enable the proof of SPR-CCA. Thus, it is interesting to ask whether a hybrid ANO-CCA KEM (or anonymous KEM combiner) can be constructed with weaker assumptions on the two input KEMs, perhaps using a method other than SPR-CCA to prove anonymity.

Acknowledgements

We thank the anonymous reviewers for their helpful comments. D.S. is supported by Natural Sciences and Engineering Research Council of Canada (NSERC) Discovery grant RGPIN-2022-03187 and NSERC Alliance grant ALLRP 578463-22.

References

- [ABD⁺21] Roberto Avanzi, Joppe Bos, Léo Ducas, Eike Kiltz, Tancrede Lepoint, Vadim Lyubashevsky, John M Schanck, Peter Schwabe, Gregor Seiler, and Damien Stehlé. CRYSTALS-Kyber: Algorithm Specifications And Supporting Documentation (version 3.02). August 2021. URL: <https://pq-crystals.org/kyber/data/kyber-specification-round3-20210804.pdf>.
- [ABH⁺21] Joël Alwen, Bruno Blanchet, Eduard Hauck, Eike Kiltz, Benjamin Lipp, and Doreen Riepel. Analysing the HPKE standard. In Anne Canteaut and François-Xavier Standaert, editors, *EUROCRYPT 2021, Part I*, volume 12696 of *LNCS*, pages 87–116. Springer, Cham, October 2021. doi:10.1007/978-3-030-77870-5_4.
- [ABJS24] Afonso Arriaga, Manuel Barbosa, Stanislaw Jarecki, and Marjan Skrobot. C’est très CHIC: A compact password-authenticated key exchange from lattice-based KEM. In Kai-Min Chung and Yu Sasaki, editors, *Advances in Cryptology – ASIACRYPT 2024*, pages 3–33, Singapore, 2024. Springer Nature Singapore.
- [ABR01] Michel Abdalla, Mihir Bellare, and Phillip Rogaway. The oracle Diffie-Hellman assumptions and an analysis of DHIES. In David Naccache, editor, *CT-RSA 2001*, volume 2020 of *LNCS*, pages 143–158. Springer, Berlin, Heidelberg, April 2001. doi:10.1007/3-540-45353-9_12.
- [AEK⁺22] Michel Abdalla, Thorsten Eisenhofer, Eike Kiltz, Sabrina Kunzweiler, and Doreen Riepel. Password-authenticated key exchange from group actions. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 699–728. Springer, Cham, August 2022. doi:10.1007/978-3-031-15979-4_24.
- [And12] Nate Anderson. Stakeout: how the FBI tracked and busted a Chicago Anon, March 2012. <https://arstechnica.com/tech-policy/2012/03/stakeout-how-the-fbi-tracked-and-busted-a-chicago-anon/>.

- [Ang22] Yawning Angel. Comment in Issue “Improve the obfs4 obfuscation protocol (#30716)” · The Tor Project / Anti-censorship / Pluggable Transports / lyrebird · GitLab, August 2022. URL: https://gitlab.torproject.org/tpo/anti-censorship/pluggable-transports/lyrebird/-/issues/30716#note_2833105.
- [AP05] Michel Abdalla and David Pointcheval. Simple password-based encrypted key exchange protocols. In Alfred Menezes, editor, *CT-RSA 2005*, volume 3376 of *LNCS*, pages 191–208. Springer, Berlin, Heidelberg, February 2005. doi:10.1007/978-3-540-30574-3_14.
- [BBB⁺24] Daniel J. Bernstein, Karthikeyan Bhargavan, Shivam Bhasin, Anupam Chattopadhyay, Tee Kiah Chia, Matthias J. Kannwischer, Franziskus Kiefer, Thales Paiva, Prasanna Ravi, and Goutam Tamvada. KyberSlash: Exploiting secret-dependent division timings in kyber implementations. Cryptology ePrint Archive, Report 2024/1049, 2024. URL: <https://eprint.iacr.org/2024/1049>.
- [BBC⁺21] Gustavo Banegas, Daniel J. Bernstein, Fabio Campos, Tung Chou, Tanja Lange, Michael Meyer, Benjamin Smith, and Jana Sotáková. CTIDH: faster constant-time CSIDH. *IACR TCHES*, 2021(4):351–387, 2021. URL: <https://tches.iacr.org/index.php/TCHES/article/view/9069>, doi:10.46586/tches.v2021.i4.351-387.
- [BBC⁺22] John Baena, Pierre Briaud, Daniel Cabarcas, Ray A. Perlner, Daniel Smith-Tone, and Javier A. Verbel. Improving support-minors rank attacks: Applications to GeMSS and rainbow. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part III*, volume 13509 of *LNCS*, pages 376–405. Springer, Cham, August 2022. doi:10.1007/978-3-031-15982-4_13.
- [BBDP01] Mihir Bellare, Alexandra Boldyreva, Anand Desai, and David Pointcheval. Key-privacy in public-key encryption. In Colin Boyd, editor, *ASIACRYPT 2001*, volume 2248 of *LNCS*, pages 566–582. Springer, Berlin, Heidelberg, December 2001. doi:10.1007/3-540-45682-1_33.
- [BBF⁺19] Nina Bindel, Jacqueline Brendel, Marc Fischlin, Brian Goncalves, and Douglas Stebila. Hybrid key encapsulation mechanisms and authenticated key exchange. In Jintai Ding and Rainer Steinwandt, editors, *Post-Quantum Cryptography - 10th International Conference, PQCrypto 2019*, pages 206–226. Springer, Cham, 2019. doi:10.1007/978-3-030-25510-7_12.
- [BBSG23] Matilda Backendal, Mihir Bellare, Felix Günther, and Matteo Scarlata. When messages are keys: Is HMAC a dual-PRF? In Helena Handschuh and Anna Lysyanskaya, editors, *CRYPTO 2023, Part III*, volume 14083 of *LNCS*, pages 661–693. Springer, Cham, August 2023. doi:10.1007/978-3-031-38548-3_22.
- [BBLW22] R. Barnes, K. Bhargavan, B. Lipp, and C. Wood. Hybrid Public Key Encryption. RFC 9180 (Informational), February 2022. URL: <https://www.rfc-editor.org/rfc/rfc9180.txt>, doi:10.17487/RFC9180.

- [BCD⁺16] Joppe W. Bos, Craig Costello, Léo Ducas, Ilya Mironov, Michael Naehrig, Valeria Nikolaenko, Ananth Raghunathan, and Douglas Stebila. Frodo: Take off the ring! Practical, quantum-secure key exchange from LWE. In Edgar R. Weippl, Stefan Katzenbeisser, Christopher Kruegel, Andrew C. Myers, and Shai Halevi, editors, *ACM CCS 2016*, pages 1006–1018. ACM Press, October 2016. doi:10.1145/2976749.2978425.
- [BCD⁺24] Manuel Barbosa, Deirdre Connolly, João Diogo Duarte, Aaron Kaiser, Peter Schwabe, Karoline Varner, and Bas Westerbaan. X-wing. *CiC*, 1(1):21, 2024. doi:10.62056/a3qj89n4e.
- [BCK96] Mihir Bellare, Ran Canetti, and Hugo Krawczyk. Keying hash functions for message authentication. In Neal Koblitz, editor, *CRYPTO’96*, volume 1109 of *LNCS*, pages 1–15. Springer, Berlin, Heidelberg, August 1996. doi:10.1007/3-540-68697-5_1.
- [BCP⁺23] Hugo Beguinet, Céline Chevalier, David Pointcheval, Thomas Ricosset, and Mélissa Rossi. GeT a CAKE: Generic transformations from key encapsulation mechanisms to password authenticated key exchanges. In Mehdi Tibouchi and Xiaofeng Wang, editors, *ACNS 23International Conference on Applied Cryptography and Network Security, Part II*, volume 13906 of *LNCS*, pages 516–538. Springer, Cham, June 2023. doi:10.1007/978-3-031-33491-7_19.
- [Beu22] Ward Beullens. Breaking rainbow takes a weekend on a laptop. In Yevgeniy Dodis and Thomas Shrimpton, editors, *CRYPTO 2022, Part II*, volume 13508 of *LNCS*, pages 464–479. Springer, Cham, August 2022. doi:10.1007/978-3-031-15979-4_16.
- [BFGJ17] Jacqueline Brendel, Marc Fischlin, Felix Günther, and Christian Janson. PRF-ODH: Relations, instantiations, and impossibility results. In Jonathan Katz and Hovav Shacham, editors, *CRYPTO 2017, Part III*, volume 10403 of *LNCS*, pages 651–681. Springer, Cham, August 2017. doi:10.1007/978-3-319-63697-9_22.
- [BHKL13] Daniel J. Bernstein, Mike Hamburg, Anna Krasnova, and Tanja Lange. Elligator: elliptic-curve points indistinguishable from uniform random strings. In Ahmad-Reza Sadeghi, Virgil D. Gligor, and Moti Yung, editors, *ACM CCS 2013*, pages 967–980. ACM Press, November 2013. doi:10.1145/2508859.2516734.
- [BPR00] Mihir Bellare, David Pointcheval, and Phillip Rogaway. Authenticated key exchange secure against dictionary attacks. In Bart Preneel, editor, *EURO-CRYPT 2000*, volume 1807 of *LNCS*, pages 139–155. Springer, Berlin, Heidelberg, May 2000. doi:10.1007/3-540-45539-6_11.
- [BR94] Mihir Bellare and Phillip Rogaway. Entity authentication and key distribution. In Douglas R. Stinson, editor, *CRYPTO’93*, volume 773 of *LNCS*, pages 232–249. Springer, Berlin, Heidelberg, August 1994. doi:10.1007/3-540-48329-2_21.
- [Can01] Ran Canetti. Universally composable security: A new paradigm for cryptographic protocols. In *42nd FOCS*, pages 136–145. IEEE Computer Society Press, October 2001. doi:10.1109/SFCS.2001.959888.

- [CD23] Wouter Castryck and Thomas Decru. An efficient key recovery attack on SIDH. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 423–447. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_15.
- [Con25] Deirdre Connolly. Hybrid PQ/T key encapsulation mechanisms. Internet Draft draft-irtf-cfrg-hybrid-kems-03, Internet Engineering Task Force, February 2025. URL: <https://datatracker.ietf.org/doc/draft-irtf-cfrg-hybrid-kems-03>.
- [DFGS15] Benjamin Dowling, Marc Fischlin, Felix Günther, and Douglas Stebila. A cryptographic analysis of the TLS 1.3 handshake protocol candidates. In Indrajit Ray, Ninghui Li, and Christopher Kruegel, editors, *ACM CCS 2015*, pages 1197–1210. ACM Press, October 2015. doi:10.1145/2810103.2813653.
- [DHRR22a] Benjamin Dowling, Eduard Hauck, Doreen Riepel, and Paul Rösler. Strongly anonymous ratcheted key exchange. Cryptology ePrint Archive, Report 2022/1187, 2022. URL: <https://eprint.iacr.org/2022/1187>.
- [DHRR22b] Benjamin Dowling, Eduard Hauck, Doreen Riepel, and Paul Rösler. Strongly anonymous ratcheted key exchange. In Shweta Agrawal and Dongdai Lin, editors, *ASIACRYPT 2022, Part III*, volume 13793 of *LNCS*, pages 119–150. Springer, Cham, December 2022. doi:10.1007/978-3-031-22969-5_5.
- [DKRV18] Jan-Pieter D’Anvers, Angshuman Karmakar, Sujoy Sinha Roy, and Frederik Vercauteren. Saber: Module-LWR based key exchange, CPA-secure encryption and CCA-secure KEM. In Antoine Joux, Abderrahmane Nitaj, and Tajjeeddine Rachidi, editors, *AFRICACRYPT 18*, volume 10831 of *LNCS*, pages 282–305. Springer, Cham, May 2018. doi:10.1007/978-3-319-89339-6_16.
- [Don17] Jason A. Donenfeld. WireGuard: Next generation kernel network tunnel. In *NDSS 2017*. The Internet Society, February / March 2017. doi:10.14722/ndss.2017.23160.
- [DSGKS20] Dana Dachman-Soled, Huijing Gong, Mukul Kulkarni, and Aria Shahverdi. Towards a ring analogue of the leftover hash lemma. *Journal of Mathematical Cryptology*, 15:87–110, 2020. URL: <https://api.semanticscholar.org/CorpusID:227129476>.
- [FJ24] Ellis Fenske and Aaron Johnson. Bytes to schlep? Use a FEP: Hiding protocol metadata with fully encrypted protocols. In *ACM CCS 2024, CCS ’24*, page 1982–1996, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690198.
- [FO99] Eiichiro Fujisaki and Tatsuaki Okamoto. Secure integration of asymmetric and symmetric encryption schemes. In Michael J. Wiener, editor, *CRYPTO’99*, volume 1666 of *LNCS*, pages 537–554. Springer, Berlin, Heidelberg, August 1999. doi:10.1007/3-540-48405-1_34.
- [Fre22] French Cybersecurity Agency. ANSSI views on the post-quantum cryptography transition, March 2022. https://cyber.gouv.fr/sites/default/files/document/EN_Position.pdf.

- [Ger24] German Federal Office for Information Security. BSI TR-02102-1, cryptographic mechanisms: Recommendations and key lengths, February 2024. Version 2024-01, <https://www.bsi.bund.de/SharedDocs/Downloads/EN/BSI/Publications/TechGuidelines/TG02102/BSI-TR-02102-1.pdf>.
- [GHJ⁺22] Qian Guo, Clemens Hlauschek, Thomas Johansson, Norman Lahr, Alexander Nilsson, and Robin Leander Schröder. Don't reject this: Key-recovery timing attacks due to rejection-sampling in HQC and BIKE. *IACR TCHES*, 2022(3):223–263, 2022. doi:10.46586/tches.v2022.i3.223–263.
- [GHJ25] Phillip Gajland, Vincent Hwang, and Jonas Janneck. Shadowfax: A deniability-preserving AKEM combiner. Cryptology ePrint Archive, Paper 2025/154, 2025. URL: <https://eprint.iacr.org/2025/154>.
- [GHP18] Federico Giacon, Felix Heuer, and Bertram Poettering. KEM combiners. In Michel Abdalla and Ricardo Dahab, editors, *PKC 2018, Part I*, volume 10769 of *LNCS*, pages 190–218. Springer, Cham, March 2018. doi:10.1007/978-3-319-76578-5_7.
- [GJK21] Yanqi Gu, Stanislaw Jarecki, and Hugo Krawczyk. KHAPE: Asymmetric PAKE from key-hiding key exchange. In Tal Malkin and Chris Peikert, editors, *CRYPTO 2021, Part IV*, volume 12828 of *LNCS*, pages 701–730, Virtual Event, August 2021. Springer, Cham. doi:10.1007/978-3-030-84259-8_24.
- [GJN20] Qian Guo, Thomas Johansson, and Alexander Nilsson. A key-recovery timing attack on post-quantum primitives using the Fujisaki-Okamoto transformation and its application on FrodoKEM. In Daniele Micciancio and Thomas Ristenpart, editors, *CRYPTO 2020, Part II*, volume 12171 of *LNCS*, pages 359–386. Springer, Cham, August 2020. doi:10.1007/978-3-030-56880-1_13.
- [GMP22] Paul Grubbs, Varun Maram, and Kenneth G. Paterson. Anonymous, robust post-quantum public key encryption. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 402–432. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_15.
- [GSU13] Ian Goldberg, Douglas Stebila, and Berkant Ustaoglu. Anonymity and one-way authentication in key exchange protocols. *DCC*, 67(2):245–269, 2013. doi:10.1007/s10623-011-9604-z.
- [GSV24] Felix Günther, Douglas Stebila, and Shannon Veitch. Obfuscated key exchange. In *ACM CCS 2024, CCS '24*, page 2385–2399, New York, NY, USA, 2024. Association for Computing Machinery. doi:10.1145/3658644.3690220.
- [Hal24] Dan Hall. Zero Trust WARP: tunneling with a MASQUE, March 2024. <https://blog.cloudflare.com/zero-trust-warp-with-a-masque>.
- [HILL99] Johan Håstad, Russell Impagliazzo, Leonid A. Levin, and Michael Luby. A pseudorandom generator from any one-way function. *SIAM Journal on Computing*, 28(4):1364–1396, 1999.
- [HR24] Julia Hesse and Michael Rosenberg. PAKE combiners and efficient post-quantum instantiations. Cryptology ePrint Archive, Paper 2024/1621, 2024. URL: <https://eprint.iacr.org/2024/1621>.

- [HSC⁺23] Senyang Huang, Rui Qi Sim, Chitchanok Chuengsatiansup, Qian Guo, and Thomas Johansson. Cache-timing attack against HQC. *IACR TCHES*, 2023(3):136–163, 2023. doi:10.46586/tches.v2023.i3.136-163.
- [IT21] J. Iyengar (Ed.) and M. Thomson (Ed.). QUIC: A UDP-Based Multiplexed and Secure Transport. RFC 9000 (Proposed Standard), May 2021. URL: <https://www.rfc-editor.org/rfc/rfc9000.txt>, doi:10.17487/RFC9000.
- [JMZ23] Haodong Jiang, Zhi Ma, and Zhenfeng Zhang. Post-quantum security of key encapsulation mechanism against CCA attacks with a single decapsulation query. In Jian Guo and Ron Steinfeld, editors, *ASIACRYPT 2023, Part IV*, volume 14441 of *LNCS*, pages 434–468. Springer, Singapore, December 2023. doi:10.1007/978-981-99-8730-6_14.
- [JRX24] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? Cryptology ePrint Archive, Report 2024/324, 2024. URL: <https://eprint.iacr.org/2024/324>.
- [JRX25] Jake Januzelli, Lawrence Roy, and Jiayu Xu. Under what conditions is encrypted key exchange actually secure? In Serge Fehr and Pierre-Alain Fouque, editors, *Advances in Cryptology – EUROCRYPT 2025*, pages 451–481, Cham, 2025. Springer Nature Switzerland.
- [KBC97] H. Krawczyk, M. Bellare, and R. Canetti. HMAC: Keyed-Hashing for Message Authentication. RFC 2104 (Informational), February 1997. Updated by RFC 6151. URL: <https://www.rfc-editor.org/rfc/rfc2104.txt>, doi:10.17487/RFC2104.
- [KE10] H. Krawczyk and P. Eronen. HMAC-based Extract-and-Expand Key Derivation Function (HKDF). RFC 5869 (Informational), May 2010. URL: <https://www.rfc-editor.org/rfc/rfc5869.txt>, doi:10.17487/RFC5869.
- [Kra10] Hugo Krawczyk. Cryptographic extraction and key derivation: The HKDF scheme. In Tal Rabin, editor, *CRYPTO 2010*, volume 6223 of *LNCS*, pages 631–648. Springer, Berlin, Heidelberg, August 2010. doi:10.1007/978-3-642-14623-7_34.
- [LHT16] A. Langley, M. Hamburg, and S. Turner. Elliptic Curves for Security. RFC 7748 (Informational), January 2016. URL: <https://www.rfc-editor.org/rfc/rfc7748.txt>, doi:10.17487/RFC7748.
- [LL24] You Lyu and Shengli Liu. Hybrid password authentication key exchange in the UC framework. Cryptology ePrint Archive, Paper 2024/1630, 2024. URL: <https://eprint.iacr.org/2024/1630>.
- [LP11] Richard Lindner and Chris Peikert. Better key sizes (and attacks) for LWE-based encryption. In Aggelos Kiayias, editor, *CT-RSA 2011*, volume 6558 of *LNCS*, pages 319–339. Springer, Berlin, Heidelberg, February 2011. doi:10.1007/978-3-642-19074-2_21.
- [LPR13] Vadim Lyubashevsky, Chris Peikert, and Oded Regev. A toolkit for ring-LWE cryptography. In Thomas Johansson and Phong Q. Nguyen, editors,

- EUROCRYPT 2013*, volume 7881 of *LNCS*, pages 35–54. Springer, Berlin, Heidelberg, May 2013. doi:10.1007/978-3-642-38348-9_3.
- [Mat21] Nick Mathewson. Ntor protocol with extra data, version 3. <https://spec.torproject.org/proposals/332-ntor-v3-with-extra-data.html>, 2021.
- [Moh10] Payman Mohassel. A closer look at anonymity and robustness in encryption schemes. In Masayuki Abe, editor, *ASIACRYPT 2010*, volume 6477 of *LNCS*, pages 501–518. Springer, Berlin, Heidelberg, December 2010. doi:10.1007/978-3-642-17373-8_29.
- [MRR20] Ian McQuoid, Mike Rosulek, and Lawrence Roy. Minimal symmetric PAKE and 1-out-of-N OT from programmable-once public functions. In Jay Ligatti, Xinming Ou, Jonathan Katz, and Giovanni Vigna, editors, *ACM CCS 2020*, pages 425–442. ACM Press, November 2020. doi:10.1145/3372297.3417870.
- [MX23] Varun Maram and Keita Xagawa. Post-quantum anonymity of Kyber. In Alexandra Boldyreva and Vladimir Kolesnikov, editors, *PKC 2023, Part I*, volume 13940 of *LNCS*, pages 3–35. Springer, Cham, May 2023. doi:10.1007/978-3-031-31368-4_1.
- [NAB⁺20] Michael Naehrig, Erdem Alkim, Joppe Bos, Léo Ducas, Karen Easterbrook, Brian LaMacchia, Patrick Longa, Ilya Mironov, Valeria Nikolaenko, Christopher Peikert, Ananth Raghunathan, and Douglas Stebila. FrodoKEM. Technical report, National Institute of Standards and Technology, 2020. available at <https://csrc.nist.gov/projects/post-quantum-cryptography/post-quantum-cryptography-standardization/round-3-submissions>.
- [NIS23] NIST. Digital signature standard (dss), February 2023. FIPS 186-5. <https://doi.org/10.6028/NIST.FIPS.186-5>.
- [Rad] Cloudflare Radar. Adoption & Usage Worldwide | Cloudflare Radar. URL: <https://radar.cloudflare.com/adoption-and-usage#post-quantum-encryption-adoption>.
- [Reg05] Oded Regev. On lattices, learning with errors, random linear codes, and cryptography. In Harold N. Gabow and Ronald Fagin, editors, *37th ACM STOC*, pages 84–93. ACM Press, May 2005. doi:10.1145/1060590.1060603.
- [ROSW24] Eric Rescorla, Kazuho Oku, Nick Sullivan, and Christopher A. Wood. TLS Encrypted Client Hello. Internet Draft draft-ietf-tls-esni-22, Internet Engineering Task Force, September 2024. <https://datatracker.ietf.org/doc/draft-ietf-tls-esni-22>.
- [SGJ23] Bruno Freitas Dos Santos, Yanqi Gu, and Stanislaw Jarecki. Randomized half-ideal cipher on groups with applications to UC (a)PAKE. In Carmit Hazay and Martijn Stam, editors, *EUROCRYPT 2023, Part V*, volume 14008 of *LNCS*, pages 128–156. Springer, Cham, April 2023. doi:10.1007/978-3-031-30589-4_5.
- [sha23] Shadowsocks. <https://shadowsocks.org/doc/what-is-shadowsocks.html>, 2023.

- [SP22] Benjamin M. Schwartz and Christopher Patton. The Pseudorandom Extension for cTLS. Internet Draft draft-cpbs-pseudorandom-ctls-01, Internet Engineering Task Force, April 2022. <https://datatracker.ietf.org/doc/draft-cpbs-pseudorandom-ctls-01>.
- [SSL20] Sven Schäge, Jörg Schwenk, and Sebastian Lauer. Privacy-preserving authenticated key exchange and the case of IKEv2. In Aggelos Kiayias, Markulf Kohlweiss, Petros Wallden, and Vassilis Zikas, editors, *PKC 2020, Part II*, volume 12111 of *LNCS*, pages 567–596. Springer, Cham, May 2020. doi:10.1007/978-3-030-45388-6_20.
- [The19] The Tor Project. obfs4 (the obfourscator), protocol specification, version c0898c2d. <https://gitlab.torproject.org/tpo/anti-censorship/pluggable-transport/lyrebird/-/blob/main/doc/obfs4-spec.txt>, 2019.
- [Tib14] Mehdi Tibouchi. Elligator squared: Uniform points on elliptic curves of prime order as uniform random strings. In Nicolas Christin and Reihaneh Safavi-Naini, editors, *FC 2014*, volume 8437 of *LNCS*, pages 139–156. Springer, Berlin, Heidelberg, March 2014. doi:10.1007/978-3-662-45472-5_10.
- [Vai21] Loup Vaillant. Elligator - Hiding Key Exchanges, 2021. URL: <https://elligator.org/key-exchange>.
- [VJW⁺25] Jelle Vos, Stanislaw Jarecki, Christopher A. Wood, Cathie Yun, Steve Myers, and Yannick Sierra. A hybrid asymmetric password-authenticated key exchange in the random oracle model. Cryptology ePrint Archive, Paper 2025/1343, 2025. URL: <https://eprint.iacr.org/2025/1343>.
- [vme19] VMess. <https://www.v2ray.com/en/configuration/protocols/vmess.html>, 2019.
- [Wes24] Bas Westerbaan. The state of the post-quantum Internet, March 2024. <https://blog.cloudflare.com/pq-2024>.
- [WJJS23] Ryan Wails, Rob Jansen, Aaron Johnson, and Micah Sherr. Proteus: Programmable protocols for censorship circumvention. In *Free and Open Communication on the Internet*, number 2, pages 50–66, 2023.
- [WMSM11] Andrew M. White, Austin R. Matthews, Kevin Z. Snow, and Fabian Monrose. Phonotactic reconstruction of encrypted VoIP conversations: Hookt on fon-iks. In *2011 IEEE Symposium on Security and Privacy*, pages 3–18. IEEE Computer Society Press, May 2011. doi:10.1109/SP.2011.34.
- [WSS⁺23] Mingshi Wu, Jackson Sippe, Danesh Sivakumar, Jack Burg, Peter Anderson, Xiaokang Wang, Kevin Bock, Amir Houmansadr, Dave Levin, and Eric Wustrow. How the great firewall of china detects and blocks fully encrypted traffic. In Joseph A. Calandrino and Carmela Troncoso, editors, *USENIX Security 2023*, pages 2653–2670. USENIX Association, August 2023.

- [WW24] Bas Westerbaan and Christopher A. Wood. X25519Kyber768Draft00 hybrid post-quantum KEM for HPKE. Internet Draft draft-westerbaan-cfrg-hpke-xyber768d00-03, Internet Engineering Task Force, May 2024. Num Pages: 20. URL: <https://datatracker.ietf.org/doc/draft-westerbaan-cfrg-hpke-xyber768d00>.
- [Xag22] Keita Xagawa. Anonymity of NIST PQC round 3 KEMs. In Orr Dunkelman and Stefan Dziembowski, editors, *EUROCRYPT 2022, Part III*, volume 13277 of *LNCS*, pages 551–581. Springer, Cham, May / June 2022. doi:10.1007/978-3-031-07082-2_20.
- [XAR⁺24] Diwen Xue, Anna Ablove, Reethika Ramesh, Grace Kwak Danciu, and Roya Ensafi. Bridging barriers: A survey of challenges and priorities in the censorship circumvention landscape. In Davide Balzarotti and Wenyan Xu, editors, *USENIX Security 2024*. USENIX Association, August 2024.

A Additional Definitions

Definition A.1 (Pseudorandom generator). *We define the pseudorandom generator (PRG) advantage of an adversary $\mathcal{A}$ against a function $G: \mathcal{S} \rightarrow \mathcal{R}$ as*

$$\text{Adv}_G^{\text{PRG}}(\mathcal{A}) := \Pr[\mathcal{A}(r_1) \Rightarrow 1 \mid s \leftarrow \mathcal{S}, r \leftarrow G(s)] - \Pr[\mathcal{A}(r_0) \Rightarrow 1 \mid r \leftarrow \mathcal{R}].$$

Definition A.2 (Pseudorandom function). *We define the pseudorandom function (PRF) advantage of an adversary $\mathcal{A}$ against a function $F: \mathcal{X} \times \mathcal{Y} \rightarrow \mathcal{Z}$ as*

$$\begin{aligned} \text{Adv}_F^{\text{PRF}}(\mathcal{A}) := & \Pr\left[\mathcal{A}^{F(k, \cdot)}() \Rightarrow 1 \mid k \leftarrow \{0, 1\}^\kappa\right] \\ & - \Pr\left[\mathcal{A}^{R(\cdot)}() \Rightarrow 1 \mid R \leftarrow \{\text{all functions: } \mathcal{Y} \rightarrow \mathcal{Z}\}\right]. \end{aligned}$$

Definition A.3 (One-time indistinguishability from random strings (OT-IND\$)). *Let SE be a symmetric encryption scheme with key space $\mathcal{K}$ and where the length $|c| = \ell(|m|)$ of the ciphertext output by encryption only depends on the input message’s length $|m|$. We define the OT-IND\$ security game as follows.*

1. The challenger samples $b \leftarrow \{0, 1\}$ and $K \leftarrow \mathcal{K}$.
2. $\mathcal{A}$ provides a chosen message m to the challenger, who returns either $\text{SE.Enc}(K, m)$ if $b = 1$ or a random string $c \leftarrow \{0, 1\}^{\ell(|m|)}$ if $b = 0$.
3. $\mathcal{A}$ outputs a guess $b' \in \{0, 1\}$.

We define the OT-IND\$ advantage of an adversary $\mathcal{A}$ as

$$\text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{A}) := 2 \cdot \Pr[b' = b] - 1.$$

Definition A.4 (nm-PRF-ODH assumption). *Let $\mathbb{G}$ be a group of prime order q with generator g , and $F: \mathbb{G} \times \{0, 1\}^* \rightarrow \{0, 1\}^\ell$ for $\ell \in \mathbb{N}$ be a function.*

We define the nm-PRF-ODH security game as follows.

1. The challenger samples $b \leftarrow \{0, 1\}$, $u, v \leftarrow \mathbb{Z}_q$, and provides $\mathbb{G}$, g , g^u , and g^v to $\mathcal{A}$, who responds with a challenge label x^* . $\mathcal{A}$ has access to an oracle $\text{ODH}_v(\cdot, \cdot)$ that on input $S \in \mathbb{G}, x \in \{0, 1\}^*$ returns $y \leftarrow F(S^v, x)$.
2. The challenger computes $y_0 = F(g^{uv}, x^*)$ and samples $y_1 \leftarrow \{0, 1\}^\ell$ uniformly at random, providing y_b to $\mathcal{A}$.
3. $\mathcal{A}$ again has access to oracle $\text{ODH}_v(\cdot, \cdot)$ that on input $S \in \mathbb{G}, x \in \{0, 1\}^*$ returns $y \leftarrow F(S^v, x)$, except that it disallows the input $(S, x) = (g^u, x^*)$.
4. Eventually, $\mathcal{A}$ stops and outputs a guess $b' \in \{0, 1\}$.

We define the nm-PRF-ODH advantage function as

$$\text{Adv}_{\mathbb{F}, \mathbb{G}}^{\text{nm-PRF-ODH}}(\mathcal{A}) := 2 \cdot \Pr[b' = b] - 1.$$

B Relations Between Anonymity Notions

Several of the security properties that we have proven for our obfuscated KEM combiner — strong pseudorandomness (SPR-CCA), ciphertext uniformity, and public key uniformity — relate to existing notions of anonymity and pseudorandomness of KEMs. We compare the goals of these notions and discuss how they relate to one another. [JRX24, Table 3] provides a nice summary of PAKE-related notions of pseudorandomness and anonymity. We reiterate their observations and extend them to include additional properties.

The *strong pseudorandomness* (SPR-CCA) property that we have shown for our KEM combiner originates in [Xag22] as a tool for proving anonymity of a KEM. In particular, [Xag22, Theorem 2.5] shows that SPR-CCA implies ANO-CCA (which we recap in Figure 8), where ANO-CCA of a KEM captures the idea that an adversary cannot distinguish which public key corresponds to a given ciphertext, shared secret pair. This definition of anonymity for KEMs was introduced in [GMP22] and an analogous definition is given for PKE schemes. The ANO-CCA property for PKE schemes from [GMP22] is equivalent to the IK-CCA property of PKE schemes introduced in [BBDP01]. IK-CCA, also called “key-privacy” or “indistinguishability of keys,” similarly captures the idea that an adversary cannot distinguish which of two public keys corresponds to a given challenge ciphertext. This idea of anonymity of PKE schemes is, again, similarly introduced by Mohassel [Moh10] for *general encryption schemes* (capturing PKE and IBE schemes). The definition in [Moh10] is similar to that of ANO/IK-CCA, except it also provides the adversary access to a number of public keys, and some secret keys that do not allow for trivial wins. This is equivalent to the ANO/IK-CCA definition up to a factor depending on the number of secret key reveals [Moh10]. Finally, all of these definitions of anonymity for PKE schemes are related to the *ciphertext anonymity* property introduced in [DHRR22a, Definition 14] for updatable randomizable PKE schemes (note that the text in the introduction and Section 5 refers to this property as *ciphertext anonymity*, while in the definition it is called *anonymity*). This definition of anonymity is a natural analogue of ANO/IK-CCA in the setting of updatable randomizable PKE schemes (i.e., additionally allows the adversary access to public keys and an oracle which re-randomizes public keys).

It follows directly from these results that our OKEM combiner provides ANO-CCA security (Figure 8). Therefore, instantiating our OKEM combiner with a DH-based OKEM and a post-quantum OKEM provides the first *hybrid anonymous KEM*. That is, the anonymity of

$\mathcal{G}_K^{\text{ANO-CCA}}(\mathcal{A})$:	$\text{Decap}_x(sk, c)$:
1 $b \leftarrow \mathcal{S}\{0, 1\}$	7 if $c = x$ then return $\perp$
2 $(pk_0, sk_0) \leftarrow \mathcal{S}\text{KGen}()$	8 $K \leftarrow \text{Decap}(sk, c)$
3 $(pk_1, sk_1) \leftarrow \mathcal{S}\text{KGen}()$	9 return K
4 $(c^*, K^*) \leftarrow \mathcal{S}\text{Encap}(pk_b)$	
5 $b' \leftarrow \mathcal{A}^{\text{Decap}_{c^*}(sk_b, \cdot)}(pk_0, pk_1, (c^*, K^*))$	
6 return $\llbracket b = b' \rrbracket$	

Figure 8: Security game for ANO-CCA of a KEM $K = (\text{KGen}, \text{Encap}, \text{Decap})$.

the scheme relies on the stronger underlying security assumption of the input KEMs. Prior hybrid KEMs, e.g., X-Wing [BCD⁺24] and those resulting from the generic constructions in [GHP18], do not achieve such a hybrid anonymity property because they simply concatenate the two ciphertexts before returning them in **Encap**. This means that being able to correlate one of the ciphertexts with its corresponding public key is enough to break the anonymity of the scheme. For example, since the anonymity of ML-KEM relies on an underlying module LWE assumption [MX23], then breaking this assumption would be sufficient to violate the anonymity of any prior hybrid construction instantiated with ML-KEM.

The notions of *public key uniformity* and *ciphertext uniformity* that we adopt from [GSV24] are similar to existing definitions from the literature on password-authenticated key exchange (PAKE). In particular, [SGJ23] says that a KEM has *uniform public keys* if the distribution of public keys output by **KGen** is indistinguishable from uniformly sampling from the public key space. This is equal to the *fuzziness* property of KEMs given in [BCP⁺23].

Definition B.1 (KEM Fuzziness [BCP⁺23]). *Let $K = (\text{KGen}, \text{Encap}, \text{Decap})$ be a KEM with public key space $\mathcal{P}$. The advantage of an unbounded adversary $\mathcal{A}$ in breaking the fuzziness of K is:*

$$\text{Adv}_K^{\text{fuzzy}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(pk_b) = b \mid \begin{array}{l} b \leftarrow \mathcal{S}\{0, 1\}, pk_0 \leftarrow \mathcal{S}\mathcal{P}, \\ (sk_1, pk_1) \leftarrow \mathcal{S}\text{KGen}() \end{array} \right] - 1$$

Public key uniformity as defined in Definition 2.6 is equivalent to fuzziness when the public key space is the set of bitstrings. Similarly, [SGJ23] and [BCP⁺23] define properties of KEMs that are similar to ciphertext uniformity as defined in Definition 2.7. [BCP⁺23] says that a KEM is *anonymous* if the ciphertexts output by **Encap** are indistinguishable from uniformly sampled ciphertexts from the ciphertext space.

Definition B.2 (KEM Anonymity [BCP⁺23]). *Let $K = (\text{KGen}, \text{Encap}, \text{Decap})$ be a KEM with ciphertext space $\mathcal{C}$. The advantage of an unbounded adversary $\mathcal{A}$ in breaking the anonymity of K is:*

$$\text{Adv}_K^{\text{ano}}(\mathcal{A}) := 2 \cdot \Pr \left[\mathcal{A}(c_b) = b \mid \begin{array}{l} b \leftarrow \mathcal{S}\{0, 1\}, c_0 \leftarrow \mathcal{S}\mathcal{C}, \\ (sk, pk) \leftarrow \mathcal{S}\text{KGen}(), \\ (c_1, K_1) \leftarrow \mathcal{S}\text{Encap}(pk) \end{array} \right] - 1$$

[SGJ23] says that a KEM is *anonymous* if two KEM ciphertexts output by **Encap** on two randomly generated public keys are indistinguishable. This is a slightly weaker property than the anonymity definition from [BCP⁺23]. Ciphertext uniformity requires that ciphertexts output by **Encap** are indistinguishable from uniform bitstrings and also provides the adversary with the KEM public key (and in the case of strong ciphertext uniformity, the secret key), so ciphertext uniformity is a stronger notion than KEM anonymity in both [BCP⁺23, SGJ23]. The relations between these notions are summarized in Figure 3

Other similar notions of anonymity for key agreement and key exchange protocols include but are not limited to *key-hiding AKEs* [GJK21], *privacy-preserving AKEs* [SSL20], *pseudorandom (unauthenticated) key agreement* protocols [MRR20], the *random-message* property of [SGJ23], *one-way anonymity* [GSU13], and *anonymous ratcheted key exchange* [DHRR22b]. As these definitions apply to protocols rather than primitives, we do not include them in our comparison.

C Security Proofs for OEINC

C.1 IND-CPA / IND-CCA Security

We first provide the proof of Theorem 3.1 (IND-CPA / IND-CCA security of OEINC). We focus on the IND-CCA case in our proof. The IND-CPA case follows analogously, merely omitting the handling of decapsulation queries.

Proof. Game 0. We start with the security game for IND-CCA, $G_{\text{OEINC}}^{\text{IND-CCA}}(\mathcal{A})$.

Case I.

We begin with the first case reducing to IND-CCA security of the outer OKEM, and proceed via a series of game hops.

Game I.0. In $G_{\text{I.0}}$, we replace the key K_{out} with a uniformly random $\tilde{K}_{\text{out}}$.

We bound the adversary $\mathcal{A}$'s advantage by a reduction $\mathcal{B}_1$ to the IND-CCA security of outOKEM. $\mathcal{B}_1$ obtains the IND-CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ as follows. It uses pk , c^* , and K^* in place of the outer pk_{out} , c_{out} , and K_{out} , respectively. Upon any $\text{Decap}(c = c_{\text{out}} \| c'_{\text{in}})$ query, $\mathcal{B}_1$ checks if $c_{\text{out}} = c^*$ and if so, replaces K_{out} with $\tilde{K}_{\text{out}}$. Else, $\mathcal{B}_1$ queries its IND-CCA decapsulation oracle and uses the response as K_{out} .

If K^* is the real KEM key then $\mathcal{B}_1$ exactly simulates G_0 for $\mathcal{A}$; else, if K^* is random then $\mathcal{B}_1$ exactly simulates $G_{\text{I.0}}$. Therefore:

$$\Pr[G_0] - \Pr[G_{\text{I.0}}] \leq \text{Adv}_{\text{outOKEM}}^{\text{IND-CCA}}(\mathcal{B}_1).$$

Game I.1. In $G_{\text{I.1}}$ we replace the output $K_{\text{oe}}, K_{\text{ok}}$ of $G(\tilde{K}_{\text{out}})$ with uniformly random $\tilde{K}_{\text{oe}}, \tilde{K}_{\text{ok}}$, in particular for the challenge. We bound the difference in this step by a reduction $\mathcal{B}_2$ to the PRG security of G . The reduction uses its oracle in place of G , simulating either $G_{\text{I.0}}$ or $G_{\text{I.1}}$, giving:

$$\Pr[G_{\text{I.0}}] - \Pr[G_{\text{I.1}}] \leq \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2).$$

Game I.2. Finally, in $G_{\text{I.2}}$, we replace evaluations of $W(\tilde{K}_{\text{ok}}, \cdot, \cdot)$ with a random function. This in particular replaces the challenge K with a uniformly random $\tilde{K}$, independent of all keys returned by the decapsulation oracle because the third input c to W is distinct from the challenge c^* for each query. We bound this step by a reduction to the skPRF security of W , where the reduction $\mathcal{B}_3$ uses its oracle in place of $W(\tilde{K}_{\text{ok}}, \cdot, \cdot)$. This yields:

$$\Pr[G_{\text{I.1}}] - \Pr[G_{\text{I.2}}] \leq \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3).$$

We now have in $G_{I.2}$ that the real and random KEM key output of the IND-CCA game are both randomly sampled, and so $\mathcal{A}$ has no better chance than guessing the challenge bit b :

$$\text{Adv}_{\text{OEINC}}^{G_{I.2}}(\mathcal{A}) = 0.$$

Case II.

This case reduces to the IND-CCA security of the inner OKEM, again proceeding via a series of game hops.

Game II.0. In $G_{II.0}$, we replace the key K_{in} with a uniformly random $\tilde{K}_{in}$.

We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{C}_1$ to the IND-CCA security of inOKEM. $\mathcal{C}_1$ obtains the IND-CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ as follows. It uses pk , c^* , and K^* in place of the inner pk_{in} , c_{in} , and K_{in} , respectively. Upon decapsulation queries from $\mathcal{A}$, $\mathcal{C}_1$ checks if (in line 20 of **Decap**) $c_{in} = c^*$ and if so, uses $\tilde{K}_{in}$ as K_{in} . Else, it queries its IND-CCA decapsulation oracle and uses the response as K_{in} . If K^* is the real KEM key then $\mathcal{C}_1$ exactly simulates G_0 to $\mathcal{A}$; else, it simulates $G_{II.0}$. Thus:

$$\Pr[G_0] - \Pr[G_{II.0}] \leq \text{Adv}_{\text{inOKEM}}^{\text{IND-CCA}}(\mathcal{C}_1).$$

Game II.1. Lastly in $G_{II.1}$, we replace evaluations of $W(\cdot, K_{in}, \cdot)$ with a random function. This replaces K with a uniformly random $\tilde{K}$. Note that on any decapsulation query, the third input c to W is distinct from the challenge c^* . We bound this by a reduction $\mathcal{C}_2$ to the skPRF security of W , where $\mathcal{C}_2$ uses its oracle in place of calls to $W(\cdot, K_{in}, \cdot)$. Therefore:

$$\Pr[G_{II.0}] - \Pr[G_{II.1}] \leq \text{Adv}_{W,2}^{\text{skPRF}}(\mathcal{C}_2).$$

Now, as before, the real and random KEM key output of the IND-CCA game are both randomly sampled, and so $\mathcal{A}$ has no better chance than guessing the challenge bit b :

$$\text{Adv}_{\text{OEINC}}^{G_{II.1}}(\mathcal{A}) = 0.$$

Collecting the bounds yields the theorem statement. □

C.2 Strong Second Pseudorandomness

We provide the proof of Theorem 3.4 (strong second pseudorandomness of OEINC).

Proof. **Game 0.** We proceed via a series of game hops, starting with the security game for strong-2nd-pr, $G_{\text{OEINC}}^{\text{strong-2nd-pr}}(\mathcal{A})$ with $b = 1$. We will end at the same game with $b = 0$.

Case I.

In the first case, we begin by reducing to IND-CPA security and strong-2nd-pr of the outer KEM. Our strategy will be to replace the encryption of the inner ciphertext c'_{in} with randomness, then replace c_{out} with randomness. The first substitution requires steps that must be undone before doing the second substitution.

Game I.0. In the OEINC.Decap procedure of $G_{I,0}$, rather than setting the outer KEM shared secret K_{out} as the decapsulation of c_{out} , we set it to the output of the encapsulation that produced c_{out} in OEINC.Encap. The probability that these values differ is precisely the correctness error of outOKEM, hence

$$\Pr[G_0] - \Pr[G_{I,0}] \leq \delta_{\text{outOKEM}}.$$

Game I.1. In $G_{I,1}$, we replace the outer KEM shared secret K_{out} with a uniformly random shared secret $\tilde{K}_{out}$. All values derived from K_{out} use the random value $\tilde{K}_{out}$.

We bound the difference in this step by a reduction $\mathcal{B}_1$ to the IND-CPA security of outOKEM. $\mathcal{B}_1$ obtains the IND-CPA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ by using pk , c^* , and K^* as pk_{out} , c_{out} , and K_{out} , respectively. (Note that there is no decapsulation under sk_{out} to be handled, due to the change in $G_{I,0}$.) If K^* is a real KEM shared secret then $\mathcal{B}_1$ has simulated $G_{I,0}$ to $\mathcal{A}$; else, if K^* is random, then $\mathcal{B}_1$ has simulated $G_{I,1}$ to $\mathcal{A}$. Therefore:

$$\Pr[G_{I,0}] - \Pr[G_{I,1}] \leq \text{Adv}_{\text{outOKEM}}^{\text{IND-CPA}}(\mathcal{B}_1).$$

Game I.2. In $G_{I,2}$, we replace (K_{oe}, K_{ok}) with uniformly random values $\tilde{K}_{oe}$, and $\tilde{K}_{ok}$ (sampling once and using them consistently for both encapsulation and decapsulation). We bound the difference in this step by a reduction $\mathcal{B}_2$ to the PRG security of G . The reduction uses its input in place of K_{oe}, K_{ok} . Since K_{out} is random by $G_{I,1}$, $\mathcal{B}_2$ simulates either $G_{I,1}$ or $G_{I,2}$, giving:

$$\Pr[G_{I,1}] - \Pr[G_{I,2}] \leq \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2).$$

Game I.3. In the OKEM.Decap procedure of $G_{I,3}$, we replace K with a uniformly random value $\tilde{K}$. The adversary's distinguishing advantage in this hop is bounded by the skPRF security of W . In the reduction $\mathcal{B}_3$, we replace the (single) evaluation of $W(\tilde{K}_{ok}, \cdot, \cdot)$ with a PRF oracle query. Since $\tilde{K}_{ok}$ is uniform, this matches the skPRF game. The bound is thus

$$\Pr[G_{I,2}] - \Pr[G_{I,3}] \leq \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3).$$

Note that this hop encodes that W never outputs $\perp$. This means that either inOKEM never returns $\perp$ on decapsulation, i.e., it is implicitly rejecting, or $\perp$ is treated as an ordinary value in the input space of W .

Game I.4. In $G_{I,4}$, we replace the encrypted inner ciphertext c'_{in} of the challenge with a random ciphertext in $\{0,1\}^{\text{cl}_{\text{inOKEM}}}$. Note that K_{oe} is random by $G_{I,3}$, so this change can be bounded by a reduction $\mathcal{B}_4$ to the OT-IND\$ security of SE. $\mathcal{B}_4$ obtains a challenge ciphertext c^* and simulates the game for $\mathcal{A}$, replacing c'_{in} with c^* . It follows that:

$$\Pr[G_{I,3}] - \Pr[G_{I,4}] \leq \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_4).$$

Game I.5. We undo $G_{I.3}$, making K again equal to $W(\tilde{K}_{ok}, K_{in}, c)$.

Game I.6. We undo $G_{I.2}$, making (K_{oe}, K_{ok}) again equal to $G(K_{out})$.

Game I.7. We undo $G_{I.1}$, making K_{out} again the real key corresponding to c_{out} .

Game I.8. We undo $G_{I.0}$, making K_{out} again equal to the decapsulation of c_{out} .

The game hops $G_{I.5}$ – $G_{I.8}$ have reductions similar to those of $G_{I.3}$ – $G_{I.0}$:

$$\Pr[G_{I.5}] - \Pr[G_{I.8}] \leq \text{Adv}_{W,1}^{\text{skPRF}}(\mathcal{B}_3) + \text{Adv}_G^{\text{PRG}}(\mathcal{B}_2) + \text{Adv}_{\text{outOKEM}}^{\text{IND-CPA}}(\mathcal{B}_1) + \delta_{\text{outOKEM}}.$$

Game I.9. Finally, in $G_{I.9}$, we replace the outer KEM ciphertext c_{out} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{outOKEM}}}$. We bound the difference by a reduction $\mathcal{B}_5$ to the **strong-2nd-pr** of **outOKEM**. $\mathcal{B}_5$ receives its challenge values (pk, c^*, K^*) and uses pk in place of pk_{out} , c^* in place of c_{out} , and K^* in place of K_{out} (in the **OKEM.Decap** procedure). This exactly simulates $G_{I.8}$ if c^* is real, and simulates $G_{I.9}$ if c^* is random. Thus,

$$\Pr[G_{I.8}] - \Pr[G_{I.9}] \leq \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{B}_5).$$

The game is now precisely the **strong-2nd-pr** game with $b = 0$, since both c_{out} and c'_{in} are uniform.

Case II.

This case reduces to **strong-2nd-pr** of **inOKEM** and **outOKEM**. Our strategy will be to replace the (unencrypted) inner ciphertext c_{in} with randomness, then replace c_{out} with randomness.

Game II.0. In the **OEINC.Encap** procedure of $G_{II.0}$, rather than getting the outer KEM shared secret K_{out} from the encapsulation, we set it to **outOKEM.Decap**(sk_{out}, c_{out}). The probability that these values differ is precisely the correctness error of **outOKEM**, hence

$$\Pr[G_0] - \Pr[G_{II.0}] \leq \delta_{\text{outOKEM}}.$$

Game II.1. In the **OEINC.Encap** procedure of $G_{II.1}$, we replace the inner ciphertext c_{in} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$. We bound the difference by a reduction $\mathcal{C}_1$ to the **strong-2nd-pr** of **inOKEM**. $\mathcal{C}_1$ receives its challenge values (pk, c^*, K^*) and uses pk in place of pk_{in} , c^* in place of c_{in} in **OEINC.Encap**, and K^* in place of K_{in} in **OEINC.Decap**. This simulates $G_{II.0}$ if c^* is real and $G_{II.1}$ if c^* is random:

$$\Pr[G_{II.0}] - \Pr[G_{II.1}] \leq \text{Adv}_{\text{inOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_1).$$

Game II.2. Next, we replace c'_{in} with a random ciphertext in $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$ in $G_{II.2}$. Since c_{in} is random by $G_{II.1}$, it follows directly from the fact that **SE.Enc**($K_{oe}, \cdot$) is a permutation over $\{0, 1\}^{\text{cl}_{\text{inOKEM}}}$, for a fixed key K_{oe} , that:

$$\Pr[G_{II.1}] = \Pr[G_{II.2}].$$

Game II.3. In $G_{II.3}$, we replace the outer ciphertext c_{out} with a random ciphertext in $\{0, 1\}^{cl_{outOKEM}}$. We bound this by a reduction $\mathcal{C}_2$ to **strong-2nd-pr** of **outOKEM**. $\mathcal{C}_2$ receives its challenge values (pk, c^*, K^*) and uses pk in place of pk_{out} , c^* in place of c_{out} , and K^* in place of K_{out} in **OEINC.Decap**. Therefore:

$$\Pr[G_{II.2}] - \Pr[G_{II.3}] \leq \text{Adv}_{\text{outOKEM}}^{\text{strong-2nd-pr}}(\mathcal{C}_2).$$

The game is now precisely the **strong-2nd-pr** game with $b = 0$, since both c_{out} and c'_{in} are uniform. $\square$

C.3 Public Key Uniformity

We give the proof of Theorem 3.5 (pk-unif of **OEINC**).

Proof. **Game 0.** We start with the security game for pk-unif ($G_{\text{OEINC}}^{\text{pk-unif}}(\mathcal{A})$).

Game 1. In G_1 we replace the outer public key pk_{out} with a uniformly random string from $\{0, 1\}^{ol_{outOKEM}}$. This game hop is bounded by a reduction $\mathcal{B}_1$ to pk-unif of **outOKEM**. We have that

$$\Pr[G_0] - \Pr[G_1] \leq \text{Adv}_{\text{outOKEM}}^{\text{pk-unif}}(\mathcal{B}_1).$$

Game 2. Now, in G_2 , we replace pk_{in} with a uniformly random string from $\{0, 1\}^{ol_{inOKEM}}$, bounding by a reduction $\mathcal{B}_2$ to the pk-unif security of **inOKEM**:

$$\Pr[G_1] - \Pr[G_2] \leq \text{Adv}_{\text{inOKEM}}^{\text{pk-unif}}(\mathcal{B}_2).$$

Now, pk is a random pair from $\{0, 1\}^{ol_{outOKEM}} \times \{0, 1\}^{ol_{inOKEM}}$, so $\mathcal{A}$ cannot win anymore and we have

$$\text{Adv}_{\text{OEINC}}^{G_2}(\mathcal{A}) = 0. \quad \square$$

D An OKEM from DHKEM

We provide a construction of an obfuscated KEM based on DHKEM, as defined in the IETF HPKE standard [BBLW22], when instantiated with X25519 and the **Elligator2** encoding.¹² Then, we demonstrate that this achieves the required security properties to be of use in our OKEM combiner.

Consider the OKEM constructed via applying **Elligator2** to DHKEM public keys and ciphertexts, following the keygen/encapsulate-then-encode paradigm defined in [GSV24]. We denote this construction **DHKEM-El2**. Specifically, we use the algorithms described in Figure 9. In DHKEM, the ciphertext output by **Encap** is in fact a public key of the underlying DH group. Therefore, the **Elligator2** map applies to both public keys and ciphertexts in the construction.

We can then apply the following results for **SPR-CCA** and **IND-CCA** security of the resulting keygen/encapsulate-then-encode OKEM.

¹²For X25519, public keys are not uniformly distributed since scalar clamping ensures that public keys are in the prime-order subgroup, which is only 1/8 of the curve. Modifications must therefore be made to key generation to recover uniformity of public keys [Vai21].

KGen(): <pre> 1 repeat 2 $sk \leftarrow \mathbb{S}[1, p-1]$ 3 $pk \leftarrow g^{sk}$ 4 $\hat{pk} \leftarrow \text{Elligator2.Encode}(pk)$ 5 until $\hat{pk} \neq \perp$ 6 return $(sk, \hat{pk})$ </pre> Decap($sk_R, \hat{c}$): <pre> 1 $c \leftarrow \text{Elligator2.Decode}(\hat{c})$ 2 $pk_E \leftarrow \text{DeserializePublicKey}(c)$ 3 $dh \leftarrow pk_E^{sk_R}$ 4 $pk_{Rm} \leftarrow \text{SerializePublicKey}(g^{sk_R})$ 5 $kem_context \leftarrow c pk_{Rm}$ 6 $K \leftarrow G(dh, kem_context)$ 7 return K </pre>	Encap(pk_R): <pre> 1 repeat 2 $sk_E \leftarrow \mathbb{S}[1, p-1]$ 3 $pk_E \leftarrow g^{sk_E}$ 4 $dh \leftarrow pk_R^{sk_E}$ 5 $c \leftarrow \text{SerializePublicKey}(pk_E)$ 6 $pk_{Rm} \leftarrow \text{SerializePublicKey}(pk_{Rm})$ 7 $kem_context \leftarrow c pk_{Rm}$ 8 $K \leftarrow G(dh, kem_context)$ 9 $\hat{c} \leftarrow \text{Elligator2.Encode}(c)$ 10 until $\hat{c} \neq \perp$ 11 return $(\hat{c}, K)$ </pre>
--	--

Figure 9: DHKEM-Ell2: Keygen/Encapsulate-then-encode OKEM from DHKEM and Elligator2, where DHKEM is defined over a nominal group $\mathbb{G}$ (cf. [ABH⁺21]) of prime-order p with generator g . G is a key derivation function.

Theorem D.1 (Keygen/encapsulate-then-encode obfuscated KEM IND-CCA security [GSV24]). *Let OKEM be a keygen/encapsulate-then-encode obfuscated KEM based on a regular KEM KEM. For any adversary $\mathcal{A}$ against the IND-CCA security of OKEM, we give an algorithm $\mathcal{B}$ such that*

$$\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{A}) \leq 1/\epsilon_{\text{OKEM}}^{\text{1kgsucc}} \cdot 1/\epsilon_{\text{OKEM}}^{\text{1encsucc}} \cdot \text{Adv}_{\text{KEM}}^{\text{IND-CCA}}(\mathcal{B}).$$

Theorem D.2 (Keygen/encapsulate-then-encode obfuscated KEM SPR-CCA security [GSV24]). *Let OKEM be a keygen/encapsulate-then-encode obfuscated KEM based on a regular KEM KEM. For any adversary $\mathcal{A}$ against the SPR-CCA security of OKEM, we give algorithms $\mathcal{B}_1, \mathcal{B}_2$ such that*

$$\begin{aligned} \text{Adv}_{\text{OKEM}}^{\text{SPR-CCA}}(\mathcal{A}) &\leq 1/\epsilon_{\text{OKEM}}^{\text{1kgsucc}} \cdot 1/\epsilon_{\text{OKEM}}^{\text{1encsucc}} \cdot \text{Adv}_{\text{KEM}, \mathcal{S}_{\text{unif}}}^{\text{SPR-CCA}}(\mathcal{B}_1) \\ &\quad + \text{Adv}_{\text{OKEM}}^{\text{reg-ctxt-unif}}(\mathcal{B}_2), \end{aligned}$$

where $\mathcal{S}_{\text{unif}}$ samples a fresh key pair (sk, pk) and outputs the ciphertext resulting from encapsulating against pk .

The terms relating to success probability of encoding $(\epsilon_{\text{DHKEM-Ell2}}^{\text{1kgsucc}}, \epsilon_{\text{DHKEM-Ell2}}^{\text{1encsucc}})$, are both $\approx 2^{-1}$, following from the fact that approximately half of the x-coordinates of input curves are quadratic residues. The ciphertext uniformity, $\text{Adv}_{\text{DHKEM-Ell2}}^{\text{reg-ctxt-unif}}(\mathcal{A})$, and public key uniformity, $\text{Adv}_{\text{DHKEM-Ell2}}^{\text{pk-unif}}(\mathcal{A})$, similarly depend on the parameters of the underlying curve and can be computed as was done in [GSV24, §2.1] for X25519 (with aforementioned keygen modifications). Furthermore, IND-CCA was proven for DHKEM in the context of DHIES [ABR01].¹³ Therefore, it only remains to show SPR-CCA security of DHKEM. We do so in the following, via the PRF-ODH assumption [BFGJ17] on the KDF and group employed in DHKEM. Specifically, we use the nm-PRF-ODH assumption which allows multiple ODH_v

¹³More precisely, the DH-based KEM used in DHIES resembles DHKEM, but with less domain separation in its KDF invocations and no input or output validation for public keys and shared secrets.

oracle queries (to the group element that will correspond to the DHKEM public key), which we recap in Definition A.4.¹⁴

Theorem D.3 (SPR-CCA Security of DHKEM). *Let $\text{DHKEM} = \text{DHKEM}[\mathbb{P}, \mathbb{G}]$ be the KEM described in [BBLW22] over a group $\mathbb{G}$ and key derivation function $\mathbb{G}$. For any adversary $\mathcal{A}$ against the SPR-CCA security of DHKEM, we give an algorithm $\mathcal{B}_1$ such that*

$$\text{Adv}_{\text{DHKEM}, \mathcal{S}_{\text{unif}}}^{\text{SPR-CCA}}(\mathcal{A}) \leq \text{Adv}_{\mathbb{G}, \mathbb{G}}^{\text{nm-PRF-ODH}}(\mathcal{B}_1)$$

where $\mathcal{S}_{\text{unif}}$ samples and outputs a random group element.

Proof. Game 0. We start with the SPR-CCA security game $\mathbb{G}_{\text{DHKEM}, \mathcal{S}_{\text{unif}}}^{\text{SPR-CCA}}(\mathcal{A})$.

Game 1. In $\mathbb{G}_1$, we replace the real challenge key K_1^* with a randomly sampled key $\tilde{K}$ from $\mathcal{K}$. We bound the difference by a reduction $\mathcal{B}_1$ to the nm-PRF-ODH security of $\mathbb{G}$ over $\mathbb{G}$. Note that in DHKEM, the ciphertext is simply an ephemeral group element g^r . The reduction obtains a nm-PRF-ODH challenge of the form (g^u, g^v, y_b^*) , where y_b^* is either a random value or $\mathbb{G}(g^{uv}, x^*)$ for a challenge label x^* . Therefore, the reduction queries as label x^* the KEM context (g^u, g^v) as per [BBLW22] and uses its nm-PRF-ODH challenge y_b^* in place of K^* , g^u in place of c^* , and g^v in place of pk . Upon $\text{Decap}(c)$ queries from $\mathcal{A}$, the reduction queries its nm-PRF-ODH oracle ODH_v on group element c and label (c, g^v) to obtain the KEM shared secret. When the challenge is real, $\mathcal{B}_1$ exactly simulates $\mathbb{G}_0$ to $\mathcal{A}$; else, $\mathcal{B}_1$ exactly simulates $\mathbb{G}_1$ to $\mathcal{A}$. It follows that

$$\Pr[\mathbb{G}_0] - \Pr[\mathbb{G}_1] \leq \text{Adv}_{\mathbb{G}}^{\text{nm-PRF-ODH}}(\mathcal{B}_1).$$

Game 2. In $\mathbb{G}_2$, we replace the challenge ciphertext with a ciphertext $c^* \leftarrow^* \mathcal{S}_{\text{unif}}$, i.e., a randomly sampled group element. Since the key $\tilde{K}$ is random by $\mathbb{G}_1$, the challenge ciphertext is no longer used to compute the key in Encap . Therefore, this change is unnoticeable to the adversary.

$$\Pr[\mathbb{G}_1] = \Pr[\mathbb{G}_2].$$

At this point, we have that c and K are both random. Hence, the adversary $\mathcal{A}$ can only guess the challenge bit b :

$$\text{Adv}_{\text{DHKEM}, \mathcal{S}_{\text{unif}}}^{\mathbb{G}_2}(\mathcal{A}) = 0. \quad \square$$

Alternative constructions. A classically secure OKEM can also be constructed using DHKEM over a prime-order curve, e.g., P-256 or Ristretto [NIS23, LHT16], in combination with Elligator² [Tib14] or Elligator2, respectively, in a similar manner. Compared to Elligator2, the Elligator² construction has 100% success probability. Naturally, substituting different encodings and/or elliptic curve groups alters the bounds accordingly.

¹⁴The nm-PRF-ODH assumption essentially encodes modeling the KDF as a random oracle and assuming the strong or gap Diffie–Hellman problem is hard in the underlying group.

E Obfuscated Key Exchange Security Model

We repeat the key exchange model introduced in [GSV24]. We specify a key exchange protocol KE through two algorithms:

- **Setup**($id, role$) $\rightarrow (sk, pk, st)$ generates the public-secret key pair (pk, sk) as well as the initial user state st for a protocol user with identity id in role $role \in \{\text{initiator}, \text{responder}\}$.
- **Run**($\pi_u^i, st_u, sk_u, pk_v, m$) $\rightarrow (\pi_u^i, st_u, m')$ processes a protocol message m delivered to session π_u^i following the protocol specification (along with inputs the session owner's state st_u and secret key sk_u , and the session's peer public key pk_v), updates π_u^i and st_u accordingly, and outputs the response message m' .

Further, we write KE.KS to denote the session key space of KE.

E.1 Session and Game Variables

Session object π_u^i captures the session information for the i th session owned by user u . Each user u is assigned a role, $u.role \in \{\text{initiator}, \text{responder}\}$, and acts accordingly as initiator or responder in the protocol.

Each session object π_u^i holds several variables. The following session variables in *italics* font are accessible by the key exchange protocol (i.e., **Run**):

- $\pi_u^i.peerid$: the identity of the session's intended peer.
- $\pi_u^i.status$: the state of execution (initially **running**, then set once by the protocol to **accepted** or **rejected**).
- $\pi_u^i.skey$: the session key.
- $\pi_u^i.sid, \pi_u^i.cid$: the session and contributive identifiers.

These following session variables in **sans-serif** font are accessible by the security game only:

- $\pi_u^i.t_{acc}$: the time at which the session accepted (initially ∞).
- $\pi_u^i.revealed, \pi_u^i.tested$: flags indicating whether the session was revealed or tested, respectively.

The security game further tracks the following game variables:

- **time**: a logical clock to order queries by the adversary.
- **users**: the number of users in the game.
- **b**: the challenge bit.
- sk_u, pk_u : the secret and public key of user u .
- $revsk_u, revpk_u$: flags indicating whether the secret or public, key of user u was revealed.

For syntactical convenience, we will interpret variables which are unset or set to ∞ as **false** in boolean conditions.

E.2 Session Identifiers, Contributive Identifiers, and Partnering

We use *session identifiers* [BPR00] to determine that two sessions π_u^i, π_v^j are *partnered* if and only if $\pi_u^i.sid = \pi_v^j.sid$. Partnering is used to define basic correctness/soundness properties and to exclude trivial attacks like testing and revealing two partnered sessions that jointly executed the protocol.

We further use *contributive identifiers* [DFGS15] to determine when a responder session has an honest communication partner. Recall that initiators are unauthenticated. Hence, to avoid trivial attacks, the adversary may test a responder session only if that session honestly received the values specified in the contributive identifier *cid*.

E.3 Security Definition

Definition E.1 (Obfuscated key exchange security). *Let KE be a key exchange protocol and $G_{KE,S}^{ObfKE}(\mathcal{A})$ be the obfuscated key exchange security game wrt. a simulator $\mathcal{S}$ defined in Figure 10 for an adversary $\mathcal{A}$. We define the advantage of $\mathcal{A}$ in breaking the ObfKE security of KE as*

$$\text{Adv}_{KE,S}^{ObfKE}(\mathcal{A}) := 2 \cdot \Pr [G_{KE,S}^{ObfKE}(\mathcal{A}) \Rightarrow 1] - 1.$$

We distinguish two flavors of ObfKE, capturing regular (rObfKE) and strong (sObfKE) obfuscation through different ObfFresh predicates (Figure 11); we omit the prefixes if the flavor is clear from context.

E.4 Single-challenge Selective Security

For our security analysis, we will establish security in a simpler version of the obfuscated key exchange game, where the adversary has to commit to winning via either:

1. a single TEST query to some pre-declared session, or
2. a single CHALLENGE query against some pre-declared server.

This variant is denoted as ObfKE-1, and is a weaker versions of the main game, asking only for selective security in a single-challenge setting. It has been shown in [GSV24] that this simpler version generically implies full security (per Definition E.1) via a hybrid argument, for the same type of obfuscation (regular or strong). The hybrid works via guessing which session or server the adversary will challenge, losing a factor $(n_s + n_r q_C)$ when reducing ObfKE to ObfKE-1 security, where n_s and n_r are the number of sessions and servers in the game and q_C the number of CHALLENGE queries the adversary makes.

F Security Proof for Drivel (Theorem 4.1)

We proceed via a series of game hops. The majority of the proof directly follows the proof of security of **pq-obfs** from [GSV24]; we recap these steps here, in part verbatim. Game hops with significant differences to **pq-obfs** are underlined. The primary difference is that, in the final steps, OT-IND\$ is applied in $G_{II.6}$ and $G_{II.7}$ rather than invoking public key and ciphertext uniformity.

```

ObfKEKE,S(A)
INITIALIZE:
1  time ← 0; users ← 0
2  b ← $ {0,1}
3  Probed ← false

NEWUSER(id, role):
4  u ← ++users
5  u.role ← role
6  (pku, sku, stu) ← $ Setup(id, role)
7  revsku ← ∞ ; revpku ← ∞

REVSESSIONKEY(u, i):
8  if πui = ⊥ or ¬πui.tacc then return ⊥
9  πui.revealed ← true
10 return πui.skey

REVPUBLICKEY(u):
11 if revpku then return ⊥
12 revpku ← ++time
13 return pku

SEND(u, i, m):
14 f ← [[πui = ⊥]] // First SEND to this session?
15 if πui = ⊥ and u.role = initiator then
16   πui.peerid ← m; m ← ε // "Virtual" first message to initiator sets peer id
17 if πui.status ∈ {accepted, rejected} then return ⊥
18 (πui, stu, m') ← $ Run(πui, stu, sku, pkπui.peerid, m)
19 if πui.status = accepted then πui.tacc ← ++time
20 if f then // Upon first message sent/received...
21   if u.role = initiator then
22     Fπui.peerid ← Fπui.peerid ∪ {m'} // Record initiators' first messages (per responder)
23   if u.role = responder and m' ≠ ε then
24     if m ∉ Fu and ¬revpku then Probed ← true // Response to a non-initiator-first message by a non-pk-revealed
25     responder is a successful probe
26   Fu ← Fu \ {m} // Consider m "consumed"
27 return (πui.status, m')

CHALLENGE(u, v):
28 πu ← πv ← ⊥ // Temporary initiator and responder sessions
29 if u.role ≠ initiator or v.role ≠ responder then return ⊥
30 trans1 ← (); πu.peerid ← v; p ← u; m ← ε
31 repeat // Execute entire protocol and collect real transcript
32   (πp, stp, m) ← $ Run(πp, stp, skp, pkπp.peerid, m)
33   trans1 ← trans1 || (m)
34   if p = u then p ← v else p ← u // Switch parties
35 until πu.status = πv.status = accepted
36 trans0 ← $ S() // Simulated transcript
37 k1 ← πu.skey; k0 ← $ KE.KS // Real-or-random session key
38 challv ← true // Mark server v as challenged
39 return (transb, kb)

REVSECRETKEY(u):
14 if revsku then return ⊥
15 revsku ← ++time
16 if ¬revpku then revpku ← time // Consider pku
    revealed, too
17 return sku

TEST(u, i):
18 if πui = ⊥ or ¬πui.tacc or πui.tested then
19   return ⊥
20 πui.tested ← true
21 k1 ← πui.skey
22 k0 ← $ KE.KS
23 return kb

FINALIZE(b'):
24 if ¬Sound then b' ← b
25 if ¬ExplicitAuth then b' ← b
26 if Probed then b' ← b
27 if ¬Fresh then b' ← 0
28 if ¬ObfFresh then b' ← 0
29 return [[b = b']]

```

Figure 10: Obfuscated key exchange security (ObfKE) game capturing key indistinguishability, explicit server authentication, obfuscation with respect to a simulator $\mathcal{S}$, and probing resistance, via predicates (Figure 11) Fresh, ExplicitAuth, ObfFresh, and flag Probed, respectively.

Fresh:

```

1 for each  $\pi_u^i : \pi_u^i.\text{tested}$ 
2   if  $\pi_u^i.\text{revealed}$  then
3     return false // tested session may not be revealed
4   if  $\exists \pi_v^j \neq \pi_u^i : \pi_v^j.\text{sid} = \pi_u^i.\text{sid} \wedge (\pi_v^j.\text{tested} \vee \pi_v^j.\text{revealed})$  then
5     return false // tested session's partnered session may not be tested or revealed
6   if  $u.\text{role} = \text{initiator} \wedge \text{revsk}_{\pi_u^i.\text{peerid}} \leq \pi_u^i.\text{t}_{\text{acc}} \wedge \neg \exists \pi_v^j \neq \pi_u^i : (\pi_u^i.\text{sid} = \pi_v^j.\text{sid})$ 
7     return false // initiators: forward secrecy (peer's sk unrevealed prior to acceptance) or passive execution
8   if  $u.\text{role} = \text{responder} \wedge \neg \exists \pi_v^j \neq \pi_u^i : (\pi_u^i.\text{cid} = \pi_v^j.\text{cid} \wedge v.\text{role} = \text{initiator})$ 
9     return false // responders: security only for passive executions (as initiators are unauthenticated)
10 return true

```

ObfFresh:

```

1 if  $\exists v : \text{revpk}_v \wedge \text{chall}_v$  then // Regular obfuscation (rObfKE)
2 if  $\exists v : \text{revsk}_v \wedge \text{chall}_v$  then // Strong obfuscation (sObfKE)
3 return false // Challenge pk revealed (fwd-secret: prior to challenge)
4 return true

```

ExplicitAuth:

```

// Explicit authentication of responders to initiators
1  $\forall \pi_u^i : ( u.\text{role} = \text{initiator} \wedge \pi_u^i.\text{t}_{\text{acc}} < \text{revsk}_{\pi_u^i.\text{peerid}} \implies \exists \pi_v^j : v = \pi_u^i.\text{peerid} \wedge \pi_u^i.\text{sid} = \pi_v^j.\text{sid} )$ 

```

Sound:

```

1 if  $\exists$  distinct  $\pi_u^i, \pi_v^j, \pi_w^k : \pi_u^i.\text{sid} = \pi_v^j.\text{sid} = \pi_w^k.\text{sid} \neq \perp$  then
2   return false // no triple sid match
3 if  $\exists \pi_u^i, \pi_v^j : \pi_u^i.\text{sid} = \pi_v^j.\text{sid} \neq \perp \wedge u.\text{role} = v.\text{role}$  then
4   return false // partnering implies different roles
5 if  $\exists \pi_u^i, \pi_v^j : \pi_u^i.\text{sid} = \pi_v^j.\text{sid} \neq \perp \wedge \pi_u^i.\text{cid} \neq \pi_v^j.\text{cid}$  then
6   return false // partnering implies same contributive identifiers
7 if  $\exists \pi_u^i, \pi_v^j : \pi_u^i.\text{sid} = \pi_v^j.\text{sid} \neq \perp \wedge u.\text{role} = \text{initiator} \wedge \pi_u^i.\text{peerid} \neq v$  then
8   return false // partnering implies agreement on responder ID
9 if  $\exists \pi_u^i, \pi_v^j : \pi_u^i.\text{sid} = \pi_v^j.\text{sid} \neq \perp \wedge \pi_u^i.\text{skey} \neq \pi_v^j.\text{skey}$  then
10   return false // partnering implies same key
11 return true

```

Figure 11: Predicates Fresh, ExplicitAuth, ObfFresh, and flag Probed for the obfuscated key exchange security (ObfKE) game (Figure 10) with regular (regular (rObfKE)) or strong (strong (sObfKE)) obfuscation.

Game 0. We start with the security game for strong obfuscation (sObfKE), $G_0 = G_{\text{Drive1}, S_{\text{Drive1}}}^{\text{sObfKE}}$, omitting the S_{Drive1} subscript from here on:

$$\text{Adv}_{\text{Drive1}}^{\text{sObfKE}}(\mathcal{A}) = \text{Adv}_{\text{Drive1}}^{G_0}(\mathcal{A}) = 2 \cdot \Pr[G_0] - 1.$$

Game 1 (Exclude KEM public key collisions). We modify Game G_0 to abort if two honest sessions sample the same ephemeral KEM key (pk_e) or if two servers sample the same long-term KEM key (pk_s). This is bounded by the public key collision probability ($\text{pkcoll}_{\text{KEM}}, \text{pkcoll}_{\text{OKEM}}$) of the ephemeral and static KEM for the at most n_s ephemeral and n_r long-term KEM keys (Definition 2.4):

$$\Pr[G_0] - \Pr[G_1] \leq \text{pkcoll}_{\text{KEM}}(n_s) + \text{pkcoll}_{\text{OKEM}}(n_r).$$

Game 2 (KEM correctness). We modify Game G_1 to abort if for any keys $(sk, pk) \leftarrow \text{KGen}()$ and encapsulation $(c, K) \leftarrow \text{Encap}(pk)$, we have that $K \neq \text{Decap}(sk, c)$. The probability of this happening for any tuple (sk, pk, c) is upper-bounded by the correctness error $\delta_{\text{KEM}}, \delta_{\text{OKEM}}$ of the ephemeral and obfuscated KEM as defined in Definition 2.3. Since there are two such tuples per session, we can bound the probability of such an abort by

$$\Pr[G_1] - \Pr[G_2] \leq n_s \cdot (\delta_{\text{KEM}} + \delta_{\text{OKEM}}).$$

Establishing soundness (Sound). We observe that in Game G_2 , the adversary cannot violate soundness anymore (i.e., the predicate **Sound** cannot be false). Checking the conditions in **Sound** (Figure 11), we have:

1. No triple sid match: For three session identifiers to match, two initiator sessions or two responder sessions would have to sample the same KEM keys. However, G_1 aborts in that case.
2. Partnering implies different roles: For two same-role sessions to be partnered, they would have to sample the same ephemeral KEM key. Again, G_1 aborts in this case.
3. Partnering implies same contributive identifiers: The contributive identifier contains a subset of entries in the session identifier. Hence, agreement on the latter implies agreement on the former.
4. Partnering implies agreement on responder ID: The session identifier contains the responder's long-term KEM key pk_s . As these do not collide by G_1 , they uniquely identify the responder.
5. Partnering implies same key: For two partnered sessions to derive a different key, the inputs to the computation of sk_{key} would have to differ. This may occur if the inputs to **Encap** (or **Decap**) are not consistent or if **Encap** (or **Decap**) has a correctness error. The latter case is prevented by G_2 . For the former, the elements in the session identifier uniquely determine the inputs; note in particular that it uniquely identifies the responder (as per above) and hence the *NodeID* input. Hence agreement on session identifiers implies that the same key is generated.

Game 3 (Prevent Probed being set). In Game G_3 , we stop setting $\text{Probed} \leftarrow \text{true}$ (i.e., $\mathcal{A}$ cannot win anymore in G_3 by breaking probing resistance). We have

$$\Pr[G_2] - \Pr[G_3] \leq \Pr[G_2 \text{ sets Probed}].$$

Recall that for Probed to be set, a responder session, whose public key (pk_S, NodeID) is not revealed, must reply to a non-initiator-first message m , with a non-empty message m' . We will bound this probability via the following, branching game hops $G_{2.1}$ – $G_{2.3}$.

Game 2.1 (Guess violated server and session). In Game $G_{2.1}$, we guess a “target server” v^* , the first server (in order of creation) on which Probed is set to true , as well as the (first) session π^* in which this happens. We let the game abort if that guess is incorrect, i.e., if Probed is not set to true when the server responds in session π^* or if Probed is set to true on a response from a server that was created earlier. This introduces an according loss in the number of servers times the number of sessions:

$$\Pr[G_3 \text{ sets Probed}] \leq n_s \cdot n_r \cdot \Pr[G_{2.1} \text{ sets Probed}].$$

Game 2.2 (Random ES). In Game $G_{2.2}$, we replace ES in any session with the target server v^* with a uniformly random value $\widetilde{ES}$.

We bound the difference in this step by a reduction to the PRF security of F_2 . The reduction $\mathcal{B}_1$ does not sample NodeID for v^* itself, but instead uses its oracle to compute evaluations of $F_2(\text{NodeID}, \cdot)$ in sessions with v^* by calling its oracle on K_S . When the oracle output is real, reduction $\mathcal{B}_1$ exactly simulates $G_{2.1}$, whereas when the output is random, it simulates $G_{2.2}$. Therefore, because NodeID is unrevealed for server v^* ,

$$\Pr[G_{2.1} \text{ sets Probed}] - \Pr[G_{2.2} \text{ sets Probed}] \leq \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_1).$$

Game 2.3 (Random MAC_C). In Game $G_{2.3}$, we replace any evaluation of F_1 on input random values $\widetilde{ES}$ in sessions with the target server v^* with a random function (per such value). This in particular replaces MAC_C in any session with v^* uniformly random values MAC_C^* .

We bound the difference in this step by a reduction to the PRF security of F_1 . The reduction $\mathcal{B}_2$ does not sample $\widetilde{ES}$ in the target session π^* itself, but instead uses its oracle to compute $F_1(\widetilde{ES}, \cdot)$; in particular for computing MAC_C . Based on the oracle’s output, $\mathcal{B}_2$ correctly simulates either $G_{2.2}$ or $G_{2.3}$. Hence,

$$\Pr[G_{2.2} \text{ sets Probed}] - \Pr[G_{2.3} \text{ sets Probed}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_2).$$

Observe that Probed in $G_{2.3}$ is set if a non-initiator-first message m yields a reply by an unrevealed responder. Such a message m can be either (1) a replay of an already consumed message output by an honest initiator for this responder, or (2) one that is different from all honest initiators’ first messages sent to this responder.

In the first case, m is rejected due to the replay check against the list of seen MAC values $st.\mathcal{S}_{MAC}$ recorded in the server’s state.

In the second case, $m = epk_e \| c_S \| P_C \| M_C \| MAC_C$ must be different from any initiator session’s message sent to this responder. By $G_{2.3}$, the target client MAC value MAC_C^* that

the session π^* with v^* that first sets probed is a random fl_1 -bit value unknown to $\mathcal{A}$. The adversary $\mathcal{A}$ can therefore only guess MAC_C^* with probability

$$\Pr[\text{G}_{2.3} \text{ sets Probed}] \leq \frac{1}{2^{\text{fl}_1}}.$$

This completes the branching game hops bounding

$$\Pr[\text{G}_2 \text{ sets Probed}] \leq n_s n_r \cdot \left(\text{Adv}_{\text{F}_2}^{\text{PRF}}(\mathcal{B}_1) + \text{Adv}_{\text{F}_1}^{\text{PRF}}(\mathcal{B}_2) + \frac{1}{2^{\text{fl}_1}} \right).$$

We now continue with the main proof from G_3 , where **Probed** is never set.

Game 4 (Prevent ExplicitAuth being violated). In Game G_4 , we abort the game if **ExplicitAuth** is violated (i.e., $\mathcal{A}$ cannot win anymore in G_4 by breaking explicit authentication). We have

$$\Pr[\text{G}_3] - \Pr[\text{G}_4] \leq \Pr[\neg \text{ExplicitAuth in G}_3].$$

Recall that for **ExplicitAuth** to be violated, an initiator session must accept with a peer whose secret key is uncompromised at this point in the game, but for which there is no session of that peer holding the same session identifier. Formally,

$$\begin{aligned} \Pr[\neg \text{ExplicitAuth in G}_3] &= \Pr \left[\exists \pi_u^i : u.\text{role} = \text{initiator} \wedge \pi_u^i.\text{t}_{\text{acc}} < \text{revsk}_{\pi_u^i.\text{peerid}} \wedge \right. \\ &\quad \left. \forall \pi_v^j \text{ s.t. } v = \pi_u^i.\text{peerid} : \pi_u^i.\text{sid} \neq \pi_v^j.\text{sid} \right]. \end{aligned}$$

We will bound this probability again through a series of branching game hops $\text{G}_{3.1}$ – $\text{G}_{3.6}$.

Game 3.1 (Guess violated initiator session and peer). In Game $\text{G}_{3.1}$, we guess a “target session” π^* , the first session (in order of creation) which makes **ExplicitAuth** evaluate to **false**, as well as that session’s peer, $v^* = \pi^*.\text{peerid}$. We let the game abort if that guess is incorrect, i.e., if **ExplicitAuth** is not violated when the session π^* accepts or if v^* is not its peer. This introduces an according loss in the number of sessions times the number of servers:

$$\Pr[\neg \text{ExplicitAuth in G}_3] \leq n_s \cdot n_r \cdot \Pr[\neg \text{ExplicitAuth in G}_{3.1}].$$

Game 3.2 (Long-term KEM IND-CCA). Let

$$(c_S, K_S) \leftarrow \text{OKEM.Encap}(pk_S)$$

be the encapsulation computed at session π^* with the long-term public key of v^* . In Game $\text{G}_{3.2}$, we replace the long-term KEM key K_S with a uniformly random $\tilde{K}_S$ in π^* . All values derived from K_S in π^* use the randomized value $\tilde{K}_S$.

We bound the adversary $\mathcal{A}$ ’s difference in advantage by a reduction $\mathcal{B}_3$ to the IND-CCA security of OKEM. $\mathcal{B}_3$ obtains the IND-CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ as follows. It uses pk as the long-term public key of v^* . In π^* , $\mathcal{B}_3$ uses c^* as the ciphertext c_S . In any session of v^* , if the ciphertext c_S received is not c^* , then $\mathcal{B}_3$ queries its IND-CCA decapsulation oracle and uses the response as K_S ; else, if $c_S = c^*$, then $\mathcal{B}_3$ uses K^* as K_S . Note that by the definition of **ExplicitAuth**, sk_S is not revealed to the adversary prior to π^* violating **ExplicitAuth** and thus $\mathcal{B}_3$ does not need to answer a **REVSECKEY**

call on v^* . If K^* is the real KEM key then $\mathcal{B}_3$ has exactly simulated $G_{3.1}$ to $\mathcal{A}$; else, if K^* is random, then $\mathcal{B}_3$ has exactly simulated $G_{3.2}$ to $\mathcal{A}$. Therefore:

$$\Pr[\neg \text{ExplicitAuth in } G_{3.1}] - \Pr[\neg \text{ExplicitAuth in } G_{3.2}] \leq \text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3).$$

Game 3.3 (Random ES). In Game $G_{3.3}$, we replace ES with a uniformly random $\widetilde{ES}$ in π^* . We bound the difference in this step by a reduction to the **swap-PRF** security of F_2 . The reduction $\mathcal{B}_4$ instead of sampling $\widetilde{K}_S$ uses its oracle to compute $F_2(\cdot, \widetilde{K}_S)$ in π^* . When the output is real, reduction $\mathcal{B}_4$ exactly simulates $G_{3.2}$, whereas when the output is random, it simulates $G_{3.3}$. Therefore,

$$\Pr[\neg \text{ExplicitAuth in } G_{3.2}] - \Pr[\neg \text{ExplicitAuth in } G_{3.3}] \leq \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_4).$$

Game 3.4 (Random ES'). In Game $G_{3.4}$, we replace evaluations $F_1(\widetilde{ES}, \cdot)$ in π^* with a random function. This in particular replaces ES' with a random value $\widetilde{ES}'$. We bound the difference in this step with the **PRF** security of F_1 , via a reduction $\mathcal{B}_5$ using its oracle in place of $F_1(\widetilde{ES}, \cdot)$:

$$\Pr[\neg \text{ExplicitAuth in } G_{3.3}] - \Pr[\neg \text{ExplicitAuth in } G_{3.4}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_5).$$

Game 3.5 (Random FS). In Game $G_{3.5}$, we replace evaluations $F_2(\widetilde{ES}', \cdot)$ in π^* with a random function. This in particular replaces FS with a random value $\widetilde{FS}$. As in the previous hops, we can bound the difference by **PRF** security of F_2 :

$$\Pr[\neg \text{ExplicitAuth in } G_{3.4}] - \Pr[\neg \text{ExplicitAuth in } G_{3.5}] \leq \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_6).$$

Game 3.6 (Random $auth$). Finally, in Game $G_{3.6}$, we replace evaluations $F_1(\widetilde{FS}, \cdot)$ in π^* with a random function. This in particular replaces $auth$ with a random value $auth^*$, and again can be bounded by **PRF** security of F_1 :

$$\Pr[\neg \text{ExplicitAuth in } G_{3.5}] - \Pr[\neg \text{ExplicitAuth in } G_{3.6}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_7).$$

Now, given that $auth^*$ is a uniformly random value unknown to $\mathcal{A}$, violating **ExplicitAuth** in π^* requires that $\mathcal{A}$ correctly guesses the fl_1 -bit value of $auth^*$. The probability of $\mathcal{A}$ guessing correctly is

$$\Pr[\neg \text{ExplicitAuth in } G_{3.6}] \leq \frac{1}{2^{\text{fl}_1}}.$$

This completes the branching, bounding

$$\begin{aligned} \Pr[\neg \text{ExplicitAuth in } G_3] &\leq n_s n_r \cdot \left(\text{Adv}_{\text{OKEM}}^{\text{IND-CCA}}(\mathcal{B}_3) + \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_4) \right. \\ &\quad \left. + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_5) + \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_6) + \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_7) + \frac{1}{2^{\text{fl}_1}} \right). \end{aligned}$$

We now continue with the main proof from Game G_4 , where **ExplicitAuth** is never violated.

Game 5 (Single-challenge selective security). We now restrict the adversary to a single-challenge selective (**sObfKE-1**) version G_5 of Game G_4 , where $\mathcal{A}$ has to commit upfront to winning either

- (I) via a single TEST query ($A = \text{TEST}$) on the pre-determined s -th created session, or
- (II) via a single CHALLENGE query ($A = \text{CHALLENGE}$) on the pre-determined p -th created server.

Recall that in G_4 , **Sound** is never violated, **Probed** is never set, and **ExplicitAuth** is never violated, hence $\mathcal{A}$ can only win via TEST or CHALLENGE queries. Applying the hybrid argument from [GSV24, Theorem 4.3], we have

$$\text{Adv}_{\text{Drive1}}^{G_4}(\mathcal{A}) \leq (n_s + n_r \cdot q_C) \cdot \text{Adv}_{\text{Drive1}}^{G_5}(\mathcal{A}).$$

Let us write G_5^T and G_5^C for the game G_5 restricted to the two winning options of $\mathcal{A}$ ($A \in \{\text{TEST}, \text{CHALLENGE}\}$). By the union bound,

$$\text{Adv}_{\text{Drive1}}^{G_5}(\mathcal{A}) \leq \text{Adv}_{\text{Drive1}}^{G_5^T}(\mathcal{A}) + \text{Adv}_{\text{Drive1}}^{G_5^C}(\mathcal{A}).$$

We will bound each term separately, through the following proof cases I and II.

Case I. Win via TEST

In this proof case, $\mathcal{A}$ commits to winning via a single TEST query on a session π^* . For the test session to satisfy **Fresh**, we must have that there is an honest partner—i.e., a session π_p^* holding the same *cid* or *sid* (“passive execution”)—or that the responder peer is uncompromised upon acceptance of the initiator session π^* (“forward secrecy”). Note that since **ExplicitAuth** is ensured to hold (by Game G_4), the latter implies that there actually must be an honest partner π_p^* holding the same *sid*, so we can at this point focus on such passive executions.

Game I.0. This case begins with Game G_5 conditioned on $A = \text{TEST}$, where the test session has an honest (*sid* or *cid*) partner:

$$\text{Adv}_{\text{Drive1}}^{G_{I.0}}(\mathcal{A}) = \text{Adv}_{\text{Drive1}}^{G_5^T}(\mathcal{A}).$$

Game I.1 (Guess the partner session). In Game $G_{I.1}$, we guess the session π_p^* that is the honest *sid* partner (if π^* is an initiator) or *cid* partner (else) of π^* . Aborting if the guess is incorrect, this introduces a loss in the number of sessions:

$$\text{Adv}_{\text{Drive1}}^{G_{I.0}}(\mathcal{A}) = n_s \cdot \text{Adv}_{\text{Drive1}}^{G_{I.1}}(\mathcal{A}).$$

Game I.2 (Ephemeral KEM IND-1CCA). Let

$$(c_e, K_e) \leftarrow \text{KEM.Encap}(pk_e)$$

be the encapsulation computed at session π^* . If π^* is an initiator then this is the ephemeral public key pk_e in *sid*, and otherwise, it is the ephemeral public key in *cid* (which must necessarily be determined by Game $G_{I.1}$).

In Game $G_{I.2}$, we replace the ephemeral KEM key K_e with a uniformly random $\tilde{K}_e$ in π^* . All values derived from K_e in π^* use the randomized value $\tilde{K}_e$.

We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_8$ to the IND-1CCA security of KEM. $\mathcal{B}_8$ obtains the IND-1CCA challenge (pk, c^*, K^*) and simulates the game for $\mathcal{A}$ as follows. In the protocol run between π^* and π_p^* , $\mathcal{B}_8$ uses pk as the ephemeral public key of the initiator and c^* as the ciphertext c_e . If the initiator session receives a ciphertext $c_e \neq c^*$, then $\mathcal{B}_8$ queries its IND-1CCA decapsulation oracle (once) and uses the response as K_e ; else, if $c_e = c^*$, then $\mathcal{B}_8$ uses K^* as K_e . If K^* is the real KEM key then $\mathcal{B}_8$ has exactly simulated $G_{I.1}$ to $\mathcal{A}$; else, if K^* is random, then $\mathcal{B}_8$ has exactly simulated $G_{I.2}$ to $\mathcal{A}$. Therefore:

$$\Pr[G_{I.1}] - \Pr[G_{I.2}] \leq \text{Adv}_{\text{KEM}}^{\text{IND-1CCA}}(\mathcal{B}_8).$$

Game I.3 (Random FS). In Game $G_{I.3}$, we replace evaluations $F_2(\cdot, K^*)$ with a random function. This in particular replaces FS with a uniformly random value $\widetilde{FS}$ in π^* . (Note that if π_p^* received the same ciphertext, then it will also use K^* .)

We bound the difference in this step by a reduction to the **swap**-PRF security of F_2 . The reduction $\mathcal{B}_9$ uses its oracle in place of $F_2(\cdot, K^*)$, simulating either $G_{I.2}$ or $G_{I.3}$, giving:

$$\Pr[G_{I.2}] - \Pr[G_{I.3}] \leq \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_9).$$

Game I.4 (Random key). Finally, in Game $G_{I.4}$, we replace evaluations $F_1(\widetilde{FS}, \cdot)$ with a random function. This in particular replaces key with a uniformly random value key^* in π^* . We again bound this game hop by a reduction to the PRF security of F_1 :

$$\Pr[G_{I.3}] - \Pr[G_{I.4}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{10}).$$

We now have in Game $G_{I.4}$ that the real and random session key output of the TEST oracle are both randomly sampled. Also, any non-partnered session keys are independent of π^* , since the context input when deriving key is precisely the session identifier sid . Hence, $\mathcal{A}$ has no better chance than guessing the challenge bit b and so

$$\text{Adv}_{\text{Drive1}}^{G_{I.4}}(\mathcal{A}) = 0.$$

Case II. Win via CHALLEXEC

In this proof case, $\mathcal{A}$ commits to making a single CHALLEXEC query on the p th-created server, denoted v^* here (and not making any TEST queries). We assume that $\mathcal{A}$ makes one CHALLEXEC call, as otherwise its advantage is 0, and hence never calls REVSECRETKEY on p , as otherwise it would violate ObfFresh and lose.

Game II.0. This case begins with Game G_5 conditioned on $A = \text{CHALLEXEC}$:

$$\Pr[G_{II.0}] = \Pr[G_5^C].$$

Game II.1 (Long-term KEM SPR\$-CCA). Let

$$(c_S, K_S) \leftarrow^s \text{OKEM.Encap}(pk_S)$$

be the encapsulation computed by the initiator when CHALLEXEC is called.

In Game $G_{II.1}$, we replace the ciphertext c_S , as well as the long-term KEM key K_S as follows. Instead of running OKEM.Encap, the initiator samples $c_S \leftarrow_{\$} \{0, 1\}^{\text{cl}}$ and $K_S \leftarrow_{\$} \mathcal{K}$ uniformly at random.

We bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_{11}$ to the SPR\$-CCA security of OKEM. $\mathcal{B}_{11}$ obtains the SPR\$-CCA challenge (pk, c^*, K^*) , and simulates the game for $\mathcal{A}$ as follows. It uses pk as the public key of the p th-created responder v^* committed to by the adversary. When CHALLEXEC is called, $\mathcal{B}_{11}$ uses c^* as the ciphertext c_S . In any session of v^* , if the ciphertext c_S received is not c^* , then $\mathcal{B}_{11}$ queries its SPR\$-CCA decapsulation oracle and uses the response as K_S ; else, if $c_S = c^*$, then $\mathcal{B}_{11}$ uses K^* as K_S . If (c^*, K^*) are the real KEM values then $\mathcal{B}_{11}$ exactly simulates $G_{II.0}$ to $\mathcal{A}$; else, if they are random, then $\mathcal{B}_{11}$ simulates $G_{II.1}$. Therefore:

$$\Pr[G_{II.0}] - \Pr[G_{II.1}] \leq \text{Adv}_{\text{OKEM}}^{\text{SPR\$-CCA}}(\mathcal{B}_{11}).$$

Game II.2 (Random ES). In Game $G_{II.2}$, we replace ES with a uniformly random $\widetilde{ES}$ in the sessions created by the CHALLEXEC call. We bound the difference in this step by a reduction $\mathcal{B}_{12}$ to the swap-PRF security of F_2 , using its oracle instead of computing $F_2(\cdot, K_S)$. This yields

$$\Pr[G_{II.1}] - \Pr[G_{II.2}] \leq \text{Adv}_{F_2}^{\text{swap-PRF}}(\mathcal{B}_{12})$$

Game II.3 (Random ES' , encryption keys, and MAC tags). Next, in Game $G_{II.3}$, we replace any evaluations of F_1 keyed with $\widetilde{ES}$ with a random function in the sessions created by CHALLEXEC. This in particular replaces ES' , the encryption keys $\widetilde{EK}_1$ and $\widetilde{EK}_2$, and the MAC tags $\widetilde{MAC}_C$, $\widetilde{M}_C$, $\widetilde{MAC}_S$, and $\widetilde{M}_S$ with uniformly random values $\widetilde{ES}'$, $\widetilde{EK}_1$, $\widetilde{EK}_2$, $\widetilde{MAC}_C^*$, $\widetilde{M}_C^*$, $\widetilde{MAC}_S^*$, and $\widetilde{M}_S^*$, respectively. We again bound the difference in this by a reduction to the PRF security of F_1 :

$$\Pr[G_{II.2}] - \Pr[G_{II.3}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{13}).$$

Game II.4 (Random FS). In Game $G_{II.4}$, we replace FS with a uniformly random $\widetilde{FS}$ in the sessions created by the CHALLEXEC call, again bounded by PRF security of F_2 :

$$\Pr[G_{II.3}] - \Pr[G_{II.4}] \leq \text{Adv}_{F_2}^{\text{PRF}}(\mathcal{B}_{14}).$$

Game II.5 (Random $skey$ and $auth$). In Game $G_{II.5}$, we replace evaluations $F_1(\widetilde{FS}, \cdot)$ with a random function in the sessions created by the CHALLEXEC call. This in particular replaces $skey$ and $auth$ with uniformly random values $skey^*$ and $auth^*$, respectively. Bounding again by PRF security of F_1 :

$$\Pr[G_{II.4}] - \Pr[G_{II.5}] \leq \text{Adv}_{F_1}^{\text{PRF}}(\mathcal{B}_{15}).$$

Note that at this point, in the CHALLEXEC sessions, the ephemeral KEM key pair (sk_e, pk_e) , the ephemeral KEM ciphertext c_e , and the shared secret K_e are not used anymore, since the values FS , $skey$, and $auth$ computed from them are sampled at random as per

Games $G_{II.4}$ and $G_{II.5}$. We will leverage this to replace the ephemeral ciphertext and public key with random strings in the transcript in the final game hops.

Game II.6 (Random epk_e). In Game $G_{II.6}$, we replace $epk_e = \text{SE.Enc}(\widetilde{EK}_1, pk_e)$ in the sessions created by the CHALLEXEC call with a random bitstring epk_e^* of the same length. We can bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_{16}$ to the OT-IND\$ security of SE, since $\widetilde{EK}_1$ is random by $G_{II.3}$. We obtain

$$\Pr[G_{II.5}] - \Pr[G_{II.6}] \leq \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{16}).$$

Game II.7 (Random ect_e). Finally, Game $G_{II.7}$ replaces $ect_e = \text{SE.Enc}(\widetilde{EK}_2, c_e)$ in the sessions created by the CHALLEXEC call with a random bitstring ect_e^* of the same length. Similar to $G_{II.6}$, we bound the adversary $\mathcal{A}$'s difference in advantage by a reduction $\mathcal{B}_{17}$ to the OT-IND\$ security of SE, since $\widetilde{EK}_2$ is random by $G_{II.3}$:

$$\Pr[G_{II.6}] - \Pr[G_{II.7}] \leq \text{Adv}_{\text{SE}}^{\text{OT-IND\$}}(\mathcal{B}_{17}).$$

Now we have that the response to the CHALLEXEC oracle is distributed independent of the challenge bit b : In both cases, the returned transcript consists of random strings of length corresponding to the protocol messages and variable padding, exactly matching the output of the simulator $\mathcal{S}_{\text{Drive1}}$ (Section 4.3): epk_e is random by $G_{II.6}$, c_S by $G_{II.1}$, ect_e by $G_{II.7}$, P_C , P_S are random padding by definition, and M_C , MAC_C , M_S , MAC_S are replaced by uniformly random values (by $G_{II.3}$), as well as the *auth* tag and the session key *skey* (by $G_{II.5}$).

Hence $\mathcal{A}$ cannot win in this case anymore and we have

$$\text{Adv}_{\text{Drive1}}^{G_{II.7}}(\mathcal{A}) = 0.$$

Collecting the bounds yields the theorem statement. $\square$

G Changelog

Version 1.1 (August 16, 2025)

- Add analysis of hybrid strong second pseudorandomness for OEINC and mention its necessity for CAKE. Also discuss its relationship with implicit rejection.
- Change StatFrodoKEM to the IND-CCA variant rather than IND-CPA. Necessary for strong second pseudorandomness.
- Unify statistical strong ciphertext uniformity with statistical regular ciphertext uniformity. These are equivalent properties.
- Clarify requirements on SE in theorems.
- Clarify uniform distribution of challenge ciphertexts by using SPR\$-CCA notation.
- Simplify OEINC's reg-ctxt-unif theorem to use SPR\$-CCA of outOKEM rather than reg-ctxt-unif plus IND-CPA of outOKEM. These are equivalent properties.
- Correct statement that P-256 can use Elligator2. It requires Elligator².
- Correct miscellaneous typos.

Version 1.0 (March 3, 2025)

Initial version